

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Studiul și implementarea soluțiilor cloud scalabile pentru aplicații web

Lucrare de licență

Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Calculatoare și tehnologia informației*
programul de studii *Ingineria informației*

Conducător științific
Ș.L.Dr.Ing. George Valentin STOICA

Absolvent
Ionescu Maria Cătălina

Anul 2025

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Studiul și implementarea soluțiilor cloud scalabile pentru aplicații web*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Septembrie 2025.

Absolvent: Ionescu Maria Cătălina

.....

Cuprins

Lista figurilor	iii
Lista tabelelor	v
Lista acronimelor	vi
1. Introducere	1
1.1. Prezentarea proiectului	1
1.2. Scopul proiectului	1
1.3. Structura lucrării	2
2. Aspectele teoretice ale proiectului și tehnologii	3
2.1. Aplicația web	3
2.1.1. Arhitecturi comune	3
2.2. Protocolul HTTP	4
2.2.1. Componentele sistemelor bazate pe HTTP	4
2.2.2. Conexiunea HTTP	7
2.2.3. Mesaje HTTP	9
2.3. Framework-ul Flask Python	17
2.3.1. Prezentare generală	17
2.3.2. Rolul Flask în aplicație dezvoltată	17
2.4. Sistemul de Baze de Date MySQL	18
2.5. Nginx (Load Balancing și Reverse Proxy)	20
3. Arhitectura și Structura Aplicației	22
3.1. Introducere	22
3.2. Arhitectura Generală a Sistemului Distribuit	22
3.3. Arhitectura Bazei de Date	24
3.3.1. Tabela users	24
3.3.2. Tabela properties	24
3.3.3. Tabela rooms	25
3.3.4. Tabela reservations	26
3.3.5. Tabela reservation_details	26
3.3.6. Structura Tabelelor și Relațiile	27
3.3.7. Fluxul Datelor și Funcționarea Generală a Bazei de Date	27
3.3.8. Importanța Integrității Referențiale	27
3.4. Structura Internă a Aplicației Flask	30
4. Mecanisme de Comunicare și Securitate (Funcționalitățile Aplicației) . . .	31
4.1. Introducere în Mecanismul de Comunicare	31
4.2. Înregistrarea și Autentificare Utilizator	31
4.2.1. Formularul de înregistrare	31

4.2.2.	Formularul de autentificare	33
4.2.3.	Mecanismele de Securitate	34
4.2.4.	Mecanismul de Gestionare a Sesiunii	36
4.2.5.	Interacțiunea cu Baza de Date	36
4.3.	Pagina Principală (Homepage)	37
4.3.1.	Prezentarea Frontend (<code>home.html</code>)	37
4.3.2.	Logica Backend (Ruta / în <code>app.py</code>)	39
4.4.	Pagina de Profil: Dashboard-ul Personalizat al Utilizatorului	39
4.4.1.	Prezentarea Frontend (<code>profile.html</code>)	39
4.4.2.	Logica Backend (Ruta / <code>profile</code> în <code>app.py</code>)	40
4.5.	Modificarea Datelor de Profil	40
4.5.1.	Prezentarea Frontend (<code>edit-profile.html</code>)	40
4.5.2.	Logica Backend (Ruta / <code>edit-profile</code> în <code>app.py</code>)	41
4.6.	Panoul de Administrare	42
4.6.1.	Prezentarea Frontend (<code>admin.html</code>)	42
4.6.2.	Logica Backend (Ruta / <code>admin</code> în <code>app.py</code>)	43
4.7.	Fluxul de Căutare și Recomandări Personalizate	43
4.7.1.	Etapa 1: Căutarea Inițială (de pe Homepage)	44
4.7.2.	Etapa 2: Afișarea Rezultatelor (<code>search_results.html</code>)	44
4.7.3.	Etapa 3: Calculatorul de Disponibilitate și Recomandări (<code>view_property.html</code>)	45
4.7.4.	Algoritmul de Recomandare (<code>get_recommendations()</code>)	45
4.8.	Adăugarea unei Proprietăți Noi	46
4.8.1.	Prezentarea Frontend (<code>add_property.html</code>)	46
4.8.2.	Logica Backend (Ruta / <code>add_property</code> în <code>app.py</code>)	47
4.9.	Gestionarea Proprietăților Personale	48
4.9.1.	Prezentarea Frontend (<code>my_properties.html</code>)	48
4.9.2.	Logica Backend (Ruta / <code>my-properties</code> în <code>app.py</code>)	49
4.10.	Modificarea unei Proprietăți Existente	49
4.10.1.	Prezentarea Frontend (<code>edit_property.html</code>)	49
4.10.2.	Logica Backend (Rutele / <code>edit_property</code> și / <code>check_room_type</code> ...) . . .	50
4.11.	Ștergerea unei Proprietăți	51
4.11.1.	Interfața Frontend (<code>my_properties.html</code>)	51
4.11.2.	Logica Backend (Ruta / <code>delete_property/<id></code> în <code>app.py</code>)	51
4.12.	Procesul de Rezervare Efectivă	52
4.13.	Confirmarea și Finalizarea Rezervării (/booking/confirm)	53
4.13.1.	Prezentarea Frontend (<code>booking_confirmation.html</code>)	53
4.13.2.	Logica Backend (/booking/confirm în <code>app.py</code>)	53
4.14.	Pagina de Confirmare a Rezervării (/booking/success)	54
4.14.1.	Prezentarea Frontend (<code>reservation_success.html</code>)	54
4.14.2.	Logica Backend (/booking/success în <code>app.py</code>)	54
4.15.	Vizualizarea și Gestionarea Rezervărilor	55
4.15.1.	Prezentarea Frontend (<code>my_reservations.html</code>)	55
4.15.2.	Logica Backend (/my-reservations în <code>app.py</code>)	56
4.16.	Anularea Rezervării (/cancel_reservation/<int:reservation_id>)	56
4.16.1.	Logica Backend (/cancel_reservation/<int:reservation_id> în <code>app.py</code>)	56
4.17.	Gestionarea Rezervărilor Proprietăților Personale	57
4.17.1.	Prezentarea Frontend (<code>manage_property_reservations.html</code>)	57

4.17.2. Logica Backend (/manage_property_reservations/<int:property_id> în app.py)	58
5. Arhitectura de Suport și Înaltă Disponibilitate	59
5.1. Rolul Nginx în contextul arhitecturii implementate	59
5.2. Rolul Replicării MySQL (Master-Slave) în Contextul Fluxurilor Reale	60
5.3. Rolul Infrastructurii de Virtualizare în Testarea Arhitecturii	61
6. Testarea Performanței și Analiza Scalabilității	63
6.1. Alegerea Instrumentului de Testare: Locust	63
6.2. Procesul de Execuție și Monitorizare	64
6.3. Arhitectura Mediului de Testare	65
6.3.0.1. Pregătirea și Rularea Testului Locust	66
6.4. Scenarii de Testare Implementate	66
6.4.1. Cazul 1: Aplicația Flask Monolitică (Fără Scalare)	66
6.4.2. Cazul 2: Aplicația Flask Aplicația Flask (Load Balancing)	71
6.5. Monolitic versus Load Balancing	74
6.6. Concluzii comparative	76
7. Concluzii și Direcții Viitoare	81
7.1. Concluzii	81
7.2. Direcții Viitoare de Dezvoltare	81
8. Încheiere	83
Bibliografie	84
Anexa A. Coduri sursă	86

Lista figurilor

2.1. Structura user-agent-ului	5
2.2. Componentele URL-ului	5
2.3. Tipul de protocol URL	6
2.4. Numele resursei și portul	6
2.5. Calea către resursă	6
2.6. Parametrii	7
2.7. Ancoră URL	7
2.8. Diagrama modului de funcționare a server-ului web	8
2.9. Diagrama conexiunii HTTP	8
2.10. Cerere HTTP	9
2.11. Răspuns HTTP	9
2.12. Răspuns HTTP	10
2.13. Exemplu de cerere HTTP	11
2.14. Structura cererii HTTP	11
2.15. Structura unei linii de cerere HTTP	11
2.16. Forma de origine	12
2.17. Forma absolută	13
2.18. Forma de autoritate	13
2.19. Forma asterisk (*)	13
2.20. Header request	13
2.21. Date din corp sub formă de key=values	14
2.22. Date din corp împărțite în mai multe componente	14
2.23. Exemplu de răspuns HTTP în care un nou utilizator este creat	15
2.24. Structura răspunsului HTTP	15
2.25. Status line - alcătuire	16
2.26. Exemplu de response și representation header	16
3.1. Diagrama arhitecturală a componentelor sistemului distribuit	23
3.2. Structura bazei de date	29
4.1. Formularul de înregistrare a utilizatorilor	32
4.2. Formularul de autentificare a utilizatorilor	33
4.3. Pagina home - utilizator neautentificat	38
4.4. Pagina home - utilizator autentificat	38
4.5. Interfața paginii de profil, funcționând ca un panou de control.	40
4.6. Pagina Editează Profilul	41
4.7. Panou Administrator	43
4.8. Formularul de autentificare a utilizatorilor	44
4.9. Rezultatele căutării	44
4.10. Formularul de autentificare a utilizatorilor	45
4.11. Formularul de autentificare a utilizatorilor	46
4.12. Formularul de autentificare a utilizatorilor	47

4.13. Formularul de autentificare a utilizatorilor	48
4.14. Formularul de autentificare a utilizatorilor	49
4.15. Formularul de autentificare a utilizatorilor	50
4.16. Lista cu proprietățile căutate	52
4.17. Pagina de vizualizare a unei proprietăți	53
4.18. Formular de rezervare	54
4.19. Formularul de autentificare a utilizatorilor	55
4.20. Pagina de administrare a rezervărilor	56
4.21. Pagina cu gestionările rezervărilor propriilor proprietăți	57
6.1. Total Requests per Second	67
6.2. Total Requests per Second	68
6.3. Total Requests per Second	68
6.4. Total Requests per Second	69
6.5. Total Requests per Second	70
6.6. Total Requests per Second	70
6.7. Total Requests per Second	70
6.8. Total Requests per Second	73
6.9. Utilizarea resurselor sistemului	74
6.10. Utilizarea resurselor sistemului după 15 minute de testare	75
6.11. Utilizarea resurselor sistemului după 15 minute de testare	76
6.12. Total Requests per Second după 15 minute	77
6.13. Timpii de răspuns după 15 minute	77
6.14. Utilizarea resurselor după 15 minute de testare	78

Lista tabelelor

6.1. Timp de răspuns agregat pentru toate cererile.	71
6.2. Timp de răspuns pentru endpoint-ul POST /login.	71
6.3. Timp de răspuns pentru endpoint-ul POST /edit-profile.	72
6.4. Timp de răspuns pentru GET /my-reservations.	72
6.5. Compararea performanței generale și a endpoint-urilor pentru două seturi de teste.	79
6.6. Configurarea mediului în cele două cazuri de testare.	79
6.7. Timp de răspuns agregat pentru toate cererile.	80
6.8. Timp de răspuns pentru endpoint-ul POST /login.	80
6.9. Timp de răspuns pentru endpoint-ul POST /edit-profile.	80
6.10. Timp de răspuns pentru endpoint-ul GET /my-reservations.	80

Lista acronimelor

HTML = HyperText Markup Language HTTP = Hypertext Transfer Protocol IoT = Internet of Things VM1 = Mașină virtuală 1 VM2 = Mașină virtuală 2

Capitolul 1

Introducere

1.1 Prezentarea proiectului

Cerințele privind performanța aplicațiilor web sunt în continuă creștere datorită gamei extinse de dispozitive inteligente și IoT (Internet of Things) folosite de către utilizatori pentru a-și îmbunătăți viața de zi cu zi. Numărul mare de dispozitive care accesează în mod concomitent un site web duce la un volum copleșitor de cereri către servere pentru a accesa informația cerută, ceea ce poate cauza o experiență întârziată sau chiar imposibilă a utilizatorului. Datorită acestor circumstanțe, traficul de date crește, iar aplicațiile web trebuie să facă față la un flux considerabil de informații. La momentul actual, o soluție pentru optimizarea aplicațiilor web expuse unui trafic mare de date, dar și pentru a spori satisfacția utilizatorilor este folosirea soluțiilor scalabile a resurselor software și hardware.

Scalabilitatea [1] reprezintă capacitatea unui sistem de a-și folosi în mod adaptiv și utiliza eficient resursele în scopul creșterii performanței și reducerii costurilor, în funcție de cerințele de procesare. Aceasta se clasifică după tipul de resurse folosite în:

1. Scalabilitatea resurselor software [2]: se referă la abilitatea unei aplicații de a-și adapta sau menține performanța atunci când este o creștere sau o scădere bruscă a volumului de utilizatori, date sau operațiuni.
2. Scalabilitatea resurselor hardware [3]: reprezintă capacitatea unui sistem de a-și mări capacitatea sau de a-și crește performanța utilizând noi resurse hardware.

De asemenea, folosirea în paralel a unei bazei de date distribuită reprezintă o soluție eficientă pentru asigurarea redundanței datelor și a performanței optime într-o aplicație complexă.

O bază de date distribuită [4] este o bază de date controlată de un sistem de gestiune a bazelor de date (Data Base Management System), în care datele sunt distribuite la mai multe calculatoare interconectate.

1.2 Scopul proiectului

Proiectul „**Studiul și implementarea soluțiilor cloud scalabile pentru aplicații web**” își propune studierea, implementarea și testarea soluțiilor scalabile software pentru o aplicație web, utilizând framework-ul Flask Python (pentru dezvoltarea aplicației web), MySQL (sistem de gestionare a bazelor de date relaționale) și Nginx (loadbalancer și server web de tip reverse proxy).

În mod ideal, scalabilitatea presupune gestionarea optimă a resurselor hardware și software, astfel încât atunci când aplicația este expusă asupra unui număr mare de cereri, ea va trebui să le gestioneze eficient, fără a compromite viteza de răspuns sau epuiza toate resursele disponibile. Scalabilitatea software [5] reprezintă implementarea unor soluțiilor de tip software care asigură funcționarea eficientă a aplicației la un volum de utilizatori brusc variabil. Aceasta folosită concomitent cu scalabilitatea hardware, are rol în extinderea resurselor fizice ale

unui sistem, poate aduce la rezultate impresionante în gestionarea traficului și a numărului de cereri semnificativ pentru a oferi utilizatorilor o experiență cât mai fluidă.

În practică, oricât de mult am încerca să avem o viteză de răspuns cât mai optimă, aceasta implică un cost foarte mare al resurselor software, hardware dar și a energiei electrice folosite. Un volum limitat de resurse hardware și software implică, de asemenea, o eficiență de gestionare a cererilor din cadrul unei aplicații web mai scăzute. Astfel, este necesară o arhitectură software și hardware bine definită pentru a reuși în atingerea unor rezultate promițătoare cu un număr minim de resurse folosite.

Scopul acestui proiect este a urmări și analiza capabilitatea aplicației de a răspunde la un număr mare de cereri simultane, îmbunătățirea performanței și reducerea redundanței prin implementarea unei soluții scalabile de tip software care include un load balancer și o bază de date distribuită master-slave.

Aplicația, o platformă de tip web destinată rezervărilor, a fost dezvoltată într-un mediu de lucru alcătuit dintr-o stație de lucru, alături de un mediu de testare, constituit de cele două mașini virtuale instalate pe stația de lucru. Împreună, cele trei mașini constituie o rețea IP/LAN privată. De asemenea, la nivel arhitectural s-a implementat o bază de date distribuită de tip master-slave pentru a optimiza operațiile de citire-scriere.

Obiectivele specifice acestei lucrări sunt:

- Dezvoltarea aplicației web folosind framework-ul Flask Python care include rute HTTP pentru funcționalități de bază precum ar fi: autentificarea (login, register), managementul utilizatorilor (profile), administrarea rezervărilor (reserve) și pagini administrative (admin).
- Implementarea arhitecturii scalabile prin intermediul sistemului de load balancing folosind Nginx. Nginx are rolul de a distribui cererile trimise între cele două servere disponibile, aplicația Flask și VM2, rezultând în creșterea performanței și puterii de procesare a aplicației web.
- Optimizarea performanței bazei de date cu un server master pe stația de lucru și un server slave pe VM1. Cele două servere îmbunătățesc operațiile de citire și scriere și asigură redundanța datelor prin atribuirea operațiilor de citire serverului master, iar operațiile de scriere serverului slave.

Semnificația practică a acestei lucrări constă în evidențierea rezultatelor obținute în urma aplicării soluțiilor scalabile implementate pentru a facilita o performanță crescută folosind un număr de resurse limitate în gestionarea eficientă a cererilor utilizatorilor. Rezultatele oferă o perspectivă clară asupra comportamentului unei aplicații web atunci când este supusă la un volum semnificativ de cereri, dar și asupra faptului cum pot fi îmbunătățite performanța și costurile de operare .

1.3 Structura lucrării

Capitolele următoare vor prezenta principalele concepte și soluții abordate în cadrul acestei lucrări, cum ar fi: aspectele teoretice ale proiectului, tehnologiile folosite în implementare și structura aplicației. Acestea sunt urmate de descrierea arhitecturii, în care sunt evidențiate diferitele module ce constituie aplicația și interacțiunea dintre ele la nivel scalabil. Capitolele vor cuprinde și exemplificări cu fragmente de cod care un rol crucial în aplicarea fenomenului de scalabilitate asupra aplicației web. De asemenea, lucrarea abordează explicații amănunțite despre modul de funcționare și utilizare a interfeței grafice a aplicației cu ajutorul capturilor de ecran.

Capitolul 2

Aspectele teoretice ale proiectului și tehnologii

2.1 Aplicația web

O aplicație web [6] este o aplicație de tip software accesată prin intermediul unui browser web care poate rula pe un server local sau cloud. Aceasta poate fi accesată de pe orice tip de dispozitiv conectat la internet cum ar fi calculatoarele, telefoanele, consolele ș.a.m.d. și este implementată prin intermediul unor limbaje de programare web ca HTML, CSS și JavaScript. Ceea ce deosebește o aplicație web față de cele mobile sau desktop este faptul că poate să ruleze pe unul sau pe mai multe servere și este nevoie doar de un browser web pentru a o accesa.

Principalele caracteristici și funcționalități ale unei aplicații web sunt:

- Scalabilitatea: datorită faptului că aplicațiile web rulează pe un server există posibilitatea de a implementa soluții scalabile care să îmbunătățească și optimizeze performanța pentru a răspunde la cerințele utilizatorilor.
- Accesibilitatea: oricine are un dispozitiv cu conexiune la internet poate să acceseze aplicațiile web, indiferent de locație sau specificațiile hardware și software pe care le are dispozitivul folosit.
- Integrarea cu alte servicii și aplicații: aplicații web fiind versatile, programatorii au posibilitatea de a rafina experiența utilizatorilor cu ajutorul serviciilor și aplicațiilor web suplimentare concepute pentru a personaliza experiența fiecărei persoane.
- Actualizarea eficientă: odată ce s-a confirmat de către echipa de testare faptul că noua versiune a aplicației web a trecut toate testele și îndeplinește toate cerințele, aceasta poate fi lansată către public fără a fi necesară intervenția din partea lui, actualizarea fiind automată pe server.

2.1.1 Arhitecturi comune

Dezvoltarea aplicațiilor web moderne a condus la adoptarea unor diverse arhitecturi software, fiecare cu avantaje și dezavantaje specifice în contextul cerințelor de performanță, scalabilitate și mentenanță. Printre cele mai răspândite se numără arhitectura monolitică, arhitectura bazată pe microservicii și arhitectura N-Tier (sau multi-strat).

Arhitectura monolitică

Acest model arhitectural [7] presupune construirea unei aplicații ca o singură unitate logică și indivizibilă, în care toate componentele funcționale – interfața cu utilizatorul, logica de business, stratul de acces la date și alte module – sunt integrate într-un singur bloc de cod și rulate ca un proces unic. Deși simplifică dezvoltarea inițială și implementarea pentru aplicații de dimensiuni mici și medii, arhitectura monolitică prezintă provocări semnificative în contextul scalabilității și mentenanței pe termen lung. Scalarea necesită replicarea întregii aplicații, chiar dacă doar o componentă specifică este suprasolicitată, ducând la ineficiență în utilizarea resurselor.

Arhitectura bazată pe microservicii

Reprezintă o abordare modernă și dinamică, în care o aplicație complexă este descompusă într-o colecție de servicii mici, independente, autonome și slab cuplate. Fiecare microserviciu rulează într-un proces propriu și este responsabil pentru o funcționalitate de business specifică, comunicând cu alte servicii prin intermediul unor API-uri bine definite. Avantajele notabile ale acestei arhitecturi includ scalabilitatea independentă a fiecărui serviciu, reziliența sporită (o defecțiune într-un serviciu nu afectează întregul sistem) și flexibilitatea în utilizarea diferitelor tehnologii pentru fiecare componentă. Cu toate acestea, introducerea microserviciilor aduce o complexitate crescută în ceea ce privește gestionarea, monitorizarea și comunicarea distribuită între servicii, reprezentând o direcție viitoare de dezvoltare pentru aplicații la scară largă.

Arhitectura N-Tier (Multi-Tier)

Această arhitectură logică [8] organizează funcționalitățile unei aplicații în straturi distincte, fiecare cu un rol specific și responsabilități separate. Modelul cel mai răspândit este cel cu trei straturi (3-tier), care include:

- **Stratul de Prezentare (Presentation Tier):** Responsabil pentru interfața cu utilizatorul (frontend), gestionează afișarea informațiilor și colectarea datelor de intrare. Este adesea implementat folosind tehnologii web precum HTML, CSS și JavaScript.
- **Stratul de Logică de Business / Aplicație (Application/Logic Tier):** Găzduiește logica principală a aplicației, procesează cererile de la stratul de prezentare, implementează regulile de business și coordonează interacțiunile cu stratul de date. În contextul acestei lucrări, framework-ul Flask Python operează la acest nivel.
- **Stratul de Date (Data Tier):** Asigură persistența datelor, stocarea și gestionarea informațiilor critice ale aplicației. Este reprezentat, în general, de un sistem de gestiune a bazelor de date relaționale, cum ar fi MySQL.

Avantajele arhitecturii N-Tier rezidă în modularitatea crescută, separarea clară a responsabilităților și posibilitatea de a scala independent fiecare strat. În cadrul proiectului dezvoltat, o astfel de arhitectură N-Tier a fost proiectată și implementată, cu stratul de Logică de Business extins ulterior într-o arhitectură distribuită multi-nod, prin integrarea soluțiilor de load balancing și replicare a bazelor de date.

2.2 Protocolul HTTP

Protocolul HTTP (Hypertext Transfer Protocol) [9] este un protocol la nivelul stratului de aplicație, fără reținerea stării (serverul nu reține date despre sesiunea activă), de tip client-server care stă la baza oricărui schimb de date între un client și un server pentru preluarea unor resurse disponibile pe internet, cum ar fi documentele HTML (HyperText Markup Language), fișiere audio, video ș.a.m.d.

2.2.1 Componentele sistemelor bazate pe HTTP

Clientul: user-agent-ul

User-agent-ul [10] este un header de tip string folosit într-o cerere HTTP pentru a trimite informații despre aplicație, sistemul de operare, furnizorul și/sau versiunea folosită către server.

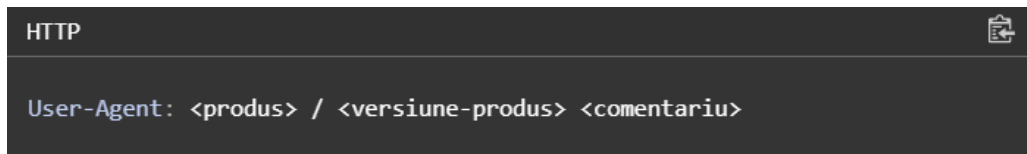


Figura 2.1: Structura user-agent-ului

Ca să afișeze o pagină web, în primă fază, browser-ul trimite o cerere inițială pentru a prelua documentul HTML solicitat de către client. În cea de-a doua fază, acesta analizează fișierul primit și trimite cereri suplimentare pentru a obține informații despre stilizarea paginii (CSS), eventuale scripturi ce trebuie executate și sub-resursele aflate în pagină (imagini sau videoclipuri). În fază finală a procesului, browser-ul combină toate aceste resurse pentru a afișa varinta finală a documentului.

Server-ul web

Server-ul web [11] se ocupă cu livrarea resurselor cerute de către client. Din punct de vedere al structurii hardware, acesta poate fi reprezentat de către o stație de lucru individuală sau de o colecție de servere interconectate în aceeași rețea care au rol în a gestiona schimbul de date stocate (de exemplu: documente HTML, imagini, fișiere JavaScript) cu alte dispozitive.

Pe de altă parte, din punct de vedere al structurii software, un server web are rolul de a interpreta adresele URL și de a procesa cererile și răspunsurile transmise prin protocolul HTTP.

Componente URL

URL (Uniform Resource Locators) [12] reprezintă adresa unei resurse unice de pe internet și unul dintre mecanismele folosite de browsere pentru a prelua resurse cum ar fi pagini HTML, documente, imagini ș.a.m.d. Orice URL introdus în bara de adrese trimite o solicitare directă către un server web pentru a accesa resursa specifică, iar în funcție de răspunsul serverului, resursa solicitată fie este afișată, fie un mesaj de eroare este returnat. Conform figurii 2.8, un URL este alcătuit din mai multe componente cum ar fi: tipul de protocol URL, numele resursei, portul, calea către resursă, parametrii și fragmentul de identificare internă.

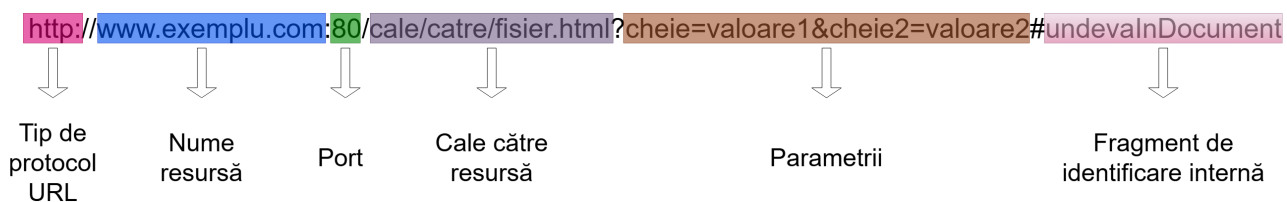


Figura 2.2: Componentele URL-ului

Tipul de protocol URL

Prima componentă regăsită într-un URL reprezintă tipul de protocol pe care browser-ul trebuie să îl folosească pentru a trimite o cerere de acces la resursă. Protocelele cele mai des întâlnite sunt HTTPS (Hypertext Transfer Protocol Secured) sau HTTP (variantea nesecurizată), dar există cazuri în care este folosit și protocolul mailto: care deschide un client de tip mail.



Figura 2.3: Tipul de protocol URL

Numele resursei și portul

Cea de-a doua componentă a URL-ului este formată din numele resursei și port. Numele resursei sau numele domeniului, separat de tipul de protocol prin șirul de caractere `://`, reprezintă adresa paginii web redactată sub forma unui șir de caractere. Orice pagină web are atribuit un IP public unic din intervalul 1.0.0.0 - 9.255.255.255, numere care sunt greu de reținut sau scris pentru orice persoană. Din acest motiv, cu ajutorul numele domeniului, procesul de accesare a unei pagini web devine mai rapid pentru utilizatori prin „traducerea în limbaj natural” a adreselor IP într-un format ușor de înțeles și accesibil pentru oricine. Portul este o „poartă” tehnică folosită pentru a accesa resursele pe serverul web. În cele mai multe cazuri, declararea portului este omisă deoarece majoritatea serverelor folosesc în mod implicit portul 80 pentru conexiunea HTTP și 443 pentru conexiunea HTTPS, dar în situațiile în care este necesară utilizarea unui alt port, acesta trebuie declarat în mod obligatoriu.

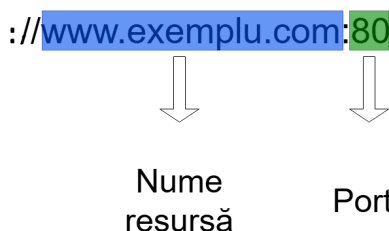


Figura 2.4: Numele resursei și portul

Calea către resursă

Următoarea componentă a unui URL este calea către resursă. Calea indică locația exactă a unei pagini, imagini sau altei resurse aflate pe server.

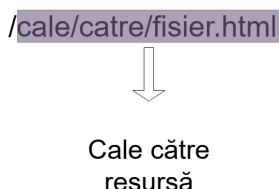


Figura 2.5: Calea către resursă

Parametrii

Cea de-a patra componentă constă într-un atribut unic, denumit parametrii. Parametrii sunt o listă de chei și valori folosite, delimitate de simbolul `&` de către serverul web utilizat

pentru transmiterea de informații suplimentare către server. Fiecare server web are o listă de chei sau de valori specifice, nu există o regulă specifică în crearea acestora.

`?cheie=valoare1&cheie2=valoare2`



Parametrii

Figura 2.6: Parametrii

Fragment de identificare internă

Ultima componentă ce intră în alcătuirea unui URL se numește fragment de identificare internă. Un fragment de identificare internă sau ancoră, delimitată de simbolul #, are rol în afișarea unei anumite secțiuni din resursă. Ancorele funcționează ca un fel de „semn de carte”, acestea indicând aplicației web până unde va trebui să parcurgă documentul ca să ajungă la secțiunea marcată și într-un final să o afișeze.

`#undevalnDocument`



Fragment de
identificare internă

Figura 2.7: Ancoră URL

2.2.2 Conexiunea HTTP

Accesarea directă a unui server HTTP se realizează prin introducerea URL-ului în bara de adrese, acesta livrând conținutul solicitat utilizatorilor. La nivel fundamental, modul de funcționare presupune ca de fiecare dată când un browser are nevoie de un fișier stocat pe un server web, acesta va iniția o cerere HTTP pentru a primi fișierul respectiv. Odată ce serverul HTTP a recepționat cererea, verifică dacă URL-ul solicitat există. În situația în care resursa este disponibilă, serverul trimite utilizatorilor conținutul corespunzător. În cazul contrar, serverul mai întâi acesta verifică dacă este necesară generarea unui fișier dinamic pentru a răspunde solicitării. În cele din urmă, dacă niciuna dintre opțiuni nu este disponibilă, serverul va returna codul de eroare 404 Not Found.

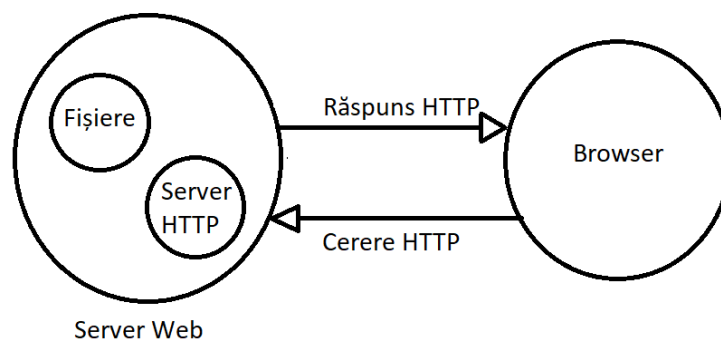


Figura 2.8: Diagrama modului de funcționare a server-ului web

Cum funcționează o conexiune HTTP

O conexiune HTTP este controlată la nivelul stratului de transport, ceea ce implică faptul că se va folosi un protocol de transport pentru gestionarea conexiunii între client și server. Pentru a transmite mesaje cu o rată de pierdere foarte mică s-a optat în folosirea protocolului TCP. Datorită fiabilității sale, se utilizează protocolul TCP care garantează trimiterea mesajelor cu un număr minim erori, cu prețul unei viteze de transfer mai mici.

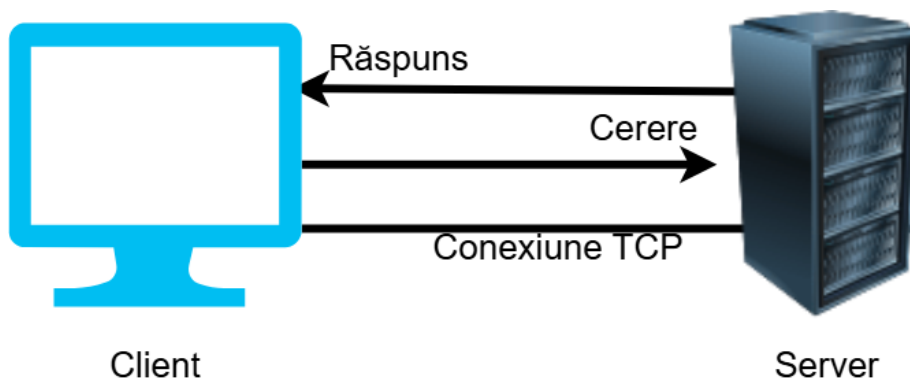


Figura 2.9: Diagrama conexiunii HTTP

Primul pas spre a începe o conexiune HTTP îl reprezintă schimbul de cerere/răspuns dintre client și server. Clientul are posibilitatea de a deschide o nouă conexiune, de a folosi o conexiune existentă sau de a deschide mai multe conexiuni TCP către servere. Acest schimb începe odată ce s-a stabilit o conexiune TCP, un proces care trimite unul sau mai multe cereri din partea clientului pentru a primi un răspuns de la server.

Al doilea pas presupune trimiterea mesajului HTTP către server și răspunsul serverului la cererea clientului cu informația cerută. Dacă serverul răspunde cu codul 200 OK, înseamnă că s-a stabilit cu succes conexiunea, iar serverul poate transmite clientului informația cerută.

```

HTTP

GET / HTTP/1.1
Host: localhost
Accept-Language: en

```

Figura 2.10: Cerere HTTP

```

HTTP

HTTP/1.1 200 OK
Date: Sat, 09 Oct 2025 14:28:02 GMT
Server: Apache
Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT
ETag: "51142bc1-7449-479b075b2891b"
Accept-Ranges: bytes
Content-Length: 29769
Content-Type: text/html

<!doctype html>... (conținutul paginii cerute)

```

Figura 2.11: Răspuns HTTP

În cele din urmă, odată ce s-a făcut schimbul de resurse, există posibilitatea de a închide sau reutiliza conexiunea pentru viitoare sesiuni.

2.2.3 Mesaje HTTP

Mesajele HTTP sunt șiruri de caractere scrise în limbaj natural, simplist, începând cu versiunea HTTP/1.1 până la versiunea HTTP/2. Acestea sunt clasificate în două categorii: cerere (request) și răspuns (response), reprezentând un mecanism de schimb de date între server și un client. O cerere este trimisă de către client serverului-ui cu scopul în a declanșa o acțiune, iar răspunsul are rolul de a furniza informația solicită înapoi către client. Crearea și modificarea mesajelor HTTP se datorează API-urilor disponibile în browser, fișierelor de configurație ale proxy-urilor sau serverelor sau altor interfețe software dedicate.

Structura mesajelor HTTP

Conform figurei 2.12 se poate observa faptul că mesajele de tip cerere și răspuns prezintă o structură asemănătoare. Partea superioară (head) a mesajului HTTP este alcătuită din linia de start și anteturi, iar corpul, deși conține date transmise între client și server, este o componentă opțională.

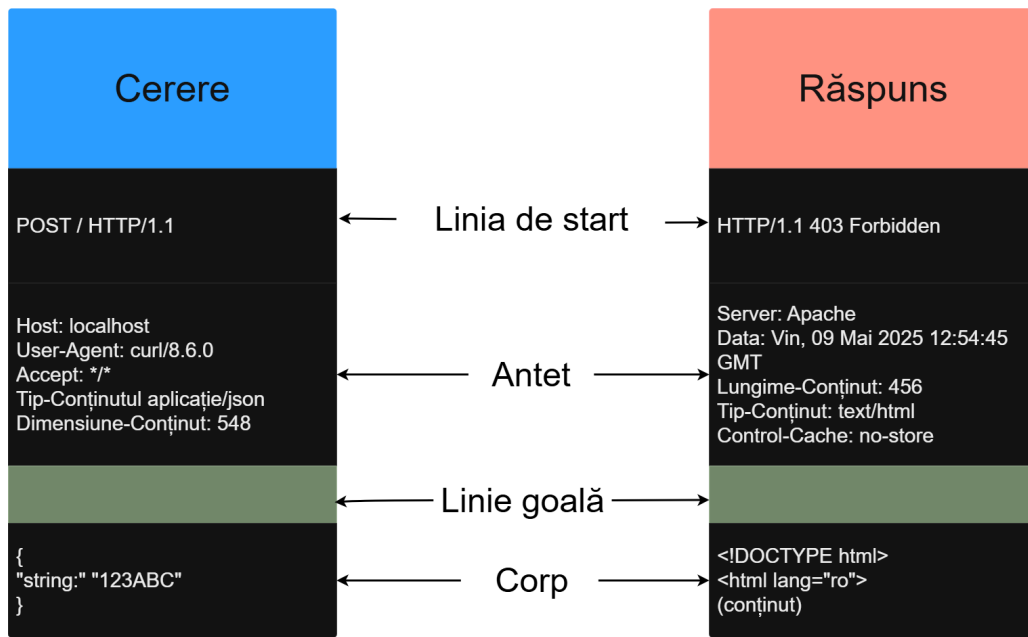


Figura 2.12: Răspuns HTTP

Partea superioară conține informații esențiale pentru interpretarea mesajului cum ar fi:

1. Linia de start, compusă dintr-un singur rând, precizează versiunea HTTP, precum și metoda cererii sau codul de răspuns.
2. Antetul (header) conține metadata folosite pentru a descrie mesajul transmis și facilitează schimbul de informații suplimentare dintre un client și un server, fie în cadrul unei cereri, fie al unui răspuns. În funcție de rolul pe care îl îndeplinesc, se remarcă mai multe categorii de antet:
 - Antet de cerere (request header): conține informații despre resursa ce urmează a fi preluată.
 - Antet de răspuns (response header): oferă informații despre răspuns, precum ar fi locația sau serverul de pe care provine.
 - Antet de reprezentare (representation header): cuprinde informații despre corpul resursei, precum ar fi tipul de conținut sau de compresie sau codare.
 - Antet cu conținut util (payload header): include informații referitoare la datele din corpul mesajului (payload) ce descriu caracteristici ale mesajului, fără a depinde de modul în care acesta este procesat sau trimis .
3. O linie goală care indică faptul că metadata mesajului este complet.

Un corp (body) pune la dispoziție datele asociate mesajului, transmiterea căruia este stabilită încă de la linia de start și din anteturile anterioare. De obicei corpul include fie o metodă trimisă de client către server, fie o resursă redirectionată de către server clientului.

Cereri HTTP

```

HTTP

POST /user HTTP/1.1
Host: exemplu.ro
Content-Type: application/x-www-form-urlencoded
Content-Length: 78

nume=nume%20&prenume&email=licenta%40exemplu.ro

```

Figura 2.13: Exemplu de cerere HTTP

O cerere HTTP [13] este un mesaj trimis de client către server cu scop de a solicita o resursă. Structura acesteia este alcătuită dintr-o parte superioară ce cuprinde linia cererii (request line), antet (header), urmată de o linie goală și opțional, un corp.

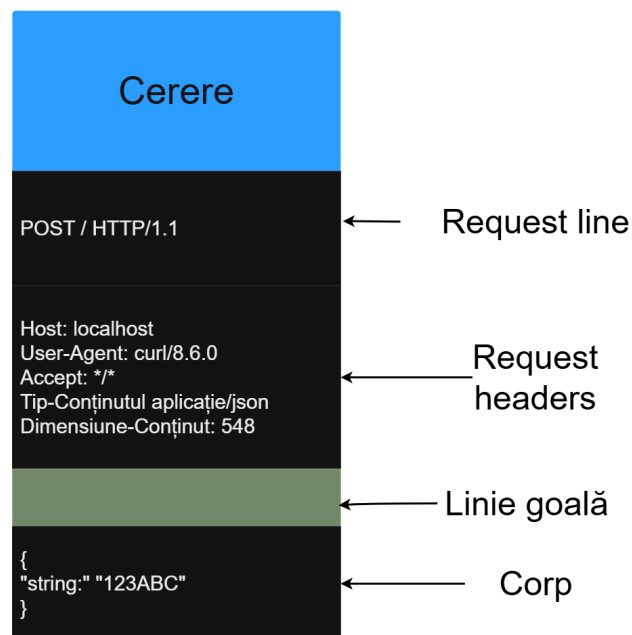


Figura 2.14: Structura cererii HTTP

Structura părții superioare a unei cereri HTTP este următoarea:

1. Linia cererii: este alcătuită din trei elemente precum ar fi metoda, resursa solicitată și protocolul.

```

HTTP

<metodă> <resursa-solicitată> <protocol>

```

Figura 2.15: Structura unei linii de cerere HTTP

Metoda HTTP

O metodă HTTP indică scopul unei cererii trimise către un server și rezultatul așteptat în cazul în care cererea a fost realizată cu succes. De obicei fiecare metodă are propriul său rol, dar pot împărtăși anumite caracteristici comune. Există mai multe tipuri de metode precum ar fi:

- **GET** este folosită pentru doar a prelua date de pe server, fără ca acestea să sufere noi modificări.
- **HEAD** se comportă în același mod ca și metoda GET, dar fără a include conținut în răspuns.
- **POST** are rol în a trimite date către server, de exemplu încărcarea unui fișier.
- **PUT** înlocuiește toate instanțele curente cu noi instanțe specificate de către client.
- **DELETE** este utilizată cu scopul de a solicita ștergerea unei resurse specificate.
- **CONNECT** crează un tunel de comunicație cu serverul identificat prin resursa țintă.
- **OPTIONS** are rol în a descrie opțiunile de comunicare posibile pentru resursa specificată.
- **TRACE** este folosită pentru efectuarea unui test de buclă (loop-back) a mesajului de-a lungul traseului către resursa specificată.
- **PATCH** aplică modificări parțiale unei resurse.

Resursa solicitată

Resursa solicitată este un URI (Uniform Resource Identifier) care identifică resursa căreia i se trimite cererea. Există mai multe tipuri de formatare utilizate pentru a descrie o resursă solicitată:

- Forma de origine, utilizată cu metodele GET, POST, HEAD și OPTIONS, este cea mai folosită formatare pentru a identifica originea resursei dintr-un server sau gateway. Presupune combinarea unei căi absolute cu informațiile aflate în antetul „Host” la care există posibilitatea adăugării unei interogări de forma *cheie = valoare* pentru a transmite informații suplimentare.

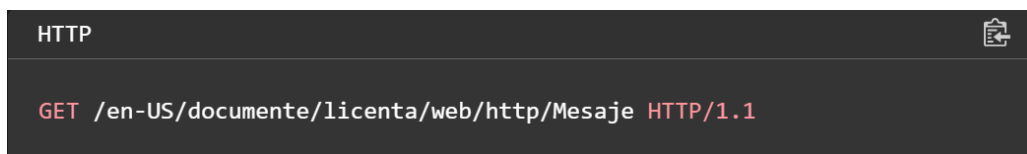
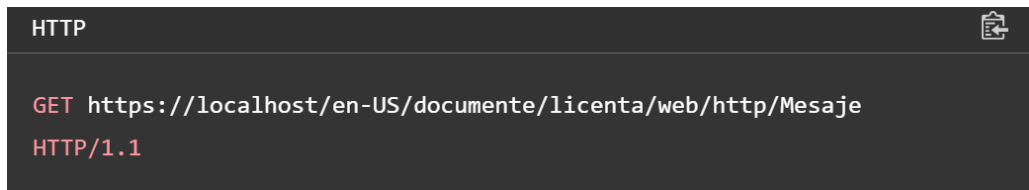


Figura 2.16: Forma de origine

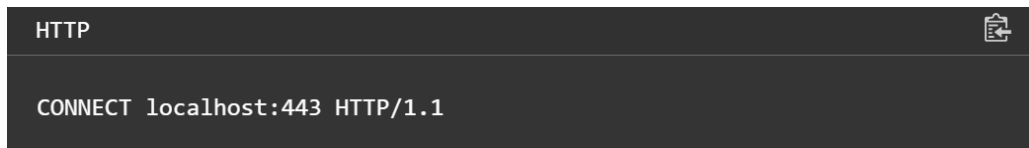
- Forma absolută este o sintaxă de tip URL complet ce include informațiile necesare pentru identificarea serverului. Atunci când cererea HTTP este făcută către un proxy, forma absolută are rol în redirectionarea răspunsului sau serviciul de la un cache valid înapoi către client, utilizând metoda GET.



```
HTTP
GET https://localhost/en-US/documente/licenta/web/http/Mesaje
HTTP/1.1
```

Figura 2.17: Forma absolută

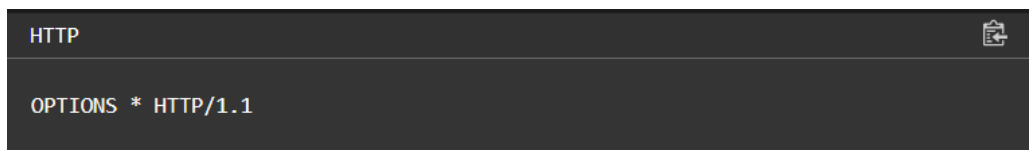
- Forma de autoritate este o structură folosită în cererile HTTP formată dintr-un nume de gazdă și opțional un port delimitat de simbolul : împreună cu metoda CONNECTION pentru a seta un tunel către resursa țintă.



```
HTTP
CONNECT localhost:443 HTTP/1.1
```

Figura 2.18: Forma de autoritate

- Forma asterisk (*) este folosită alături de metoda OPTIONS atunci când cererea HTTP nu se aplică unei resurse particulare, ci serverul-ui ca un întreg.



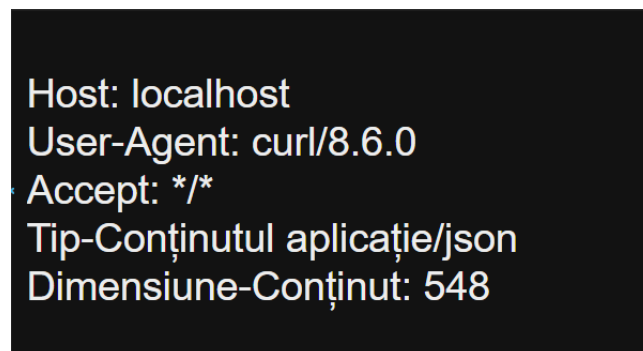
```
HTTP
OPTIONS * HTTP/1.1
```

Figura 2.19: Forma asterisk (*)

Protocol

Protocolul este un indicator care reprezintă versiunea protocolului HTTP utilizată în timpul procesului de trimitere a cererii folosit având rol în interpretarea corectă a mesajelor transmise. Singura versiune care trebuie declarată în acest câmp este HTTP/1.1, deoarece versiunile HTTP/0.9, HTTP/1.0 sunt învechite și nu se mai folosesc, iar versiunile mai noi de HTTP/2.0 sunt configurate în timpul conexiunii.

2. Antetul (header) conține metadata despre mesajul ce urmează a fi transmis și se distinge în mai multe tipuri:

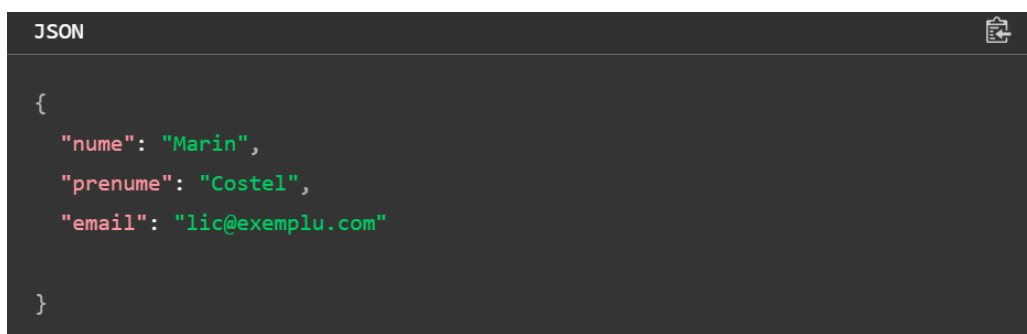


```
Host: localhost
User-Agent: curl/8.6.0
Accept: */*
Tip-Conținutul aplicație/json
Dimensiune-Conținut: 548
```

Figura 2.20: Header request

- Antet de cerere (request header) furnizează context suplimentar în cadrul unei cereri sau adaugă instrucțiuni despre cum ar trebui să fie un mesaj tratat de către server
- Antet de reprezentare (representation header) este folosit numai în cazul în care cererea prezintă și o parte superioară. Un representation request îi permite serverului să reconstruiască resursa așa cum era înainte de a fi transmisă în rețea și să precizeze forma de codare aplicată asupra mesajului. Câteva exemple des folosite într-o cerere sunt:
 - Accept-Charset
 - Host
 - User-Agent
 - Accept-Language
 - Accept-Encoding
 - Connection
 - Referer

Ultima componentă a unei cereri, compatibilă doar cu metodele PATCH, POST și PUT, este constituită din corpul mesajului care are rol în transmiterea informației către server. Corpul unui mesaj poate conține o cantitate scăzută de date sub formă de perechi de tip key=value (cheie=valoare) (2.21) sau date în mai multe componente (2.22):

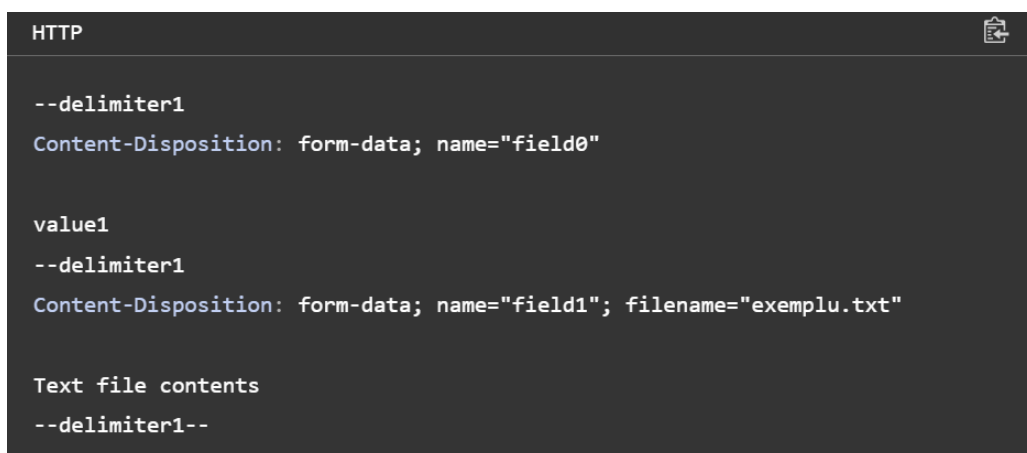


```

JSON
{
  "nume": "Marin",
  "prenume": "Costel",
  "email": "lic@exemplu.com"
}

```

Figura 2.21: Date din corp sub formă de key=values



```

HTTP
--delimiter1
Content-Disposition: form-data; name="field0"

value1
--delimiter1
Content-Disposition: form-data; name="field1"; filename="exemplu.txt"

Text file contents
--delimiter1--

```

Figura 2.22: Date din corp împărțite în mai multe componente

Răspunsuri HTTP

```

HTTP
-----
HTTP/1.1 201 Created
Content-Type: application/json
Location: http://exemplu.ro/user/321

{
  "mesaj": "Utilizator nou creat",
  "user": {
    "id": 321,
    "nume": "Marin",
    "prenume": "Costel",
    "email": "lic@exemplu.com"
  }
}

```

Figura 2.23: Exemplu de răspuns HTTP în care un nou utilizator este creat

Răspunsurile HTTP [14] sunt mesaje trimise de către server către clienți, în care răspunsul anunță clientul rezultatul determinat în urma cererii. Partea superioară (head) a mesajului HTTP este alcătuită din linia de status și anteturi, iar corpul, deși conține date transmise între client și server, este o componentă opțională.

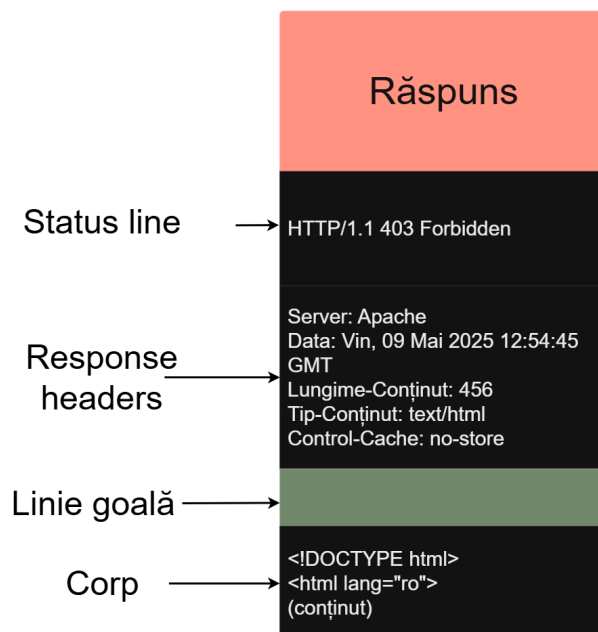


Figura 2.24: Structura răspunsului HTTP

Structura părții superioare a unui răspuns HTTP este următoarea:

1. Status line este format din trei componente:
 - Protocolul reprezintă versiunea protocolului HTTP pe care serverul l-a folosit pentru a transmite mesajul către client.
 - Codul de stare indică dacă starea cu care s-a finalizat transmiterea mesajului HTTP. Există cinci tipuri de coduri de stare cum ar fi: răspunsuri informaționale (cu valori

între 100 și 199) ce descriu transmiterea cu succes (200-299), mesaje de redirecționare (300-399), răspunsuri pentru erori de client (400-499) și răspunsuri pentru erori de server (500-599). Cele mai des întâlnite coduri de stare sunt 200 - OK (mesajul a fost transmis cu succes), 404 - Not Found (serverul nu poate găsi resursa cerută, URL-ul nu este recunoscut de către browser) și 302 - Found (URL-ul asociat resursei căutate a fost schimbat temporar).

- Textul de stare este un text pur informativ care explică semnificația codului de stare pentru a facilita o mai bună înțelegere a rezultatului generat în urma răspunsului.



Figura 2.25: Status line - alcătuire

2. Antetul unui răspuns conține metadate despre mesajul ce urmează a fi transmis clientului și se clasifică în:

Response header

Un response header este un antet cu metadatele trimise odată cu răspunsul de server către client. Fiecare antet este un șir de caractere care nu face diferențe între majuscule și litere mici, urmate de simbolul : și o valoare care este dependentă de tipul de antet folosit. Scopul response header-ului este de a oferi informații suplimentare despre mesaj sau de a explica logica cu care va trebui clientul să gestioneze cererile ce o să urmeze.

Representation header

Representation header-ul descrie forma datelor din mesaj, orice codare aplicată asupra acestuia și poate exista doar dacă mesajul are un corp.

Un representation header oferă unui destinatar instrucțiuni despre cum ar trebui să reconstruiască resursa înainte de a fi trimisă în rețea.

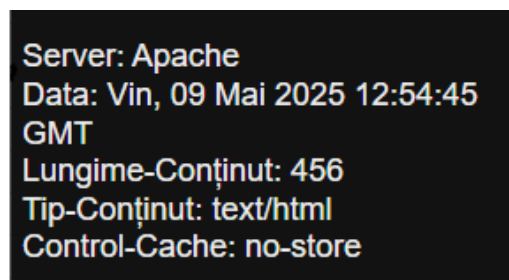


Figura 2.26: Exemplu de response și representation header

Ultima componentă a unui răspuns HTTP este corpul, prezent în majoritatea cazurilor atunci când un server trimite un răspuns către client. În cazul răspunsurilor reușite, corpul conține datele solicitate de către client prin intermediul metodei GET. În schimb, dacă apar erori în procesul de transmitere a mesajului, corpul mesajului va include detalii despre problemele care au cauzat imposibilitatea trimiterii răspunsului.

2.3 Framework-ul Flask Python

2.3.1 Prezentare generală

Flask [15] [16] este un framework web minimalist flexibil creat în limbajul Python, deseori numit „micro-framework” datorită accesibilității de a dezvolta aplicații web folosind doar componente esențiale, fără necesitatea folosirii unor biblioteci suplimentare sau structuri complicate și posibilitatea implementării rapide a unor noi funcționalități în aplicațiile web.

Flask a fost creat [17] în anul 2004 de către Armin Ronacher, un membru al grupului de pasionați de Python, Poccoo. Inițial, crearea celebrului framework a fost doar o aluzie la celebra zi a glumelor, 1 aprilie, dar mai târziu a devenit realitate datorită cererii mari a publicului. Numele „flask” (în engleză înseamnă flacon) este o referință la precedentul framework Bottle (în engleză sticlă). În anul 2016, echipa Poccoo și-a încheiat activitatea, iar dezvoltarea Flask-ului și a librăriilor acestuia a fost preluată de către echipa nou fondată numită Pallets.

2.3.2 Rolul Flask în aplicație dezvoltată

În contextul acestei lucrări de licență, am ales să implementez serverul aplicației web în Flask datorită funcționalităților de construire personalizată a site-urilor, definirea rutelor care facilitează redirectionarea către pagini atunci când un link este accesat și gestionarea sesiunilor de autentificare a utilizatorilor, dar și a cererilor trimise de către acesta aplicației web. Printre avantajele ale framework-ului Flask se regăsesc:

- Este un framework backend minimalist cu posibilitatea extinderii aplicației în funcție de cerințele impuse.
- Este ușor de învățat deoarece API-ul intuitiv oferă posibilitatea chiar și începătorilor să se familiarizeze cu el.
- Este un framework flexibil deoarece are abilitatea de a permite personalizarea și extinderea aplicației conform cerințelor cerute.
- Poate fi folosit cu o paletă largă de baze de date precum MySQL prin intermediul extensiilor sau al bibliotecilor externe.
- Routing HTTP care se ocupă cu definirea rutelor URL și asocierea acestora cu funcții care gestionează cererile și răspunsurile HTTP.
- Sesiunile de utilizatori pot fi ușor gestionate, deoarece Flask are posibilitatea de a stoca informații despre acestea în mod securizat pe server și client utilizând cookie-uri semnate.
- Posibilă integrarea cu Jinja2 care oferă o paletă largă de unelte pentru generarea paginilor HTML dinamice.

O primă funcționalitate pe care o are Flask este gestionarea logică a aplicației web și baza de date asociată. Atunci când o acțiune de tipul citire, scriere, update sau delete are loc la nivel de frontend, Flask va fi intermediarul între utilizator și baza de date în care va traduce comanda inițiată în limbajul SQL pentru executare, în final rezultatul fiind vizibil și la nivel de frontend.

Cea de a doua funcționalitate constă în permiterea autentificării și existența sesiunii de utilizator. Flask se ocupă cu crearea noilor conturi, logarea utilizatorilor, reținerea cine este logat și ce acțiuni îi sunt permise.

O altă funcționalitate se referă la afișarea paginilor dinamice folosind template-uri HTML.

Ultima funcționalitate o reprezintă controlul fluxului aplicației în care utilizatorii după efectuarea anumitor acțiuni sunt redirecționați spre alte pagini. De exemplu, la crearea unui nou cont, utilizatorii sunt redirecționați către pagina de login unde se vor conecta la aplicație.

2.4 Sistemul de Baze de Date MySQL

O bază de date reprezintă [18] o colecție de informații corelate organizate sub forma unui spațiu de stocare menite pentru a facilita procesele de gestionare, accesare a informațiilor de către utilizatori. Sistemul de gestiune a bazelor de date (SGBD) este un software fundamental ce stă la baza multor aplicații cum ar fi sisteme de rezervări, cumpărături online în care utilizatorii au posibilitatea să definească, să creeze, să întrețină și să controleze accesul la baza de date.

Concepte de Baze de Date Relaționale și SGBD

O bază de date relațională este o bază de date ce se bazează pe modelul relațional al datelor organizat sub forma unor tabele. Un tabel (sau relație) este format din rânduri și coloane. Rândurile, denumite și tupluri, reprezintă o entitate sau o înregistrare individuală în baza de date. Coloanele sau atributele, reprezintă caracteristicile sau proprietățile entităților. Fiecare coloană are un identificator unic și un tip de date specific.

Baze de Date Distribuite și Replicarea Master-Slave

O bază de date distribuită este o bază de date logică unică divizată în mai multe părți stocate fizic pe mai multe computere sau noduri interconectate în rețea. La nivel de frontend, o bază de date distribuită funcționează ca o singură bază de date centralizată, nu este nevoie ca utilizatorii să folosească mai multe interfețe, dar la nivel de backend datele sunt distribuite și replicate în diverse locații. Principalele obiective pe care o bază de date distribuită le prezintă includ scalabilitatea, disponibilitatea, fiabilitatea și performanța îmbunătățită prin alocarea unor locații cu roluri specifice.

Configurația master-slave este o structură de replicare a bazei de date în care datele de pe un master sunt copiate pe un slave.

- Un master este nodul principal în care au loc toate operațiile de scriere (INSERT, UPDATE, DELETE). Orice modificare efectuată asupra master-ului este transmisă și slave-lui.
- Slave-ul este un nod secundar ce are rol în efectuarea tuturor operațiilor de citire (SELECT) și în actualizarea datelor primite din partea master-ului.

În contextul prezentei lucrări, s-a implementat o arhitectură de tip master-slave datorită următoarelor avantaje:

- Scalabilitatea operațiilor de citire: într-un sistem de rezervări volumul operațiilor de citire depășește semnificativ volumul operațiilor de scriere. Așadar, o arhitectură de tip master-slave devine esențială prin direcționarea cererilor de citire către nodurile slave, eliberarea resurselor consumate de către nodul master și eficientizarea operațiilor de scriere importante. De asemenea, flexibilitatea sistemului asigură posibilitatea de extindere orizontală cu noi noduri slave, distribuind uniform și eficient cererile de citire pe măsură ce volumul de date crește.

- Separarea sarcinilor: arhitectura master-slave stabilește două roluri bine separate între cele două noduri ale bazei de date. Nodul master-ul se ocupă exclusiv cu procesarea și validarea operațiilor de scriere (INSERT, UPDATE, DELETE) pentru a asigura consistența și integritatea datelor. Nodurile slave au rol în preluarea și executarea operațiilor de citire (SELECT). Această separare a rolurilor optimizează performanța operațiilor de citire și scriere prin minimizarea conflictelor de blocare și contopire a resurselor.
- Îmbunătățirea disponibilității: configurația master-slave este o metodă preventivă în cazul unei defecțiuni sau indisponibilități a unei resurse. În cazul în care master-ul nu mai funcționează, slave-ul preia atribuțiile de scriere și este promovat la rolul de master. Acest mecanism se numește failover și are rol în menținerea continuității operațiilor de citire și scriere prin minimizarea timpului de nefuncționare a aplicației.

Limbaajul SQL și Securitatea Datelor

SQL (Structured Query Language) [18] este limbaajul principal utilizat pentru lucrul cu baze de date relaționale, fiind recunoscut ca standard în industrie. Acesta a fost inițial dezvoltat de IBM la începutul anilor '70, în cadrul proiectului System R, sub denumirea SEQUEL (Structured English Query Language). Ulterior, numele a fost abreviat la forma actuală, SQL, odată cu standardizarea sa. Limbaajul a fost conceput pentru a permite interogarea și manipularea datelor într-un mod declarativ, concentrându-se pe rezultatul dorit, mai degrabă decât pe pașii necesari obținerii acestuia.

SQL este strâns legat de modelul relațional, servind drept interfață standard pentru interacțiunea cu sistemele de gestiune a bazelor de date relaționale. În acest model, datele sunt organizate sub formă de tabele (relații), fiecare rând corespunzând unui tuplu, iar fiecare coloană unui atribut. Limbaajul permite definirea structurii bazei de date, manipularea conținutului și gestionarea accesului, oferind astfel o abstracție eficientă asupra modului în care datele sunt stocate și accesate. Prin natura sa declarativă, SQL permite exprimarea interogărilor într-un mod logic, independent de modul fizic de stocare, ceea ce îl face portabil între diverse implementări de SGBD-uri.

SQL este organizat în mai multe subcomponente funcționale, fiecare având un rol bine definit în interacțiunea cu baza de date. Comenzile de tip DDL (Data Definition Language) permit definirea și modificarea schemelor de date prin instrucțiuni precum **CREATE**, **ALTER** și **DROP**. Comenzile DML (Data Manipulation Language) includ operații precum **SELECT**, **INSERT**, **UPDATE** și **DELETE**, care sunt utilizate pentru interogarea și manipularea efectivă a datelor. De asemenea, SQL oferă suport pentru controlul accesului prin comenzi de tip DCL (Data Control Language), precum **GRANT** și **REVOKE**, și pentru gestionarea tranzacțiilor prin comenzi TCL, cum ar fi **COMMIT** și **ROLLBACK**. Această structurare clară a limbaajului contribuie la separarea responsabilităților în cadrul aplicațiilor care gestionează date relaționale.

În contextul aplicației Flask de rezervări, limbaajul SQL este utilizat ca principalul instrument pentru interacțiunea cu baza de date MySQL. Operațiile efectuate în backend, precum înregistrarea unui utilizator, autentificarea, crearea sau anularea unei rezervări sunt transpuse în comenzi SQL care sunt transmise serverului de baze de date pentru execuție. SQL pune la dispoziție o interfață standardizată pentru aceste operații, indiferent de complexitatea logicii aplicației, asigurând o gestionare uniformă și eficientă a datelor salvate. Prin folosirea unui limbaj standardizat, aplicația poate menține portabilitatea și compatibilitatea cu alte sisteme bazate pe același model relațional.

Deși SQL [19] oferă mecanisme integrate pentru definirea permisiunilor de acces asupra obiectelor din baza de date, responsabilitatea implementării unei securități eficiente nu se

oprește la nivelul SGBD-ului. În mod special, aplicațiile web trebuie să adopte practici suplimentare pentru a preveni vulnerabilități precum SQL injection. Astfel, în cadrul aplicației, sunt utilizate interogări parametrizate care înlocuiesc concatenarea directă a datelor în instrucțiunile SQL, limitând posibilitatea introducerii de cod malițios. Acest aspect este esențial pentru protejarea integrității și confidențialității datelor gestionate.

2.5 Nginx (Load Balancing și Reverse Proxy)

Nginx (Engine-X) [20] este un server web de înaltă performanță, un reverse proxy și un load balancer. A fost ales pentru a îndeplini rolurile de server web, cât și reverse proxy cu capabilități de load balancing. Această componentă este poziționată strategic în fața serverelor de aplicații Flask, acționând ca un "distribuitor de trafic" și un punct de intrare unic pentru toate cererile utilizatorilor.

Principiul de Funcționare al Load Balancing-ului

Load balancing reprezintă procesul de distribuire a traficului de rețea sau a sarcinilor de lucru între mai multe servere, având ca obiective utilizarea eficientă a resurselor, creșterea debitului, reducerea timpului de răspuns și prevenirea supraîncărcării unui singur server. În absența unui load balancer, un volum ridicat de trafic poate suprasolicita un server unic, ceea ce conduce la degradarea performanței aplicației sau chiar la indisponibilitatea acesteia. Configurat ca load balancer, Nginx preia toate cererile HTTP/HTTPS de la clienți și le distribuie către instanțele aplicației Flask (denumite upstreams în terminologia Nginx). Distribuirea se poate realiza pe baza unor algoritmi specifici de load balancing, pe care Nginx îi suportă nativ:

- Round Robin: Este cea mai simplă și cea mai comună strategie. Cererile sunt distribuite secvențial către fiecare server din lista upstreams. Fiecare server primește o cerere la rând
- Least Connections: Nginx direcționează cererile către serverul cu cel mai mic număr de conexiuni active. Aceasta este adesea mai eficientă atunci când timpii de procesare a cererilor variază.
- IP Hash: Direcționează cererile pe baza adresei IP a clientului. Aceasta asigură că un client anume va fi întotdeauna trimis către același server, util pentru aplicațiile care depind de persistența sesiunii pe server.

Integrarea Nginx cu Flask ca Reverse Proxy

Pe lângă rolul său de load balancer, Nginx acționează și ca un reverse proxy pentru aplicația Flask. Un reverse proxy este un server care primește cereri de la clienți și le retransmite către serverele interne, gestionând traficul între clienți și infrastructura internă. Configurația Nginx implică definirea unui bloc upstream care listează adresele (IP și port) ale tuturor instanțelor Flask disponibile. Apoi, într-un bloc server care ascultă pe porturile HTTP/HTTPS standard (80/443), cererile sunt redirectionate către grupul upstream folosind directive precum proxy_pass. Un alt aspect esențial al integrării Nginx constă în capacitatea sa de a furniza fișiere statice, precum CSS, JavaScript sau imagini încărcate de utilizatori. Deși aplicațiile Flask pot livra fișiere statice, acestea nu sunt optimizate pentru această sarcină, iar gestionarea directă a fișierelor statice poate consuma resurse semnificative ale serverului de aplicații. Nginx este extrem de eficient în servirea fișierelor statice direct din sistemul de fișiere, reducând semnificativ sarcina pe serverul Flask. De exemplu, cererile către /static/ sau /uploads/ pot fi

configurate în Nginx pentru a fi servite direct, în timp ce toate celelalte cereri dinamice sunt transmise către Flask. Un exemplu simplificat de configurație Nginx ar putea arăta astfel:A.1

Această configurație exemplifică modul în care Nginx preia controlul traficului, direcționând cererile statice direct și proxy-ind cererile dinamice către grupul de servere Flask. Directivele `proxy_set_header` sunt importante pentru a transmite informații despre cererea originală (IP-ul clientului, host, protocol) către serverul Flask, care altfel ar vedea doar IP-ul Nginx-ului.

Impactul asupra Fluxurilor de Aplicație

Rolul Nginx ca load balancer și reverse proxy îmbunătățește performanța, scalabilitatea și reziliența tuturor fluxurilor de aplicație prezentate anterior:

- Înregistrare și autentificare: Chiar și în condiții de trafic intens, cererile de înregistrare și autentificare sunt distribuite, asigurând că procesul de creare de conturi noi și de accesare a conturilor existente rămâne rapid și responsiv. Dacă un server Flask devine indisponibil, Nginx redirecționează cererile către serverele active, asigurând funcționarea continuă a aplicației.
- Căutare și vizualizare proprietăți: Fluxul de căutare, care poate genera un număr mare de cereri SELECT către backend, este optimizat prin distribuția încărcării realizată de Nginx. Astfel, timpii de răspuns pentru utilizatori rămân constanți, indiferent de volumul cererilor.
- Crearea unei rezervări: Deși procesul de rezervare reprezintă o tranzacție complexă cu mai mulți pași, distribuirea cererilor inițiale (din formularul de căutare sau de vizualizare a proprietății) către servere multiple menține receptivitatea sistemului. Nginx contribuie la stabilitatea conexiunii cu serverul Flask care procesează rezervarea.
- Fluxurile de management (proprietari și admin): Chiar dacă aceste fluxuri generează un volum redus de trafic, ele se desfășoară cu aceeași reziliență. Administratorii și proprietarii pot efectua operațiuni asupra proprietăților și rezervărilor fără întârzieri, chiar dacă alte componente ale sistemului sunt supuse unei încărcări ridicate sau dacă o instanță de server devine indisponibilă. Nginx garantează disponibilitatea, performanța și capacitatea aplicației de a scala în funcție de cerințele utilizatorilor.

Capitolul 3

Arhitectura și Structura Aplicației

3.1 Introducere

Creșterea exponențială a numărului de utilizatori și a volumului de date în mediul online a impus noi provocări în dezvoltarea aplicațiilor web. Sistemele software moderne, în special platformele care intermediază servicii, trebuie să gestioneze un număr ridicat de interacțiuni simultane fără a compromite performanța, securitatea și fiabilitatea. Trecerea procesului de rezervare din mediul tradițional către platforme digitale, accelerată de răspândirea dispozitivelor conectate la internet, impune constrângeri tehnice semnificative, în special în gestionarea cererilor simultane și a volumelor mari de date.

În acest context, lucrarea urmărește studiul, proiectarea și implementarea unei platforme web pentru rezervări de cazare, utilizată ca studiu de caz pentru aplicarea principiilor avansate de inginerie software. Scopul principal nu se limitează la crearea unei aplicații funcționale, ci constă în realizarea unui sistem cu arhitectură proiectată pentru performanță și fiabilitate. Proiectul a fost ghidat de un set de obiective tehnice bine definite:

1. **Performanță:** Optimizarea timpului de răspuns pentru interacțiunile utilizatorului, atât în cazul operațiunilor de afișare rapidă, cât și în contextul proceselor mai complexe, precum căutarea proprietăților și efectuarea rezervărilor.
2. **Modularitate:** Organizarea logicii aplicației în componente distincte, fiecare având un rol clar definit, pentru a facilita testarea independentă, întreținerea facilă și actualizările punctuale.
3. **Scalabilitate:** Proiectarea unei arhitecturi capabile să gestioneze un volum crescut de utilizatori, fără afectarea performanței, prin soluții ce permit extinderea sistemului pe orizontală.
4. **Capacitatea de testare:** Realizarea componentelor software într-o formă care permite aplicarea eficientă a testelor unitare și de integrare, contribuind la menținerea unui comportament previzibil și fiabil al aplicației.

3.2 Arhitectura Generală a Sistemului Distribuit

Soluția arhitecturală propusă pentru îndeplinirea acestor obiective este una distribuită, multi-nod. Sistemul nu se bazează pe o singură mașină, ci pe un ansamblu de componente interconectate, fiecare cu un rol specializat. Această structură, prezentată vizual în Figura 3.1, stă la baza strategiei de disponibilitate și echilibrare a sarcinilor.

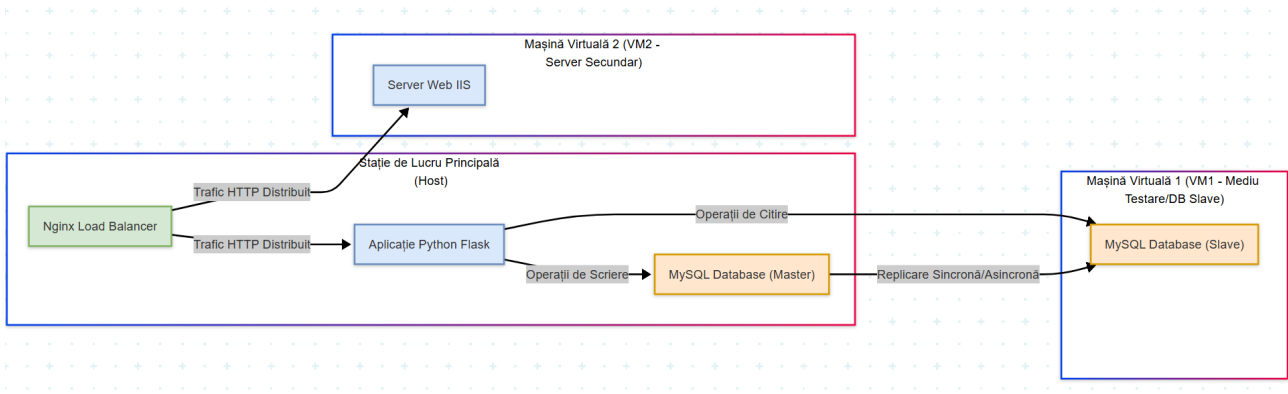


Figura 3.1: Diagrama arhitecturală a componentelor sistemului distribuit

1. Stația de lucru principală (Host): constituie mediul principal de dezvoltare și găzduire a componentelor esențiale ale aplicației. Pe stație rulează o instanță a aplicației Python Flask, care implementează logica de business, și serverul MySQL Master, responsabil pentru operațiile de scriere în baza de date. De asemenea, Nginx este configurat ca load balancer, gestionând și distribuind traficul HTTP către celelalte instanțe Flask.
2. Mașină virtuală 1 (VM1 - Mediu Testare / DB Slave): Această mașină virtuală este destinată mediului de testare și găzduiește serverul MySQL Slave. Rolul său principal constă în preluarea și executarea operațiilor de citire din baza de date, precum și asigurarea redundanței datelor prin replicarea modificărilor de la serverul Master. Optimizarea operațiilor de citire-scriere reprezintă un obiectiv central al acestei configurații.
3. Mașină virtuală 2 (VM2 - server secundar): găzduiește o instanță secundară a aplicației Flask. Rolul său constă în preluarea cererilor distribuite de Nginx, contribuind la creșterea performanței și a capacității de procesare a aplicației web prin load balancing. Împreună cu instanța de pe Host, această configurație asigură scalabilitatea orizontală a stratului de aplicație.
4. Nginx load balancer: instalată pe stația de lucru principală, componenta Nginx funcționează ca reverse proxy și distribuitor de trafic. Aceasta preia toate cererile utilizatorilor și le direcționează către instanțele Flask disponibile pe Host și VM2, conform strategiilor de load balancing. În plus, Nginx este configurat pentru a servi fișiere statice, reducând astfel încărcarea serverelor Flask.
5. Aplicație Python Flask: reprezintă stratul de logică de business al aplicației web și rulează pe Host și VM2. Aceasta gestionează cererile utilizatorilor, interacționează cu bazele de date MySQL Master și Slave și se ocupă de sesiuni și autentificare.
6. MySQL Database (Master/Slave): configurația bazei de date este de tip master-slave, unde serverul Master (pe Host) gestionează toate scrierile, iar serverul Slave (pe VM1) se ocupă de citiri. Această separare a sarcinilor optimizează performanța operațiilor de citire și scriere și asigură înalta disponibilitate a datelor prin replicare.

Împreună, cele trei mașini (Stația de Lucru Principală, VM1 și VM2) constituie o rețea IP/LAN privată, formând un mediu izolat și consistent, esențial pentru testarea scenariilor de eșec și a rezilienței arhitecturii.

3.3 Arhitectura Bazei de Date

Introducere

Baza de date a aplicației "rezervari" este concepută pentru a stoca și gestiona eficient toate informațiile legate de utilizatori, proprietăți, camere, rezervări și plățile aferente. Structura sa relațională asigură integritatea datelor și permite interogări complexe necesare funcționalităților aplicației. Baza de date este compusă din șase tabele principale, interconectate prin chei primare și străine, asigurând integritatea referențială și organizarea logică a datelor.

3.3.1 Tabela users

Tabela **users** reprezintă componenta centrală a aplicației, stocând informațiile esențiale despre utilizatorii platformei. Aceasta conține date necesare pentru autentificare, autorizare și personalizarea experienței utilizatorilor.

Structură:

- **user_id**: Acesta este identificatorul unic primar pentru fiecare utilizator, generat automat. Este cheia principală prin care alte tabele se vor lega la un utilizator specific.
- **username**: Numele de utilizator ales de către utilizator la înregistrare, care trebuie să fie unic pe platformă. Este folosit, de obicei, pentru login.
- **email**: Adresa de email a utilizatorului, de asemenea, unică, esențială pentru comunicare și recuperarea contului.
- **password**: Stochează parola utilizatorului, dar nu în format text clar, ci ca un hash securizat. Această abordare criptografică este vitală pentru a proteja credențialele utilizatorilor chiar și în cazul unei breșe de securitate a bazei de date.
- **is_admin**: Un flag boolean (0 pentru utilizator normal, 1 pentru administrator) care definește rolul și nivelul de acces al utilizatorului în cadrul aplicației.
- **created_at**: Timestamp-ul la care a fost creat contul utilizatorului.

Relații și implicații:

- **‘users’ (1) — ‘properties’ (M)**: Un utilizator poate deține zero sau mai multe proprietăți. Relația este stabilită prin cheia străină **owner_id** din tabela **properties**, care referă la **user_id** din **users**. Aceasta permite aplicației să afișeze proprietățile unui anumit proprietar.
- **‘users’ (1) — ‘reservations’ (M)**: Un utilizator poate efectua zero sau mai multe rezervări. Această legătură este realizată prin cheia străină **user_id** din tabela **reservations**, care referă la **user_id** din **users**. Este esențială pentru a urmări istoricul rezervărilor unui client.

3.3.2 Tabela properties

- **Scop**: Tabela **properties** este dedicată stocării detaliilor complete despre fiecare unitate de cazare (hotel, pensiune, apartament) înregistrată pe platformă.
- **Coloane cheie și semnificația lor**:

- **property_id**: Cheia primară auto-incrementată, identificatorul unic al proprietății.
- **owner_id**: O cheie străină care face legătura cu tabela **users**, specificând cine este proprietarul acestei proprietăți.
- **name**: Numele comercial al proprietății (ex: "Hotel Continental", "Pensiunea La Cetate").
- **description**: O descriere detaliată a proprietății, utilă pentru a oferi informații complete utilizatorilor.
- **address, city, country**: Coloane pentru detaliile de localizare, esențiale pentru căutări și orientare.
- **created_at**: Timestamp-ul adăugării proprietății în sistem.

- **Relații și implicații:**

- **‘properties’ (1) — ‘rooms’ (M)**: O proprietate poate conține una sau mai multe camere (sau tipuri de camere). Această legătură este realizată prin cheia străină **property_id** din tabela **rooms**, care referă la **property_id** din **properties**. Aceasta asigură organizarea structurată a spațiilor de cazare pentru fiecare proprietate.
- Relația cu **users** este bidirecțională: proprietățile sunt listate în funcție de proprietarul lor, iar ștergerea unui utilizator șterge în cascadă și proprietățile deținute de acesta (**ON DELETE CASCADE**).

3.3.3 Tabela rooms

- **Scop**: Tabela **rooms** stochează informații detaliate pentru fiecare cameră sau tip de cameră (în funcție de semnificația coloanei **number_of**) disponibilă pentru rezervare.

- **Coloane cheie și semnificația lor:**

- **idrooms**: Cheia primară auto-incrementată pentru fiecare cameră înregistrată.
- **property_id**: Cheia străină către proprietatea căreia îi aparține camera.
- **name**: Numele tipului de cameră (ex: "Cameră single", "Apartament cu 2 dormitoare").
- **description**: O scurtă descriere a facilităților camerei.
- **capacity**: Numărul maxim de persoane pe care această cameră le poate găzdui. Esențial pentru algoritmul de recomandare și validare a rezervărilor.
- **price**: Prețul standard pe noapte pentru această cameră.
- **available**: Flag boolean care indică disponibilitatea generală a camerei (dacă poate fi rezervată sau nu, independent de perioadă).

- **Relații și implicații:**

- **rooms (1) — reservations (M)**: O cameră poate fi inclusă în zero sau mai multe rezervări de-a lungul timpului. Această legătură se face prin cheia străină **room_id** din tabela **reservations**, fiind necesară pentru a urmări ce cameră a fost rezervată.
- Relația cu **properties** asigură că ștergerea unei proprietăți duce la ștergerea în cascadă a camerelor asociate (**ON DELETE CASCADE**).

3.3.4 Tabela `reservations`

- Scop: Tabela `reservations` reprezintă componenta centrală a sistemului de rezervări, înregistrând fiecare tranzacție efectuată în cadrul platformei.
- Coloane cheie și semnificația lor:
 - `reservation_id`: Cheia primară auto-incrementată pentru identificarea unică a fiecărei rezervări.
 - `user_id`: Cheia străină care indică utilizatorul care a inițiat rezervarea.
 - `room_id`: Cheia străină către camera specifică care a fost rezervată.
 - `start_date`, `end_date`: Definește intervalul de timp pentru care este făcută rezervarea. Aceste câmpuri sunt critice pentru verificările de disponibilitate și pentru a preveni "double-booking".
 - `status`: Statusul curent al rezervării (e.g., 'pending' după inițiere, 'confirmed' după plată/verificare, 'cancelled' dacă a fost anulată).
 - `created_at`: Timestamp-ul la care a fost creată înregistrarea rezervării.
- Relații și implicații:
 - **'reservations' (1) — 'reservation_details' (1)**: O rezervare are un singur set de detalii de contact suplimentare. Legătura este prin cheia străină `reservation_id` din `reservation_details`.
 - **'reservations' (1) — 'payments' (M)**: O rezervare poate avea zero sau mai multe plăți asociate (util pentru scenarii de plăți parțiale, reîncercări de plată sau eșecuri). Legătura se face prin cheia străină `reservation_id` din tabela `payments`.
 - Relațiile cu `users` și `rooms` sunt gestionate prin `user_id` și `room_id`. Ștergerea unui utilizator sau a unei camere va duce la ștergerea în cascadă a rezervărilor asociate ('ON DELETE CASCADE').

3.3.5 Tabela `reservation_details`

- Scop: Tabela stochează detalii de contact ale clientului care a efectuat rezervarea, precum și prețul total al acesteia. Separarea acestor informații respectă principiile de normalizare, reducând redundanța și facilitând gestionarea datelor sensibile.
- Coloane cheie și semnificația lor:
 - `detail_id`: Cheia primară auto-incrementată pentru fiecare intrare de detalii.
 - `reservation_id`: Cheia străină care leagă direct aceste detalii la o înregistrare specifică din tabela `reservations`.
 - `full_name`: Numele complet al persoanei care a făcut rezervarea.
 - `phone`: Numărul de telefon de contact. `email`: Adresa de email de contact.
 - `total_price`: Prețul total final calculat pentru rezervarea respectivă.
- Relații și implicații:
 - Relația este de tip One-to-One cu tabela `reservations`: fiecare rezervare are (sau ar trebui să aibă) exact un set de detalii asociate (`reservations.reservation_id` -> `reservation_details.reservation_id`).

- Este important de notat că în schema furnizată, `reservation_id` din această tabelă este definită doar ca un `KEY`, nu ca `FOREIGN KEY` cu o regulă `ON DELETE CASCADE` explicită. Aceasta înseamnă că, deși relația logică există, baza de date nu o impune fizic, iar aplicația trebuie să asigure integritatea datelor (ex: ștergerea detaliilor la ștergerea rezervării).

3.3.6 Structura Tabelelor și Relațiile

inserare grafic mare

3.3.7 Fluxul Datelor și Funcționarea Generală a Bazei de Date

Funcționalitatea aplicației se bazează pe interacțiuni cu această structură de baze de date, gestionând toate etapele procesului de rezervare:

1. **Înregistrarea utilizatorului:** Când un utilizator nou se înregistrează, o înregistrare este inserată în tabela `users`, atribuindu-i un `user_id` unic. Parola este stocată hashed.
2. **Adăugarea Proprietății:** Un utilizator cu rol de proprietar adaugă o proprietate. O înregistrare este creată în tabela `properties`, cu `owner_id` referind la `user_id`-ul proprietarului. Ulterior, camerele sunt adăugate în tabela `rooms`, fiecare legată de `property_id`-ul proprietății.
3. **Căutarea Proprietăților:** Când un client caută o proprietate, aplicația interoghează tabelele `properties` și `rooms` pentru a găsi proprietățile care corespund criteriilor și a verifica disponibilitatea camerelor pentru o anumită perioadă. Acestea sunt în principal operații de citire.
4. **Crearea Rezervării:** Acesta este un flux complex și tranzacțional:
 - Aplicația inserează o nouă înregistrare în tabela `reservations`, legând-o de `user_id`-ul clientului și `room_id`-ul camerei selectate, împreună cu datele de start și end.
 - Concomitent, detaliile de contact și prețul total sunt inserate în tabela `reservation_details`, cu o legătură la `reservation_id`-ul nou creat.
 - Operațiunile se efectuează în cadrul unei tranzacții ACID, asigurând că toate înregistrările sunt create cu succes sau niciuna. Aceasta previne stările inconsistente ale datelor, cum ar fi o rezervare fără detalii de contact.
5. **Gestionarea Rezervărilor:** Utilizatorii și proprietarii pot vizualiza rezervările, interogând tabela `reservations` (cu JOIN-uri către `rooms`, `properties` și `reservation_details`).
6. **Anularea Rezervării:** La anulare, statusul rezervării în tabela `reservations` este actualizat la "cancelled".

3.3.8 Importanța Integrității Referențiale

Utilizarea cheilor străine (`FOREIGN KEY`) cu reguli precum `ON DELETE CASCADE` și `ON UPDATE CASCADE` este esențială pentru menținerea integrității referențiale a bazei de date. De exemplu, ștergerea unui utilizator din tabela `users` determină eliminarea automată a tuturor

proprietăților și rezervărilor asociate din tabelele **properties** și **reservations**. Acest mecanism previne apariția înregistrărilor neasociate și simplifică logica aplicației, delegând gestionarea consistenței către sistemul de gestiune a bazelor de date. În situațiile în care **FOREIGN KEY** nu este definită la nivel de schemă, aplicația trebuie să asigure manual integritatea datelor.

Dacă arhitectura generală descrie interacțiunile la nivel de sistem, arhitectura internă detaliază modul de construcție al componentei centrale: aplicația web. Pentru implementarea acestui nucleu s-a utilizat framework-ul Flask, o alegere justificată de caracterul său minimalist și flexibil.

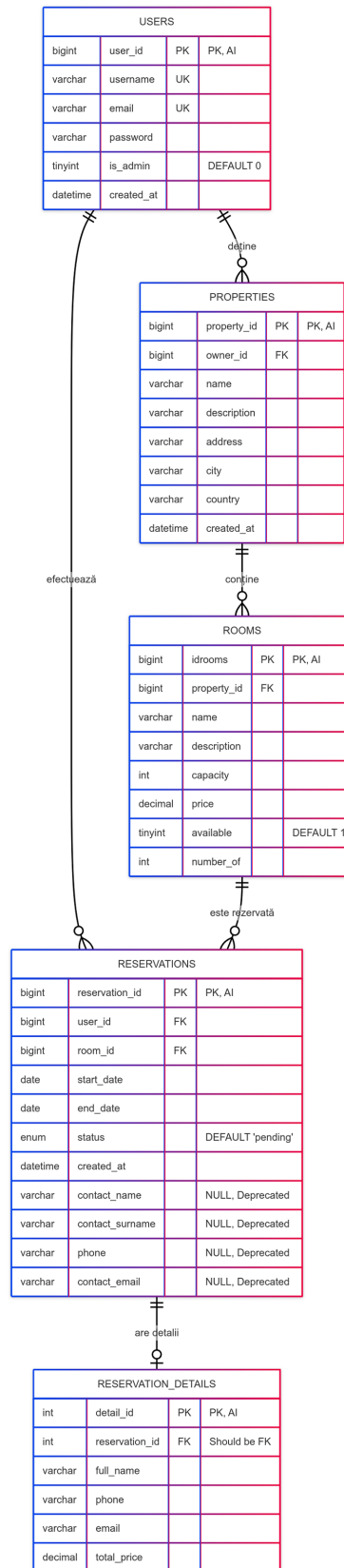


Figura 3.2: Structura bazei de date

3.4 Structura Internă a Aplicației Flask

Organizarea codului sursă

Aplicația Flask realizată este bine structurată și se bazează pe o separare clară a responsabilităților, ceea ce o face scalabilă și ușor de întreținut. Fiecare componentă a aplicației este concepută să gestioneze sarcini distincte, permițând dezvoltarea, testarea și scalarea independentă fără a afecta funcționarea celorlalte părți ale sistemului.

Inițializarea aplicației începe prin crearea unei instanțe standard Flask. La acest pas, este setată o cheie secretă, esențială pentru gestionarea securizată a sesiunilor și pentru protejarea operațiunilor sensibile, precum autentificarea utilizatorilor. Baza de date este configurată într-o arhitectură Master-Slave, unde toate operațiunile de scriere, cum ar fi înregistrarea sau actualizarea datelor, sunt direcționate către baza de date principală, în timp ce operațiunile de citire, cum ar fi vizualizarea proprietăților, sunt preluate de baza de date secundară. Această separare permite distribuirea eficientă a traficului și eliberează baza de date principală să se concentreze pe operațiuni critice de scriere. În plus, aplicația utilizează pooling de conexiuni pentru ambele baze de date, prin `mysql.connector.pooling`, ceea ce reduce semnificativ timpul de răspuns, deoarece conexiunile existente sunt reutilizate în loc să fie create altele noi pentru fiecare cerere.

Interacțiunea cu baza de date este gestionată prin context managers în Python, care asigură deschiderea și închiderea automată a conexiunilor chiar și în cazul apariției unor erori. De asemenea, gestionarea cursorilor se face printr-un context manager dedicat, simplificând logica de acces la date în cadrul fiecărei rute. Fiecare rută este configurată să folosească în mod deliberat fie conexiunea către master pentru scriere, fie conexiunea către slave pentru citire, asigurând integritatea datelor și performanța optimă a sistemului. Operațiuni precum înregistrarea, adăugarea unei proprietăți sau confirmarea unei rezervări folosesc conexiunea master, în timp ce vizualizarea proprietăților, căutările și afișarea profilului utilizează conexiunea slave.

Structura rutelor este organizată logic pe funcționalități, chiar dacă aplicația nu folosește Blueprints. Astfel, rutele pentru autentificare, gestionarea utilizatorilor, administrarea proprietăților, rezervări și zona de administrare sunt grupate și ușor de identificat. Fiecare funcție asociată unei rute verifică autentificarea utilizatorului, gestionează cererile GET și POST, interacționează cu baza de date prin context managers, afișează mesaje flash pentru feedback-ul utilizatorului și redă template-uri HTML cu datele necesare.

Pe lângă logica principală, aplicația include funcții utilitare care sporesc claritatea și reutilizarea codului. Funcția `is_logged_in()` verifică rapid dacă un utilizator este autentificat, iar filtrul Jinja2 `format_ro_date` afișează datele într-un format prietenos pentru utilizator. Motorul de recomandări `get_recommendations` realizează logica complexă pentru selectarea camerelor potrivite pe baza criteriilor de căutare, iar funcția `check_server_status` permite monitorizarea stării serverelor în panoul de administrare.

În ceea ce privește implementarea pentru producție, aplicația nu rulează pe serverul de dezvoltare al Flask. În schimb, se utilizează `waitress`, un server WSGI robust, capabil să gestioneze multiple cereri concurente. Configurarea acestuia permite accesul aplicației pe rețea și asigură stabilitate și performanță ridicată chiar și în condiții de trafic intens.

Capitolul 4

Mecanisme de Comunicare și Securitate (Funcționalitățile Aplicației)

4.1 Introducere în Mecanismul de Comunicare

În această secțiune sunt prezentate componentele funcționale fundamentale ale aplicației, cu accent pe modul în care sunt gestionate cererile utilizatorilor - de la interacțiunea cu interfața grafică, la prelucrarea pe server și până la salvarea datelor în baza de date. Analiza este structurată pe etape distincte, pentru a evidenția clar modul în care aceste componente colaborează. Am urmărit să subliniez procesele care contribuie direct la asigurarea unei funcționări stabile, sigure și coerente a aplicației.

4.2 Înregistrarea și Autentificare Utilizator

Mecanismele de înregistrare și autentificare reprezintă punctul de acces principal în cadrul sistemului, constituind atât prima interacțiune a utilizatorilor noi cu aplicația, cât și mijlocul prin care utilizatorii existenți accesează funcționalitățile individualizate. În realizarea acestui modul, am urmărit să asigur un echilibru între simplitatea în utilizare și protecția solidă a datelor. Componenta de autentificare reprezintă un element critic în arhitectura aplicației, întrucât orice vulnerabilitate apărută în acest punct poate compromite atât confidențialitatea datelor cu caracter personal, cât și funcționarea generală a sistemului. Securitatea procesului de autentificare nu trebuie privită ca un simplu pas procedural, ci ca o condiție fundamentală pentru construirea oricărui mecanism care implică identificarea utilizatorului și accesul diferențiat la funcționalități. În absența unei implementări solide, celelalte componente ale aplicației nu pot opera în condiții de siguranță și fiabilitate.

4.2.1 Formularul de înregistrare

Procesul de înregistrare este poarta de intrare a utilizatorilor în aplicație. Implementarea sa este împărțită între interfața cu utilizatorul, responsabilă de colectarea datelor, și logica de backend, care asigură validarea, securizarea și persistența acestora.

Interfața Frontend

Interfața de înregistrare este un formular HTML simplu și intuitiv, conceput pentru a colecta informațiile esențiale de la un utilizator nou. Formularul conține următoarele câmpuri:

- **Nume de utilizator:** Un câmp text unic pentru identificarea în platformă.
- **Email:** Folosit pentru comunicare și ca un identificator secundar.
- **Parolă:** Un câmp de tip parolă, al cărui conținut este mascat în browser.

- **Autentică-te:** Pentru utilizatorii care deja au cont și nu mai este necesară autentificarea.



Înregistrare Cont Nou

Nume utilizator:

Email:

Parolă:

Înregistrează-te

Ai deja cont? [Autentică-te](#)

Figura 4.1: Formularul de înregistrare a utilizatorilor

Formularele HTML pentru înregistrare utilizează elemente standard pentru interacțiunea cu utilizatorul. Câmpurile `<input type="text">` sunt destinate introducerii numelui de utilizator, iar `<input type="email">` asigură validarea automată a formatului adresei de email. Parola este introdusă în câmpul `<input type="password">`, care ascunde caracterele tastate, protejând astfel confidențialitatea datelor și prevenind expunerea acestora în mediul public („shoulder surfing”). Atributul `required` impune completarea obligatorie a tuturor câmpurilor, oferind o validare preliminară la nivel de browser. Totuși, validarea finală și securizarea datelor sunt realizate la nivelul serverului, pentru a preveni atacurile prin manipularea cererilor HTTP și pentru a menține integritatea și funcționalitatea corectă a aplicației.

Fragmentul de cod HTML de mai jos ilustrează structura simplă și eficientă a formularului de înregistrare: A.3

Codul folosește clase CSS pentru organizarea și stilizarea elementelor formularului. Aceste clase contribuie la crearea unei interfețe moderne, clare și responsivă, facilitând o experiență utilizator coerentă și intuitivă. O astfel de abordare este importantă pentru menținerea interesului și confortului utilizatorilor pe durata interacțiunii cu aplicația.

Logica Backend

Procesarea datelor se realizează în funcția asociată rutei `/register`, care gestionează cererile de tip POST. Logica de business urmează un flux de validare strict, în mai mulți pași:

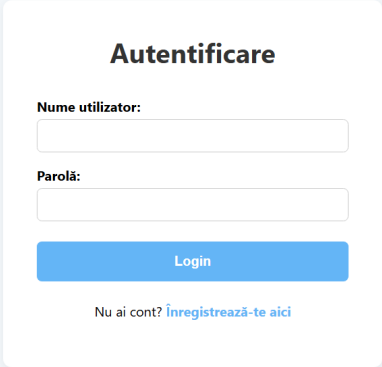
1. **Preluarea Datelor:** Informațiile trimise prin formular sunt extrase din obiectul `request.form`.
2. **Validarea Unicității:** Se execută o interogare SQL `SELECT` pentru a verifica dacă numele de utilizator sau adresa de email introduse există deja în tabelul `users`. Dacă se găsește o înregistrare existentă, procesul este oprit, iar utilizatorul este informat printr-un mesaj flash.
3. **Securizarea Parolei:** Dacă datele sunt unice, parola introdusă de utilizator este procesată folosind biblioteca `bcrypt`. Funcția `bcrypt.hashpw()` generează un hash securizat, care este non-reversibil. Aceasta este cea mai importantă etapă de securitate.

4. **Persistența Datelor:** Numele de utilizator, email-ul și hash-ul parolei sunt inserate ca o nouă înregistrare în tabelul `users` printr-o comandă `SQL INSERT`. Operațiunea este finalizată printr-un `commit` la baza de date.
5. **Feedback și Redirecționare:** După succesul operațiunii, se afișează un mesaj de confirmare, iar utilizatorul este redirecționat către pagina de `/login` pentru a se putea autentifica cu noul cont creat.

A.4

4.2.2 Formularul de autentificare

Procesul de autentificare este mecanismul prin care un utilizator înregistrat obține acces la funcționalitățile protejate ale platformei. Acesta este implementat prin intermediul rutei `/login` și implică atât o componentă de interfață, cât și o logică de validare robustă în backend.



Formularul de autentificare este prezentat într-un mod minimalist. Are un titlu centralizat "Autentificare". Sub acesta sunt două câmpuri de text: primul este etichetat "Nume utilizator:" și al doilea "Parolă:". Ambele câmpuri sunt simple, fără borduri de eroare. Sub câmpurile este un buton albastru cu textul "Login" în alb. În partea de jos a formularului, sub buton, este textul "Nu ai cont? [Înregistrează-te aici](#)" în albastru.

Figura 4.2: Formularul de autentificare a utilizatorilor

Interfața Frontend

Interfața de autentificare este un formular `HTML` minimalist, care solicită utilizatorului introducerea a două date esențiale:

- **Nume de utilizator:** Câmpul de identificare al contului.
- **Parolă:** Câmpul de verificare a identității, de tip parolă.

La trimitere, datele sunt transmise serverului printr-o cerere de tip **HTTP POST**. Interfața este, de asemenea, capabilă să afișeze mesaje de eroare (ex: "Date de autentificare incorecte") prin sistemul `flash` al `Flask`, oferind un feedback clar utilizatorului în cazul unui eșec.

Logica Backend

Funcția `login()` din backend este responsabilă cu validarea credențialelor. Procesul este securizat și eficient, urmând mai mulți pași logici:

1. **Preluarea Credențialelor:** Datele (nume de utilizator și parolă) sunt extrase din cererea POST.
2. **Căutarea Utilizatorului în Baza de Date:** Se execută o comandă SQL `SELECT` pentru a găsi înregistrarea din tabelul `users` care corespunde numelui de utilizator introdus. Dacă nu se găsește niciun utilizator, autentificarea eșuează.
3. **Verificarea Criptografică a Parolei:** Acesta este pasul critic de securitate. Parola în clar, introdusă de utilizator, este comparată cu hash-ul stocat în baza de date folosind funcția `bcrypt.checkpw()`. Această funcție extrage automat "salt"-ul din hash-ul stocat, aplică operația de hashing parolei introduse și compară rezultatele. Este un proces deliberat lent, pentru a contracara atacurile de tip brute-force.
4. **Crearea Sesiunii de Utilizator:** Dacă parola este corectă, se inițiază o sesiune pentru utilizator. Informații cheie, precum `user_id`, `username` și rolul `is_admin`, sunt stocate în obiectul `session` al Flask. Aceste date sunt trimise clientului sub forma unui cookie semnat criptografic, care va fi folosit pentru a identifica utilizatorul la cererile ulterioare.
5. **Redirecționare:** După succesul autentificării, utilizatorul este redirecționat către pagina principală a aplicației (`/home`).

4.2.3 Mecanismele de Securitate

Securitatea conturilor de utilizator reprezintă o componentă esențială a aplicației, asigurată prin metode criptografice bine implementate. Controlul accesului se realizează prin două rute principale definite în fișierul `app.py`: `/register` și `/login`. Aceste rute nu se limitează la preluarea și procesarea datelor trimise din partea frontend-ului, ci includ și mecanisme avansate menite să protejeze informațiile sensibile ale utilizatorilor.

În momentul înregistrării unui cont nou, parola este prelucrată printr-un algoritm de hashing securizat, astfel încât nicio variantă lizibilă a acesteia să nu fie salvată în baza de date. Această operațiune este realizată cu ajutorul bibliotecii **bcrypt**.

Funcția `bcrypt.hashpw()` combină parola (codificată în format binar) cu o valoare "salt" unică, generată de `bcrypt.gensalt()`. La autentificare, parola introdusă este verificată cu hash-ul stocat folosind funcția `bcrypt.checkpw()`. Astfel, protecția credențialelor este menținută la un nivel foarte înalt, conform bunelor practici din securitatea modernă.

Unul dintre cele mai critice aspecte de securitate în orice aplicație web modernă este modul în care sunt stocate parolele utilizatorilor. Salvarea acestora în forma lor originală, lizibilă, este o vulnerabilitate fundamentală care, în cazul unei breșe de securitate, ar expune imediat toate conturile. Pentru a contracara acest risc, aplicația implementează un mecanism de hashing robust, fundamentat pe standarde criptografice verificate.

Funcționalitățile de hashing sunt furnizate de biblioteca **bcrypt**, o soluție modernă și foarte sigură, recunoscută ca standard de facto pentru protecția parolilor. Spre deosebire de funcțiile de hash rapide, precum `SHA-256`, concepute pentru viteză și inadecvate pentru parole, **bcrypt** este un algoritm **adaptiv**, proiectat să fie intenționat lent și costisitor din punct de vedere computațional. Această caracteristică, denumită factor de lucru (*work factor*), poate fi ajustată pentru a contracara creșterea puterii de calcul a hardware-ului modern, menținând astfel

relevanța sa pe termen lung. Securitatea oferită de **bcrypt** se bazează pe doi piloni: utilizarea unei valori "salt" și un "factor de lucru" adaptiv.

- **Salt-ul (sarea)** este un șir de date aleatoriu, generat automat de funcția `bcrypt.gensalt()` pentru fiecare parolă în parte. Acesta este combinat cu parola înainte de hashing, asigurând că două parole identice vor avea hash-uri finale complet diferite. Această tehnică anulează eficiența atacurilor bazate pe tabele pre-calulate, cum ar fi **rainbow tables**.
- **Factorul de lucru (work factor)** este caracteristica adaptivă a **bcrypt**. Acesta controlează complexitatea computațională, determinând numărul de iterații interne ale algoritmului. Un factor de lucru mai mare înseamnă un timp de calcul mai lung pentru fiecare hash. Acest "cost" computațional ridicat face atacurile de tip **brute-force** extrem de lente și, prin urmare, nefezabile din punct de vedere economic și temporal pentru un atacator.

În practică, biblioteca **bcrypt** abstractizează elegant această complexitate. Funcția `bcrypt.hashpw()`, combinată cu `bcrypt.gensalt()`, gestionează automat generarea salt-ului și aplicarea factorului de lucru, producând la final un singur șir de caractere. Acest hash rezultat încorporează atât algoritmul, cât și salt-ul și factorul de lucru. Ulterior, la autentificare, funcția `bcrypt.checkpw()` utilizează aceste informații. Ea extrage automat salt-ul din hash-ul stocat în baza de date pentru a procesa parola introdusă de utilizator și a valida corectitudinea acesteia. Implementarea acestui mecanism este, așadar, esențială pentru integritatea sistemului și pentru încrederea utilizatorilor în platformă.

Procesul de Înregistrare

Când un utilizator își creează un cont nou, parola introdusă de la frontend este supusă imediat unui proces robust de hashing. Funcția cheie utilizată este `bcrypt.hashpw()` din biblioteca **bcrypt**.

- **Codificarea parolei:** Inițial, parola este un șir de caractere (`string`). Deoarece **bcrypt** operează pe date binare, parola este mai întâi codificată în format UTF-8 folosind `password.encode('utf-8')`.
- **Generarea unui Salt Unic:** Funcția `bcrypt.gensalt()` este apelată pentru a genera o valoare "salt" (sare) unică și aleatorie. Acest salt este vital pentru securitate, deoarece asigură că două parole identice (aparținând unor utilizatori diferiți) vor avea hash-uri finale complet diferite, făcând astfel ineficiente atacurile de tip **rainbow table**.
- **Funcția `bcrypt.hashpw()`:** Această funcție centralizează procesul, combinând parola codificată binar cu salt-ul unic generat. Rezultatul este un hash securizat care încorporează salt-ul direct în structura sa. Această valoare finală, auto-conținută, este cea care se stochează în siguranță în baza de date.

Principiul fundamental al acestui mecanism este că **doar hash-ul calculat este stocat** în baza de date, niciodată parola în forma sa originală. Acest lucru asigură că, în scenariul unei **breșe de securitate** care ar compromite baza de date, atacatorii ar avea acces doar la aceste șiruri de caractere criptografice. Deși obținute, hash-urile sunt concepute pentru a fi **practic ireversibile** din punct de vedere computațional, protejând astfel în mod eficient credențialele reale ale utilizatorilor. A.6

Codul implementează atât mecanisme de securizare a parolei prin **bcrypt**, cât și validări pentru unicitatea numelui de utilizator și a adresei de email, prevenind astfel crearea conturilor duplicate și menținând consistența datelor din sistem.

Procesul de autentificare

În momentul autentificării, parola introdusă de utilizator este comparată cu hash-ul corespunzător, stocat anterior în baza de date.

Verificarea se realizează prin apelul funcției `bcrypt.checkpw(password.encode('utf-8'), account['password'].encode('utf-8'))`, care compară parola furnizată (după codificare în format binar) cu hash-ul existent. Această metodă este esențială pentru securitatea procesului de autentificare, garantând că parola nu este procesată într-un format nesecurizat. A.7

Fragmentul de cod asociat rutei `/login` implementează logica de verificare a credențialelor și, în cazul unei autentificări reușite, actualizează obiectul `session` cu informații esențiale despre utilizator, precum identificatorul unic (ID), numele de utilizator și rolul acestuia. În paralel, ruta `/logout` are rolul de a invalida sesiunea curentă, eliminând datele asociate autentificării.

4.2.4 Mecanismul de Gestiune a Sesiunii

Atunci când un utilizator se autentifică, informațiile din obiectul `session` (cum ar fi ID-ul, numele de utilizator și drepturile de acces) sunt trimise către browser sub forma unui cookie. Acest cookie nu este expus direct, ci este **semnat criptografic** de server, folosind cheia secretă definită în aplicație (`app.secret_key`), ceea ce previne falsificarea sau manipularea lui.

Mecanismul criptografic folosit asigură două aspecte esențiale:

- **Integritatea Datelor:** Previne orice tentativă de manipulare a datelor din cookie de către utilizator (de exemplu, un utilizator care încearcă să își schimbe singur rolul în `is_admin = True`). Orice modificare ar invalida semnătura.
- **Autenticitatea:** Garantează că respectivul cookie provine de la serverul nostru și nu a fost falsificat de o terță parte.

Confidențialitatea valorii `app.secret_key` este esențială, deoarece compromiterea acesteia ar permite unui atacator să genereze sesiuni autentice false pentru orice utilizator. A.8

Momentul cheie în care sesiunea este creată este imediat după o autentificare reușită. Urătorul fragment de cod, extras din ruta `/login`, arată exact ce date sunt stocate în sesiune. A.9

Odată ce cheia `'user_id'` este prezentă în sesiune, aplicația consideră utilizatorul ca fiind autentificat. Securizarea rutelor protejate se simplifică la o singură verificare: dacă un utilizator neautentificat (a cărui sesiune nu conține `'user_id'`) încearcă să acceseze o astfel de rută, el este redirecționat automat către pagina de autentificare, prevenind astfel accesul neautorizat la resursele protejate.

4.2.5 Interacțiunea cu Baza de Date

Operațiunile de înregistrare și autentificare se bazează pe interacțiunea cu baza de date pentru a asigura persistența și regăsirea datelor utilizatorilor.

Crearea unui Utilizator Nou (Operațiunea INSERT)

La înregistrarea unui utilizator nou, informațiile acestuia (nume de utilizator, email și parola deja procesată prin hashing) sunt salvate în tabelul `users` printr-o comandă `INSERT`. Folosirea placeholder-elor (`%s`) în locul concatenării directe a valorilor în interogare este necesară, deoarece reprezintă metoda standard prin care driver-ul bazei de date previne atacurile de tip **SQL injection**. A.11

Regăsirea Datelor la Autentificare (Operațiunea SELECT)

În timpul autentificării, după ce utilizatorul introduce numele de utilizator și parola, se execută o interogare `SELECT` pentru a regăsi înregistrarea corespunzătoare din baza de date. Scopul este de a compara hash-ul parolei stocate cu cel generat din parola introdusă. Dacă interogarea returnează un rezultat, sunt preluate și alte informații relevante, precum ID-ul și rolul utilizatorului, necesare pentru crearea sesiunii. A.10

Terminarea Sesiunii Utilizatorului

Ruta `/logout` asigură încheierea sigură și ordonată a sesiunii utilizatorului autentificat. Deși implementarea este concisă, ea este extrem de eficientă și robustă. Funcția asociată acestei rute execută următoarele acțiuni:

- **Invalidarea Completă a Sesiunii:** Logica fundamentală constă în apelarea metodei `session.clear()`. Această funcție golește complet dicționarul de sesiune, eliminând dintr-o singură operațiune toate cheile stocate la autentificare (precum `user_id`, `username` și `is_admin`). Abordarea este robustă și asigură că nicio dată reziduală de sesiune nu este lăsată accidental în urmă.
- **Feedback pentru Utilizator:** După golirea sesiunii, se utilizează funcția `flash()` pentru a trimite un mesaj de confirmare explicit ("Deconectare reușită."), informând utilizatorul că acțiunea s-a finalizat cu succes.
- **Redirecționare:** La final, utilizatorul este redirecționat către pagina de login folosind `redirect(url_for('login'))`, fiind ghidat în mod logic către următorul pas posibil.

Prin ștergerea completă a datelor din sesiune, ruta `/logout` garantează că nicio acțiune ulterioară nu poate fi realizată în numele utilizatorului deconectat, fiind o componentă esențială a arhitecturii de securitate a aplicației. A.12

4.3 Pagina Principală (Homepage)

Pagina principală, servită de ruta rădăcină `/`, reprezintă punctul central de intrare în aplicație. Rolul său este de a oferi o primă impresie vizuală, de a facilita navigarea și de a prezenta o listă relevantă de proprietăți disponibile.

4.3.1 Prezentarea Frontend (home.html)

Interfața paginii principale este structurată în jurul a trei componente cheie, care sunt randate dinamic folosind motorul de template-uri Jinja2.

1. **Bara de Navigație Dinamică:** Aceasta este o componentă esențială pentru experiența utilizatorului. Conținutul său se adaptează în funcție de starea de autentificare a utilizatorului.
 - Pentru un **utilizator neautentificat** (figura 4.3), bara afișează link-uri către paginile de `/login` și `/register`.
 - Pentru un **utilizator autentificat** (figura 4.4), link-urile de autentificare sunt înlocuite cu opțiuni specifice contului: un buton pentru a adăuga o proprietate nouă, un meniu dropdown cu acces rapid către profil, rezervări și proprietățile personale, și un buton de deconectare (`/logout`). Această logică condițională este implementată în template folosind un bloc `{% if logged_in %}`.

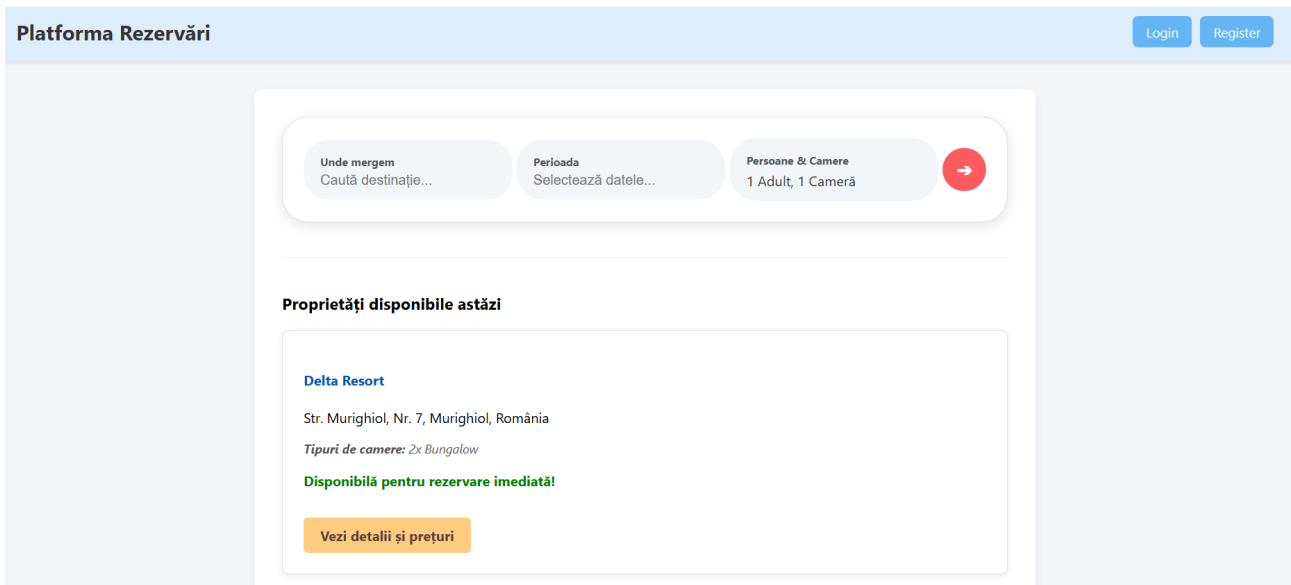


Figura 4.3: Pagina home - utilizator neautentificat

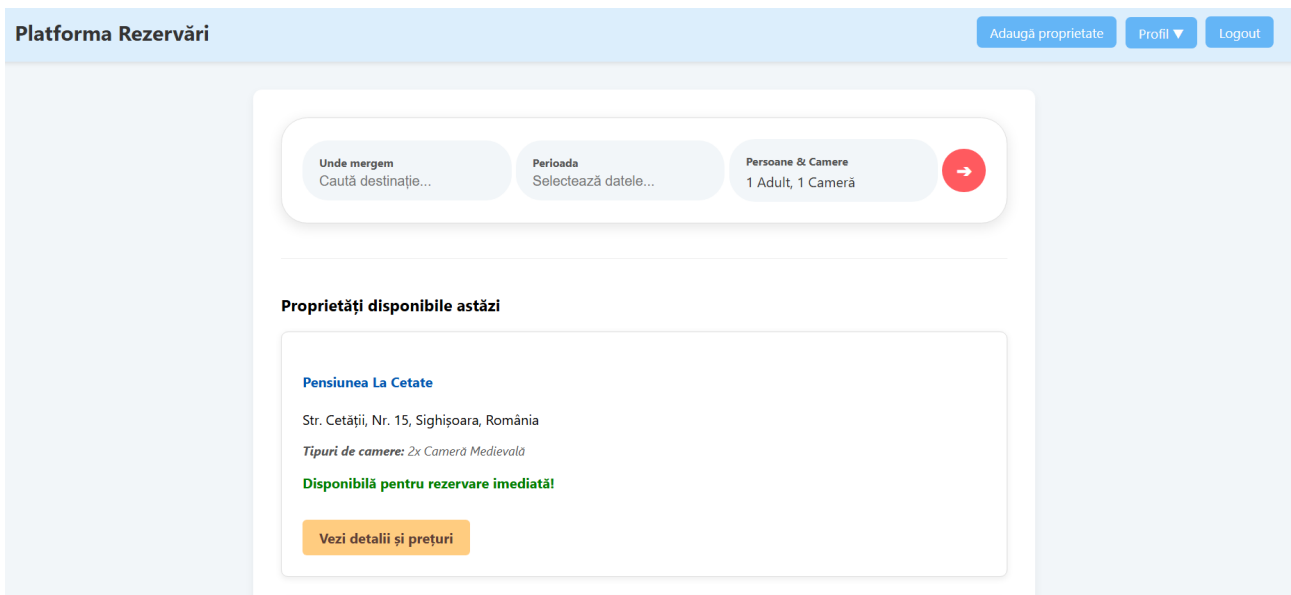


Figura 4.4: Pagina home - utilizator autentificat

2. **Componenta de Căutare:** Deși detaliile logicii de căutare sunt prezentate în altă secțiune, prezența formularului de căutare în partea superioară a paginii este o decizie de design deliberată, pentru a oferi un "call-to-action" clar și imediat pentru vizitatori.
3. **Zona de Afișare a Proprietăților:** Sub bara de căutare, pagina prezintă o listă de proprietăți sub formă de carduri. Fiecare card afișează informații esențiale precum numele proprietății, locația și un sumar al tipurilor de camere disponibile. Pentru a crește relevanța, aplicația indică în mod proeminent dacă o proprietate este disponibilă pentru rezervare imediată.

Fragmentul de cod HTML ilustrează implementarea unei bare de navigație dinamice, adaptată în funcție de starea de autentificare a utilizatorului. A.13

4.3.2 Logica Backend (Ruta / în `app.py`)

Funcția `home()` din backend este responsabilă cu pregătirea datelor pentru a fi afișate pe pagina principală.

- **Preluarea Proprietăților:** Se execută o interogare SQL `SELECT` care folosește o clauză `JOIN` pentru a uni tabela `properties` cu tabela `users`, permițând astfel afișarea numelui de utilizator al proprietarului lângă fiecare cazare.
- **Personalizarea Conținutului:** Logica implementează o caracteristică importantă de personalizare: dacă un utilizator este autentificat, interogarea este modificată dinamic printr-o clauză `WHERE` pentru a exclude proprietățile deținute de utilizatorul respectiv din lista principală, evitând astfel ca un proprietar să își vadă propriile oferte ca sugestii de cazare.

Datele colectate și filtrate sunt apoi trimise către template-ul `home.html` pentru a fi randate dinamic, conform descrierii de mai sus.

Fragmentul de cod `Python` de mai jos ilustrează logica de personalizare și de preluare a datelor din backend: A.14

4.4 Pagina de Profil: Dashboard-ul Personalizat al Utilizatorului

Pagina de profil, accesibilă prin ruta `/profile`, reprezintă un centru de comandă personalizat pentru fiecare utilizator autentificat. Aceasta nu este o pagină statică, ci un "dashboard" interactiv care îi permite utilizatorului să își gestioneze toate activitățile pe platformă.

4.4.1 Prezentarea Frontend (`profile.html`)

Interfața web este proiectată pentru a afișa date agregate într-un mod structurat și intuitiv, folosind motorul de template-uri Jinja2. Pagina întâmpină utilizatorul cu un mesaj de bun venit personalizat și prezintă două carduri principale interactive:

- **Cardul "Proprietățile Mele":** Afișează numărul total de proprietăți deținute de utilizator. Acest număr este calculat dinamic în template folosind filtrul `|length` aplicat listei de proprietăți primite de la backend. Cardul conține un buton care direcționează utilizatorul către o pagină dedicată gestionării proprietăților sale.
- **Cardul "Rezervările Mele":** În mod similar, afișează un sumar al rezervărilor efectuate și oferă un link către o pagină cu istoricul detaliat al acestora.

Mai jos de aceste carduri, pagina listează în detaliu proprietățile și rezervările, permițând acțiuni specifice pentru fiecare.

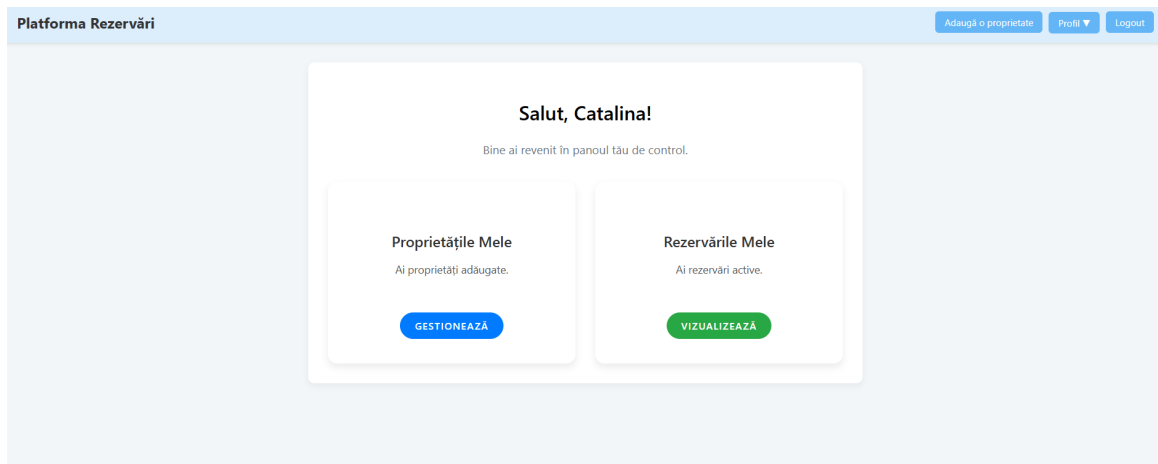


Figura 4.5: Interfața paginii de profil, funcționând ca un panou de control.

Un scurt fragment HTML din `profile.html` care ilustrează afișarea dinamică a numelui: A.16

4.4.2 Logica Backend (Ruta `/profile` în `app.py`)

Pentru a popula dinamic aceste secțiuni ale interfeței, funcția asociată rutei `/profile` are un rol esențial de **agregator de date**. Folosind ID-ul utilizatorului stocat în `session`, funcția execută mai multe interogări `SELECT` specializate către baza de date:

1. O interogare pentru a prelua detaliile personale ale utilizatorului din tabelul `users`.
2. O a doua interogare pentru a extrage toate proprietățile pe care utilizatorul le deține (`owner_id`).
3. O a treia interogare, mai complexă, care implică operațiuni de `JOIN` între tabelele `reservations`, `rooms` și `properties`, pentru a lista detaliile complete ale rezervărilor făcute de utilizator.

Toate aceste seturi de date sunt apoi transmise ca și context la randarea template-ului `profile.html`. Fragmentul de cod de mai jos arată logica din spatele acestei rute. A.15

4.5 Modificarea Datelor de Profil

Funcționalitatea de editare a profilului permite utilizatorilor să își actualizeze informațiile personale. Acest proces este orchestrat de ruta `/edit-profile`, care gestionează atât afișarea formularului, cât și procesarea securizată a datelor modificate.

4.5.1 Prezentarea Frontend (`edit-profile.html`)

Interfața de editare constă într-un formular HTML care, la o cerere de tip `GET`, este pre-populat cu datele curente ale utilizatorului (nume de utilizator, email), extrase din obiectul `user` trimis de backend. Formularul este împărțit logic în mai multe secțiuni:

- **Date de Bază:** Câmpuri pentru modificarea numelui de utilizator și a adresei de email.
- **Schimbarea Parolei (Opțional):** O secțiune dedicată actualizării parolei, care conține câmpuri pentru noua parolă și confirmarea acesteia. Aceste câmpuri sunt opționale.

- **Verificarea Identității:** Un câmp obligatoriu pentru introducerea **parolei curente**. Această măsură de securitate asigură că doar proprietarul contului, care cunoaște parola actuală, poate efectua modificări asupra datelor sensibile.

La trimitere, datele sunt transmise serverului printr-o cerere de tip POST.

Figura 4.6: Pagina Editează Profilul

4.5.2 Logica Backend (Ruta /edit-profile în app.py)

Logica din spatele acestei interfețe, gestionată la o cerere de tip POST, este un proces multi-etapă care prioritizează securitatea și integritatea datelor:

1. **Verificarea Identității:** Primul și cel mai important pas este validarea parolei curente. Aplicația preia hash-ul parolei utilizatorului din baza de date și folosește `bcrypt.checkpw()` pentru a verifica dacă parola introdusă în formular corespunde. Dacă verificarea eșuează, procesul este oprit imediat, iar utilizatorul este informat printr-un mesaj de eroare.
2. **Validarea Unicității Datelor Noi:** Înainte de a efectua orice modificare, se execută interogări SQL pentru a verifica dacă noul nume de utilizator sau noua adresă de email nu sunt deja folosite de un alt cont, prevenind astfel crearea de duplicate.
3. **Hashing-ul Noii Parole:** Dacă utilizatorul a completat câmpurile pentru o parolă nouă (și dacă aceasta se potrivește cu confirmarea), parola este procesată prin `bcrypt.hashpw()` pentru a genera un nou hash securizat.
4. **Actualizarea în Baza de Date:** Se construiește și se execută dinamic o comandă SQL UPDATE care modifică înregistrarea utilizatorului în tabelul `users`. Această abordare este eficientă, actualizând doar câmpurile care au fost efectiv modificate de utilizator.

5. **Actualizarea Sesiunii:** Dacă numele de utilizator a fost schimbat, se actualizează și valoarea din `session['username']` pentru a reflecta imediat modificarea în interfața aplicației (de exemplu, în bara de navigație).

La finalul procesului, utilizatorul este redirecționat către pagina de profil, însoțit de un mesaj de succes.

Fragmentul de cod de mai jos prezintă implementarea logicii rutei `/edit-profile`: A.17

4.6 Panoul de Administrare

Panoul de administrare, accesibil prin ruta `/admin`, este o componentă cu acces restricționat, proiectată pentru a oferi o imagine de ansamblu asupra întregii platforme. Acesta reprezintă un instrument de monitorizare și analiză, nu unul de gestiune personală.

Diferența față de Profilul de Utilizator

Spre deosebire de pagina de profil a unui utilizator normal, care afișează date filtrate și relevante doar pentru acel utilizator (ex: "rezervările mele", "proprietățile mele"), panoul de administrare operează la un nivel global. Interogările sale nu sunt condiționate de `user_id`-ul din sesiune, ci rulează pe întreg setul de date pentru a calcula indicatori cheie de performanță (KPIs) și statistici agregate la nivel de sistem. Accesul la această rută este, prin urmare, cea mai importantă funcționalitate protejată de rolul de administrator.

4.6.1 Prezentarea Frontend (`admin.html`)

Interfața panoului de administrare este concepută ca un "dashboard" de business intelligence, prezentând administratorului o imagine clară a stării curente a platformei. Componentele vizuale includ:

- **Carduri de Statistici:** Afișarea unor date agregate esențiale, precum: numărul total de utilizatori, numărul total de proprietăți înregistrate și numărul de rezervări active sau anulate.
- **Analiza Ocupanței:** Un formular interactiv cu un selector de perioadă (*datepicker*) care permite administratorului să aleagă un interval de date. La trimitere, backend-ul calculează și afișează numărul de camere ocupate și disponibile în acea perioadă, oferind o perspectivă valoroasă asupra gradului de ocupare a platformei.
- **Indicatori status servere:**
 1. **Status Server Flask:** Indică starea serverului de aplicații pe care rulează logica Python.
 2. **Status Server IIS:** Indică starea unui potențial server de producție (Internet Information Services), pe care aplicația ar putea fi implementată într-un scenariu real.

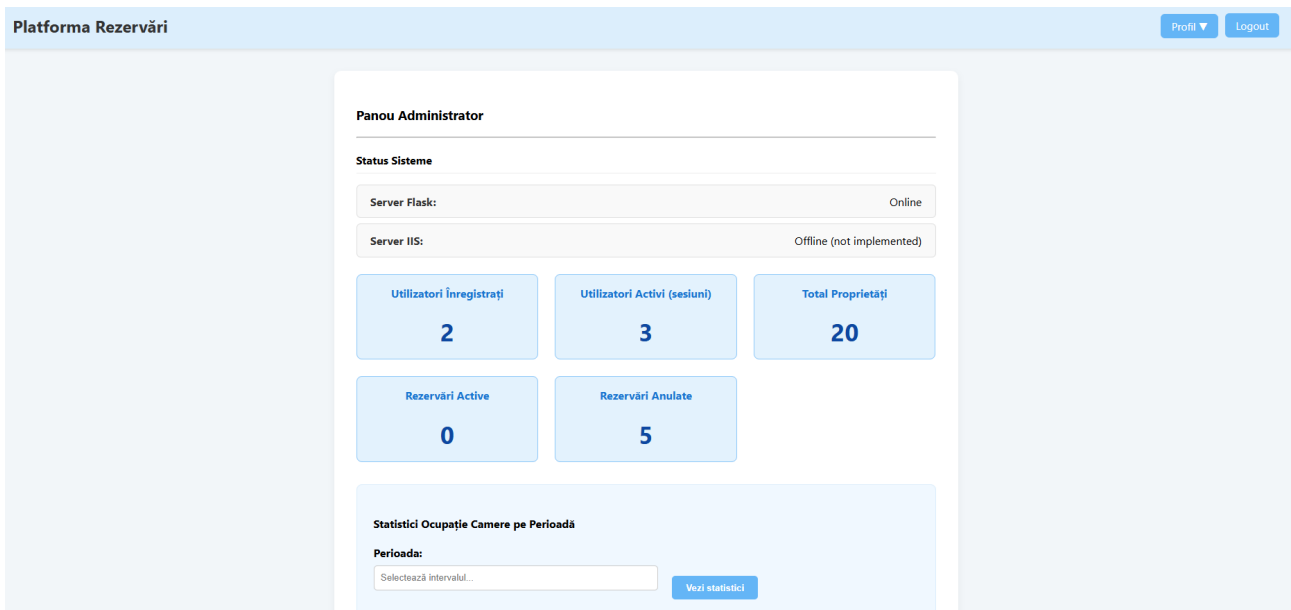


Figura 4.7: Panou Administrator

4.6.2 Logica Backend (Ruta /admin în app.py)

Funcția `admin()` din backend este protejată de un mecanism de control al accesului și are un rol intensiv de agregare de date.

1. **Controlul Accesului Bazat pe Rol:** Primul pas executat este verificarea `if not session.get('is_admin')`. Dacă utilizatorul logat nu are rolul de administrator (valoarea `'is_admin'` din sesiune nu este `'True'`), accesul este imediat refuzat cu un cod de status HTTP 403 (Forbidden).
2. **Agregarea Statisticilor Generale:** Se execută o serie de interogări SQL simple, de tipul `SELECT COUNT(*)`, pentru a calcula rapid statisticile de bază (total utilizatori, proprietăți, etc.).
3. **Calcularea Ocupanței la Cerere:** Dacă administratorul trimite un interval de date prin formular, funcția procesează aceste date și execută o interogare SQL mai complexă. Aceasta numără camerele distincte care au rezervări ce se suprapun cu intervalul selectat, pentru a calcula gradul de ocupare.
4. **Randarea Datelor:** Toate statisticile calculate sunt transmise către template-ul `admin.html` pentru a fi afișate.

Fragmentul de cod de mai jos prezintă implementarea mecanismul de protecție bazat pe rol a rutei `/admin`. A.18

4.7 Fluxul de Căutare și Recomandări Personalizate

Funcționalitatea de căutare este nucleul interactiv al platformei, permițând utilizatorilor să găsească și să rezerve cazare conform unor criterii specifice. Fluxul de căutare nu se limitează la o simplă filtrare, ci include un algoritm de recomandare care propune pachete optimizate de camere. Acest proces se desfășoară în mai multe etape, orchestrate de multiple rute și template-uri.

4.7.1 Etapa 1: Căutarea Inițială (de pe Homepage)

- **Frontend (home.html):** Punctul de plecare este formularul de căutare de pe pagina principală. Acesta colectează patru informații esențiale: destinația (nume sau oraș), perioada dorită (selectată dintr-un calendar interactiv *Litepicker*), numărul de adulți și numărul de camere dorite. La trimitere, datele sunt transmise prin metoda POST către ruta `/search_results`.

Figura 4.8: Formularul de autentificare a utilizatorilor

- **Backend (Ruta `/search_results`):** Această funcție din `app.py` acționează ca un prim filtru. Ea preia criteriile și execută o interogare SQL pentru a identifica toate proprietățile care se potrivesc destinației. Ulterior, pentru fiecare proprietate candidată, execută o a doua interogare complexă pentru a determina dacă există camere disponibile în perioada selectată. În acest punct, se apelează funcția de recomandare `get_recommendations()` pentru a valida dacă se poate construi cel puțin un pachet de cazare valid. Doar proprietățile care trec de acest filtru avansat sunt trimise către frontend pentru afișare.

4.7.2 Etapa 2: Afișarea Rezultatelor (`search_results.html`)

- **Frontend:** Acest template primește lista de proprietăți eligibile și le afișează sub formă de carduri. Un detaliu tehnic crucial aici este modul în care se construiește link-ul pentru fiecare rezultat. Link-ul către pagina de vizualizare a proprietății (`view_property`) include, pe lângă `property_id`, și toți parametrii căutării inițiale (perioadă, adulți, camere). Acest lucru permite transmiterea contextului căutării către următoarea etapă.

Figura 4.9: Rezultatele căutării

Fragmentul de cod HTML prezintă un șablon Jinja utilizat pentru afișarea rezultatelor unei căutări. Pentru fiecare proprietate returnată de backend, se generează dinamic un link care include în URL parametrii inițiali ai căutării (perioadă, număr de persoane, camere). Această abordare permite păstrarea contextului căutării la navigarea către pagina individuală a fiecărei proprietăți. A.19

4.7.3 Etapa 3: Calculatorul de Disponibilitate și Recomandări (view_property.html)

- **Frontend:** Pagina de vizualizare a proprietății este componenta centrală a fluxului.
 1. **Calculatorul:** În partea de sus, conține un formular ("calculator") care este pre-populat automat cu parametrii primiți de la pagina de rezultate. Utilizatorul poate modifica aceste criterii (ex: altă perioadă, alt număr de persoane) și poate re-trimite formularul către aceeași rută, pentru a recalcula oferta.
 2. **Afișarea Recomandărilor:** Sub calculator, pagina afișează o listă de pachete de cazare personalizate, generate de backend. Dacă nu există o căutare activă, pagina afișează o listă generică a tuturor tipurilor de camere disponibile la acea proprietate.

The screenshot displays the 'Platforma Rezervări' interface. At the top, there's a navigation bar with 'Adaugă proprietate', 'Profil', and 'Logout' buttons. The main content area shows details for 'Pensiunea La Cetate' located at 'Str. Cetății, Nr. 15, Sighișoara, România'. Below the address is a description: 'Situată în inima cetății medievale, pensiunea oferă o incursiune în istorie.' A search section titled 'Găsește cazarea perfectă' contains input fields for 'Perioada:' (2025-06-09 to 2025-06-10), 'Persoane:' (1), and 'Camere:' (1), along with a 'Caută / Actualizează' button. Underneath, it shows 'Opțiuni de rezervare pentru 1 persoane'. A highlighted box labeled 'Opțiunea optimă (1 camere)' lists '1x Cameră Medievală (capacitate 2)' with a price of '380.00 lei / noapte' and a 'Rezervă Pachetul' button.

Figura 4.10: Formularul de autentificare a utilizatorilor

- **Backend (Ruta /property/<id>):** Această funcție este cea mai complexă. Ea primește parametrii de căutare și execută logica finală: extrage toate camerele disponibile pentru proprietatea și perioada specificate, apoi apelează algoritmul de recomandare `get_recommendations()` pentru a construi și sorta pachetele optime. Aceste pachete, împreună cu detaliile proprietății, sunt trimise către template pentru a fi afișate.

4.7.4 Algoritmul de Recomandare (`get_recommendations()`)

Această funcție Python reprezintă nucleul inteligent al sistemului de căutare. Ea primește ca intrare o listă a camerelor disponibile și cerințele utilizatorului. Folosind

`itertools.combinations`, generează eficient toate combinațiile posibile de camere și le filtrează pe cele care satisfac capacitatea totală necesară. La final, sortează combinațiile valide pe baza unui scor calculat, care prioritizează numărul de camere cerut, apoi capacitatea totală minimă care satisface cerința și, în ultimul rând, prețul cel mai mic. Funcția returnează cele mai bune 5 pachete, oferind utilizatorului opțiuni optimizate. A.20

4.8 Adăugarea unei Proprietăți Noi

O funcționalitate esențială pentru proprietari este posibilitatea de a adăuga noi unități de cazare în platformă. Acest proces este gestionat de ruta `/add_property` și implică o interfață dinamică și o logică de backend capabilă să proceseze date structurate complex.

4.8.1 Prezentarea Frontend (`add_property.html`)

Interfața pentru adăugarea unei proprietăți este un formular multi-etapă, care permite definirea atât a detaliilor generale ale proprietății, cât și a diverselor tipuri de camere disponibile în cadrul acesteia.

- **Date Generale:** Prima parte a formularului conține câmpuri text standard pentru numele proprietății, adresă, oraș și țară.

Figura 4.11: Formularul de autentificare a utilizatorilor

- **Gestiune Dinamică a Camerelor:** Componenta cea mai complexă a interfeței este secțiunea de adăugare a camerelor. Utilizatorul nu este limitat la un număr fix de tipuri de camere. Printr-un buton "+ Adaugă un tip de cameră", se execută o funcție **JavaScript** care adaugă dinamic în DOM un nou set de câmpuri. Fiecare set permite definirea unui tip de cameră distinct (ex: "Cameră Dublă Standard"), împreună cu atributele sale specifice: capacitate, preț/noapte, numărul de camere identice de acel tip și disponibilitatea generală. Utilizatorul poate adăuga oricâte tipuri de camere dorește și le poate șterge individual.

Această abordare dinamică oferă o flexibilitate maximă proprietarilor, permițându-le să modeleze cu acuratețe structura oricărei unități de cazare.

Figura 4.12: Formularul de autentificare a utilizatorilor

4.8.2 Logica Backend (Ruta `/add_property` în `app.py`)

La trimiterea formularului, backend-ul primește datele printr-o cerere `POST`. Procesarea acestor date este o operațiune tranzacțională complexă:

1. **Inserarea Proprietății Principale:** Prima comandă SQL `INSERT` adaugă detaliile generale ale proprietății în tabelul `properties`. ID-ul proprietarului este preluat din sesiunea curentă. Imediat după inserare, se obține `property_id`-ul noii înregistrări folosind `cursor.lastrowid`.
2. **Procesarea Datelor Dinamice:** Datele pentru tipurile de camere sunt primite de la formular sub formă de liste. Se utilizează funcția `request.form.getlist()` pentru a extrage toate valorile pentru nume, capacitate, preț, etc.
3. **Inserarea Camerelor (One-to-Many):** Aplicația iterează prin lista de tipuri de camere primite. Pentru fiecare tip, se execută o buclă imbricată care rulează de un număr de ori egal cu "Număr de camere de acest tip" specificat de utilizator. În interiorul acestei bucle, se execută o comandă SQL `INSERT` pentru a adăuga fiecare cameră individuală în tabelul `rooms`, legând-o de proprietatea mamă prin `property_id`-ul obținut la primul pas.
4. **Finalizarea Tranzacției:** Toate operațiunile de inserare (atât pentru proprietate, cât și pentru toate camerele asociate) sunt învelite într-o singură tranzacție. La finalul procesării, se apelează `cnx.commit()` pentru a salva atomic toate modificările în baza de date. Dacă apare orice eroare, întreaga tranzacție este anulată.

Fragmentul de cod de mai jos, extras din ruta `/add_property`, ilustrează logica de procesare a datelor dintr-un formular dinamic, incluzând inserarea în tabele multiple. A.21

4.9 Gestiunea Proprietăților Personale

Pagina "Cazările mele", accesibilă prin ruta `/my-properties`, este un panou de control dedicat utilizatorilor care au rolul de proprietari. Această secțiune le permite să vizualizeze și să gestioneze toate unitățile de cazare pe care le-au înregistrat pe platformă.

4.9.1 Prezentarea Frontend (`my-properties.html`)

Interfața acestei pagini este proiectată pentru a oferi o imagine de ansamblu clară și acces rapid la acțiuni de management.

- **Listarea Proprietăților:** Componenta centrală este o listă de carduri, unde fiecare card reprezintă o proprietate deținută de utilizatorul curent. Informațiile de bază, precum numele și adresa proprietății, sunt afișate proeminent.
- **Acțiuni de Management:** Fiecare card de proprietate include un set de butoane de acțiune care permit utilizatorului să interacționeze direct cu înregistrarea respectivă:
 - **Editează:** Un link către ruta `/edit_property/<id>`, care deschide formularul de editare pentru acea proprietate specifică.
 - **Vezi și gestionează rezervările:** Un link către o pagină dedicată unde proprietarul poate vedea toate rezervările (active sau viitoare) făcute de alți utilizatori pentru proprietatea sa.
 - **Șterge:** Un buton care inițiază acțiunea de ștergere a proprietății. Pentru a preveni acțiunile accidentale, acest link include un dialog de confirmare JavaScript (`onclick="return confirm(...)"`).
- **Stare Goală (Empty State):** În cazul în care un utilizator nu a adăugat încă nicio proprietate, pagina afișează un mesaj informativ și un link direct către pagina de adăugare, ghidând astfel utilizatorul către pasul următor.

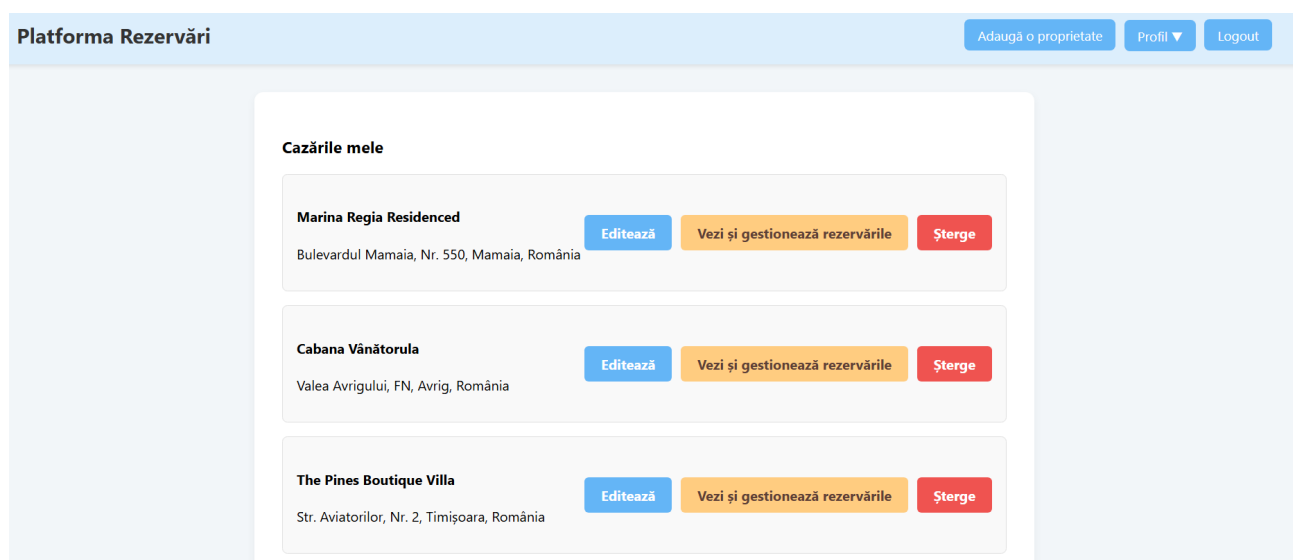


Figura 4.13: Formularul de autentificare a utilizatorilor

4.9.2 Logica Backend (Ruta /my-properties în app.py)

Funcția din backend care deservește această pagină este simplă, dar esențială pentru securitatea datelor.

1. **Protecția Rutei:** Prima acțiune este verificarea dacă un utilizator este autentificat. Dacă nu, acesta este redirecționat către pagina de login.
2. **Filtrarea Datelor pe Baza Proprietarului:** Se execută o comandă SQL `SELECT` pe tabela `properties`. Clauza fundamentală a acestei interogări este `WHERE owner_id = %s`, unde valoarea pentru placeholder este preluată din `session['user_id']`. Acest filtru este critic, deoarece asigură că un utilizator poate vizualiza și gestiona **doar și numai** proprietățile pe care le deține, garantând o separare strictă a datelor între conturi.
3. **Randarea Template-ului:** Lista de proprietăți rezultată este trimisă către template-ul `my_properties.html` pentru a fi afișată dinamic.

Fragmentul de cod Python demonstrează simplitatea și eficiența logicii de backend, în timp ce fragmentul HTML arată cum sunt generate dinamic lista de proprietăți și butoanele de acțiune. A.22

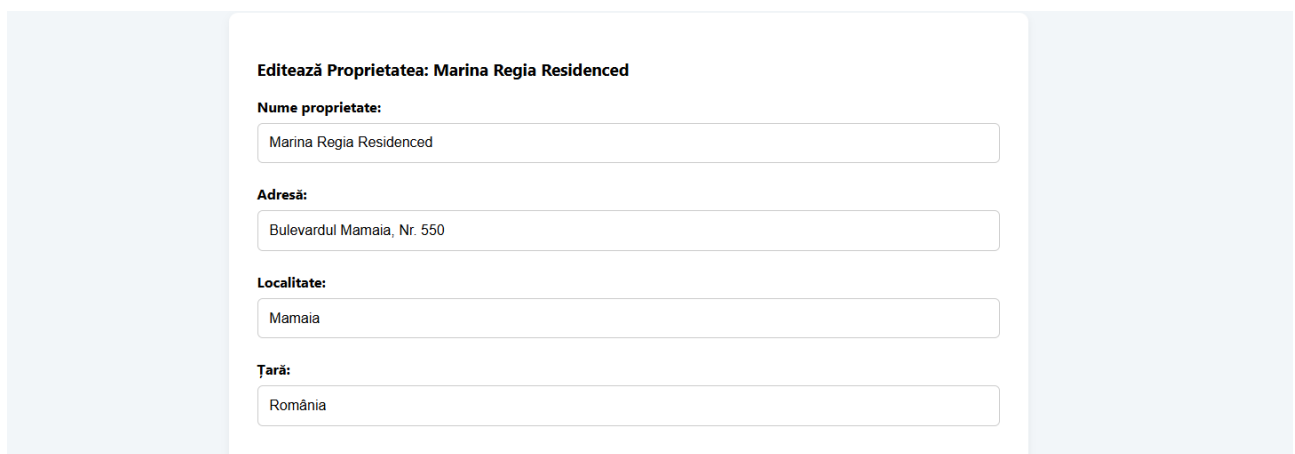
4.10 Modificarea unei Proprietăți Existente

Funcționalitatea de editare a unei proprietăți este una dintre cele mai complexe din aplicație, oferind proprietarilor un control granular asupra unităților de cazare listate. Procesul este orchestrat de ruta dinamică `/edit_property/<property_id>` și implică o interfață interactivă, validări asincrone și o logică tranzacțională robustă în backend.

4.10.1 Prezentarea Frontend (`edit_property.html`)

Interfața de editare este un formular avansat, care la încărcare este pre-populat cu toate datele existente ale proprietății și ale camerelor asociate.

- **Date Generale:** Prima parte a formularului conține câmpurile pentru detaliile proprietății (nume, adresă etc.), permițând modificarea acestora.



Editează Proprietatea: Marina Regia Residenced

Nume proprietate:

Adresă:

Localitate:

Țară:

Figura 4.14: Formularul de autentificare a utilizatorilor

- **Gestiune Dinamică a Camerelor:** Componenta centrală este secțiunea de management al camerelor. Tipurile de camere existente sunt afișate în grupuri distincte, fiecare cu propriile atribute (preț, capacitate, număr). Utilizatorul poate modifica aceste atribute, poate adăuga tipuri de camere complet noi folosind butonul "+ **Adaugă un tip nou de cameră**" (care declanșează o funcție JavaScript), sau poate șterge un tip de cameră existent.

Figura 4.15: Formularul de autentificare a utilizatorilor

- **Validare Asincronă la Ștergere:** O caracteristică notabilă este mecanismul de ștergere a unui tip de cameră. La apăsarea butonului de ștergere, se execută o funcție JavaScript care trimite o cerere **asincronă (AJAX/Fetch)** către un endpoint dedicat din backend (`/check_room_type_reservations/...`). Acest endpoint verifică în timp real dacă există rezervări active pentru acel tip de cameră. Doar dacă nu există rezervări, interfața permite eliminarea vizuală a grupului de câmpuri, prevenind erorile și oferind un feedback instantaneu.

4.10.2 Logica Backend (Rutele `/edit_property` și `/check_room_type...`)

Backend-ul gestionează acest proces prin două rute distincte:

1. Ruta principală `/edit_property/<id>`:

- **La cerere GET:** Verifică dacă utilizatorul logat este proprietarul de drept al resursei cerute. Dacă da, extrage toate datele despre proprietate și camerele grupate și le trimite pentru a popula formularul din frontend.
- **La cerere POST:** Execută o logică tranzacțională complexă. Într-o singură tranzacție cu baza de date, execută comenzi `UPDATE` pentru detaliile proprietății și ale camerelor modificate, comenzi `INSERT` pentru tipurile de camere noi și comenzi `DELETE` pentru tipurile de camere eliminate. Dacă oricare dintre operațiuni eșuează, se execută un `ROLLBACK` pentru a menține consistența datelor.

2. **Ruta AJAX** `/check_room_type_reservations/...`: Acest endpoint specializat răspunde cererilor de la scriptul frontend. Primește un ID de proprietate și un nume de tip de cameră, interoghează baza de date pentru rezervări active și returnează un răspuns JSON care informează clientul dacă ștergerea este sigură.

Fragmentele de cod de mai jos ilustrează logica tranzacțională din backend. A.23

4.11 Ștergerea unei Proprietăți

Funcționalitatea de ștergere a unei proprietăți este o operațiune distructivă care necesită un grad înalt de securitate și validare pentru a preveni pierderea accidentală de date și pentru a menține integritatea bazei de date. Procesul este inițiat din interfața de management a proprietarului și este gestionat de o rută dedicată în backend.

4.11.1 Interfața Frontend (`my_properties.html`)

Pe pagina "Cazările mele", fiecare proprietate listată are un buton de "Șterge". Pentru a preveni acțiunile accidentale din partea utilizatorului, acest buton nu declanșează imediat ștergerea, ci implementează o măsură de siguranță pe partea de client:

- **Dialog de Confirmare JavaScript:** Butonul are un atribut `onclick` care execută funcția nativă a browser-ului `return confirm(...)`. Aceasta afișează o fereastră de dialog modală care cere utilizatorului o a doua confirmare explicită. Doar dacă utilizatorul apasă "OK", cererea de ștergere este trimisă către server. În caz contrar, acțiunea este anulată.

4.11.2 Logica Backend (Ruta `/delete_property/<id>` în `app.py`)

Backend-ul tratează ștergerea ca pe o operațiune securizată și tranzacțională, executând un set riguros de verificări înainte de a efectua orice modificare în baza de date.

1. **Verificarea Autentificării și a Dreptului de Proprietate:** Prima acțiune este de a valida dacă utilizatorul este autentificat. Apoi, se execută o interogare `SELECT` pentru a verifica dacă `user_id`-ul utilizatorului curent (stocat în sesiune) corespunde cu `owner_id`-ul proprietății care se dorește a fi ștearsă. Dacă nu există o potrivire, operațiunea este imediat oprită, prevenind astfel ca un utilizator să poată șterge proprietățile altuia.
2. **Validarea Regulilor de Business:** Se impune o regulă de business esențială: o proprietate cu rezervări active sau viitoare nu poate fi ștearsă. Se execută o a doua interogare `SELECT` pentru a căuta în tabela `reservations` rezervări cu statusul "confirmată" a căror dată de sfârșit este în viitor. Dacă se găsește cel puțin o astfel de rezervare, procesul este anulat, iar proprietarul este informat.
3. **Gestiunea Tranzacției și Ștergerea în Cascadă:** Dacă toate verificările de mai sus trec cu succes, se inițiază operațiunile de ștergere. Deși schema bazei de date are definite constrângeri de tip `ON DELETE CASCADE` pentru a propaga ștergerea automat, codul implementează explicit ștergerea datelor din tabelele dependente (`reservation_details`, `reservations`, `rooms`) înainte de a șterge înregistrarea finală din tabela `properties`. Toate aceste comenzi `DELETE` sunt executate într-o singură tranzacție. La final, se apelează `cnx.commit()` pentru a confirma toate ștergerile. În caz de eroare, întreaga tranzacție este anulată automat de context manager, menținând starea bazei de date consistentă.

Fragmentul de cod de mai jos prezintă implementarea completă a rutei `/delete_property`, evidențiind verificările de securitate și logica tranzacțională. A.24

4.12 Procesul de Rezervare Efectivă

Procesul de rezervare efectivă, de la alegerea unei proprietăți până la confirmarea finală, este un flux complex gestionat de mai multe rute și componente, asigurând integritatea datelor și o experiență clară pentru utilizator.

Căutarea și Inițierea Rezervării (`/search_results` și `/property/<id>`)

Fluxul începe cu afișarea rezultatelor căutării, unde utilizatorul poate alege o proprietate. Odată selectată proprietatea, pagina de vizualizare a acesteia (`/property/<id>`) permite inițierea procesului de rezervare prin selectarea perioadei și a numărului de persoane.

Prezentarea Frontend (`search_results.html` și `view_property.html`)

După o căutare inițială, pagina `search_results.html` afișează proprietățile relevante. Fiecare proprietate listată în aceste rezultate are un link dedicat care direcționează utilizatorul către pagina sa de vizualizare detaliată.

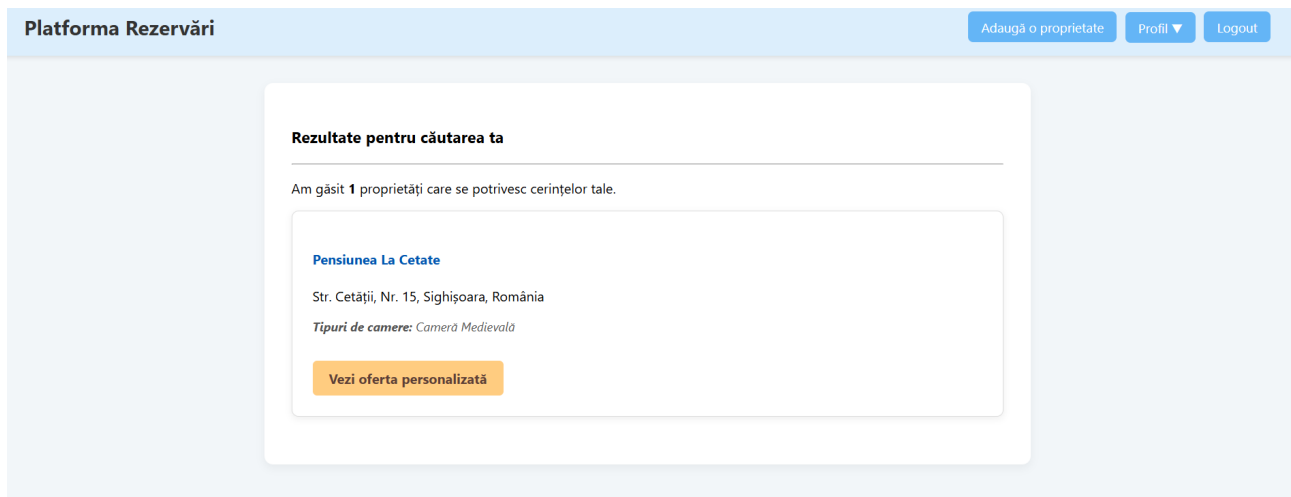


Figura 4.16: Lista cu proprietățile căutate

Pe pagina `view_property.html`, un "calculator" de disponibilitate este pre-populat automat cu parametrii căutării inițiale. Aici, utilizatorul poate vizualiza pachete de cazare recomandate sau poate ajusta criteriile pentru a recalcula oferta. Când utilizatorul apasă pe butonul de rezervare pentru un pachet, este redirecționat către ruta de confirmare.

Logica Backend (`/property/<id>` în `app.py`)

Funcția `view_property()` preia parametrii de căutare din URL sau din formular. Extrage camerele disponibile pentru proprietatea și perioada specificate, apoi apelează algoritmul de recomandare `get_recommendations()` pentru a construi și sorta pachetele optime. Aceste pachete, împreună cu detaliile proprietății, sunt trimise către template pentru afișare. Dacă nu se găsește nicio combinație, se afișează un mesaj informativ.

Platforma Rezervări Adaugă proprietate Profil ▼ Logout

Pensiunea La Cetate
Adresă: Str. Cetății, Nr. 15, Sighișoara, România

Descriere
 Situată în inima cetății medievale, pensiunea oferă o incursiune în istorie.

Găsește cazarea perfectă

Perioada: 2025-06-09 to 2025-06-10 **Persoane:** 1 **Camere:** 1 Caută / Actualizează

Opțiuni de rezervare pentru 1 persoane

★ **Opțiunea optimă (1 camere)**

- 1x Cameră Medievală (capacitate 2)

380.00 lei / noapte Rezervă Pachetul

Figura 4.17: Pagina de vizualizare a unei proprietăți

4.13 Confirmarea și Finalizarea Rezervării (/booking/confirm)

Odată ce utilizatorul a ales un pachet de camere, este redirecționat către pagina de rezervare, unde furnizează detaliile de contact și finalizează tranzacția.

4.13.1 Prezentarea Frontend (booking_confirmation.html)

Pagina de confirmare a rezervării prezintă un sumar detaliat al pachetului de camere selectat. Aceasta include perioada rezervării, numărul de nopți, detaliile fiecărei camere (nume, capacitate, preț per noapte) și prețul total de plată. De asemenea, conține un formular pentru introducerea datelor de contact ale clientului (nume complet, telefon, email).

4.13.2 Logica Backend (/booking/confirm în app.py)

Logica de backend pentru ruta /booking/confirm gestionează atât afișarea formularului de confirmare (GET), cât și procesarea finală a rezervării (POST).

1. **Verificare autentificare și parametri:** Ruta necesită ca utilizatorul să fie autentificat, și preia ID-urile camerelor și datele de start/end din URL, validând perioada.
2. **Cerere GET (afișare formular):** Preia detaliile camerelor și ale proprietății, calculează prețul total și randează template-ul.
3. **Cerere POST (finalizare rezervare) - Tranzacție ACID:** Se inițiază o tranzacție de bază de date pentru a asigura atomicitatea operațiunilor. Se re-verifică disponibilitatea camerelor și prețurile direct în baza de date. Pentru fiecare cameră din pachet, se inserează o înregistrare în tabelul `reservations`, iar detaliile de contact și prețul total sunt salvate într-un tabel separat. Tranzacția este confirmată (`cnx.commit()`) dacă toate operațiunile reușesc, sau anulată (`cnx.rollback()`) în caz de eroare, prevenind rezervările parțiale. Utilizatorul este redirecționat către pagina de succes.

Confirmare și Finalizare Rezervare

Sumar Pachet

Perioada: **09 iunie 2025 - 10 iunie 2025** (1 noapte)

1x Cameră Medievală (2 pers.) - 380.00 lei/noapte

Total de plată: 380.00 lei

Date de contact

Nume complet:

Ion Gigel

Telefon:

0712345678

Email:

abc@xyz.ro

Confirmă și Trimite Rezervarea

Figura 4.18: Formular de rezervare

4.14 Pagina de Confirmare a Rezervării (/booking/success)

Această rută este responsabilă pentru afișarea unui mesaj de succes și a detaliilor rezervării după ce aceasta a fost confirmată.

4.14.1 Prezentarea Frontend (reservation_success.html)

Pagina de succes include un mesaj vizual de confirmare ("Mulțumim, rezervarea a fost realizată cu succes!"). Detaliază complet rezervarea: proprietate, perioadă, număr de nopți, preț total, precum și numele, emailul și telefonul clientului. De asemenea, oferă un buton pentru a redirecționa utilizatorul către pagina "Rezervările mele".

4.14.2 Logica Backend (/booking/success în app.py)

Logica de backend pentru ruta /booking/success gestionează afișarea detaliilor rezervării după ce aceasta a fost confirmată cu succes.

1. **Verificare sesiune:** Verifică dacă utilizatorul este logat și dacă există un `last_booking_id` stocat în sesiune.
2. **Preluare detalii rezervare:** Folosește `last_booking_id` pentru a prelua toate detaliile rezervării din baza de date.

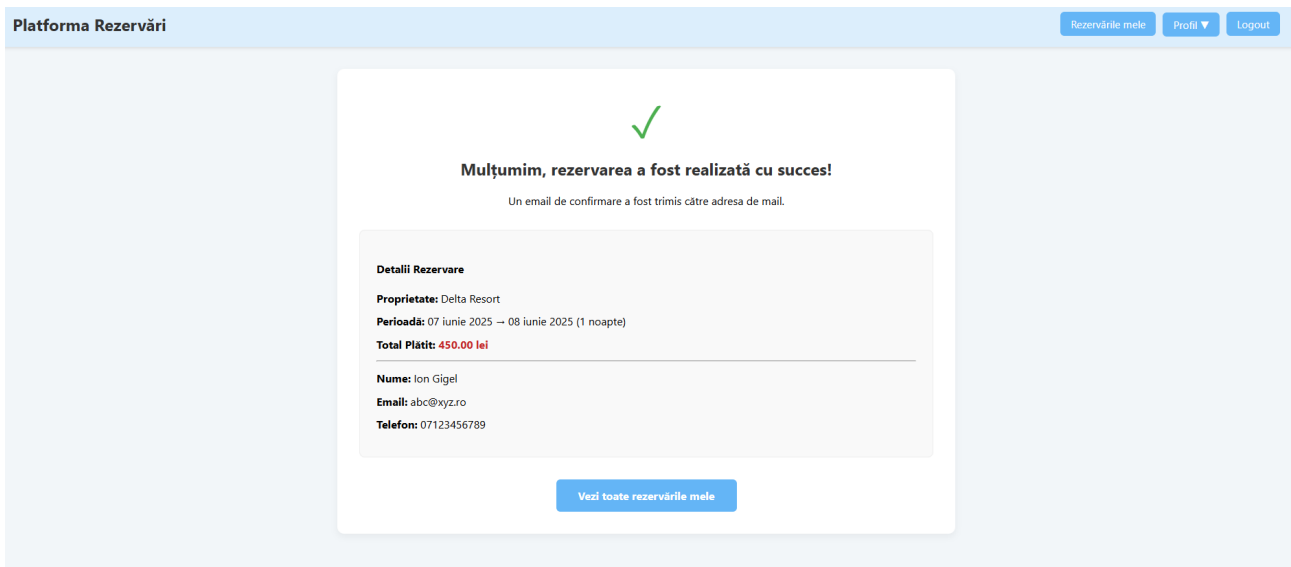


Figura 4.19: Formularul de autentificare a utilizatorilor

3. **Randare template:** Randează `reservation_success.html` cu detaliile rezervării.

4.15 Vizualizarea și Gestionarea Rezervărilor

Secțiunea "Rezervările mele" este un panou de control dedicat utilizatorilor non-administratori, oferindu-le posibilitatea de a vizualiza și gestiona toate rezervările pe care le-au efectuat în cadrul platformei.

4.15.1 Prezentarea Frontend (`my_reservations.html`)

Interfața paginii "Rezervările mele" este proiectată pentru a oferi o imagine de ansamblu clară și acces rapid la informațiile esențiale ale rezervărilor. Fiecare rezervare este afișată într-un card distinct, conținând următoarele detalii:

- Numele proprietății la care s-a efectuat rezervarea.
- Numele specific al camerei rezervate.
- Perioada exactă a rezervării, incluzând data de început și data de sfârșit, formate într-un mod prietenos pentru utilizator.
- Statusul curent al rezervării (de exemplu, "confirmed" sau "cancelled").
- Prețul total al rezervării, dacă este disponibil.
- Detalii de contact ale clientului, cum ar fi numele complet, adresa de email și numărul de telefon, pentru o referință rapidă.

De asemenea, dacă statusul rezervării este "confirmed", este afișat un buton "Anulează Rezervarea". Acest buton include o confirmare JavaScript (`onclick="return confirm(...)"`) pentru a preveni anulările accidentale, asigurând o măsură suplimentară de siguranță.

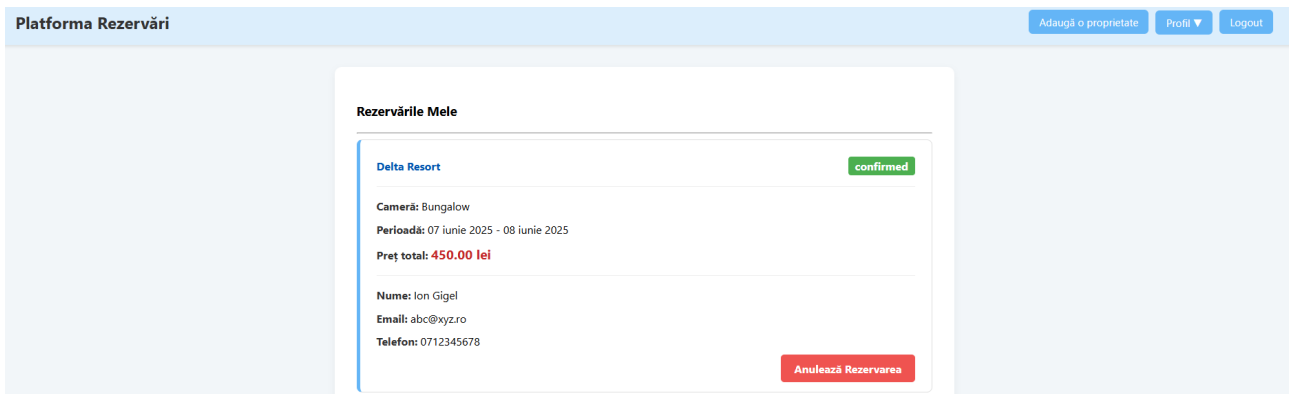


Figura 4.20: Pagina de administrare a rezervărilor

4.15.2 Logica Backend (/my-reservations în app.py)

Funcția de backend asociată rutei `/my-reservations` este responsabilă cu preluarea și pregătirea datelor pentru afișarea în interfață.

1. **Verificare autentificare:** Primul pas esențial este verificarea sesiunii utilizatorului. Dacă utilizatorul nu este autentificat, acesta este redirecționat automat către pagina de login, protejând astfel accesul neautorizat la informațiile personale.
2. **Preluarea datelor:** Pentru utilizatorii autentificați, funcția execută o interogare complexă către baza de date. Această interogare utilizează operațiuni `JOIN` pentru a extrage toate rezervările asociate cu `user_id`-ul utilizatorului din sesiune, combinând informații din tabelele `reservations`, `rooms`, `properties` și `reservation_details`. Scopul este de a aduna toate detaliile necesare pentru afișare, cum ar fi numele camerei, numele proprietății, detaliile de contact ale clientului și prețul total. Rezultatele sunt apoi sortate în ordine descrescătoare după ID-ul rezervării pentru o vizualizare cronologică.
3. **Randare template:** Datele colectate și procesate sunt transmise template-ului `my_reservations.html`, unde sunt randate dinamic pentru a construi interfața vizibilă utilizatorului.

4.16 Anularea Rezervării (/cancel_reservation/<int:reservation_id>)

Această rută permite utilizatorilor să anuleze o rezervare existentă, cu verificări stricte ale drepturilor și ale stării rezervării.

4.16.1 Logica Backend (/cancel_reservation/<int:reservation_id> în app.py)

1. **Verificare autentificare:** Ruta necesită ca utilizatorul să fie logat.
2. **Verificare drepturi de acces:** Se preiau informații despre rezervare și proprietar pentru a se asigura că utilizatorul logat este fie creatorul rezervării, fie proprietarul proprietății. Operațiunea este interzisă în caz contrar.
3. **Verificare status rezervare:** Dacă rezervarea este deja anulată, se afișează un mesaj informativ.
4. **Actualizare status:** Statusul rezervării este actualizat la "cancelled" în baza de date.

5. **Commit:** Modificarea este confirmată printr-un `cnx.commit()`.
6. **Redirecționare:** Utilizatorul este redirecționat fie către pagina "Rezervările mele", fie către pagina de gestionare a rezervărilor proprietății.

4.17 Gestionarea Rezervărilor Proprietăților Personale

Această secțiune este un panou de control dedicat proprietarilor de proprietăți, oferindu-le o vizualizare centralizată a tuturor rezervărilor active și viitoare aferente unei proprietăți specifice. Spre deosebire de pagina generală "Rezervările mele" (care afișează rezervările făcute de utilizator), aceasta listează rezervările primite pentru una dintre proprietățile sale.

4.17.1 Prezentarea Frontend (`manage_property_reservations.html`)

Interfața este concepută pentru a oferi proprietarului o imagine clară asupra stării rezervărilor pentru o anumită proprietate. Pe lângă numele, adresa și orașul proprietății afișate proeminent, pagina prezintă o listă de carduri, unde fiecare card reprezintă o rezervare individuală. Detaliile afișate pentru fiecare rezervare includ:

- Numele camerei și ID-ul camerei.
- Perioada rezervării (dată de început și de sfârșit).
- Statusul rezervării (de exemplu, "confirmed" sau "cancelled").
- Numele complet al clientului care a efectuat rezervarea, alături de adresa sa de email și numărul de telefon.
- Prețul total al rezervării.

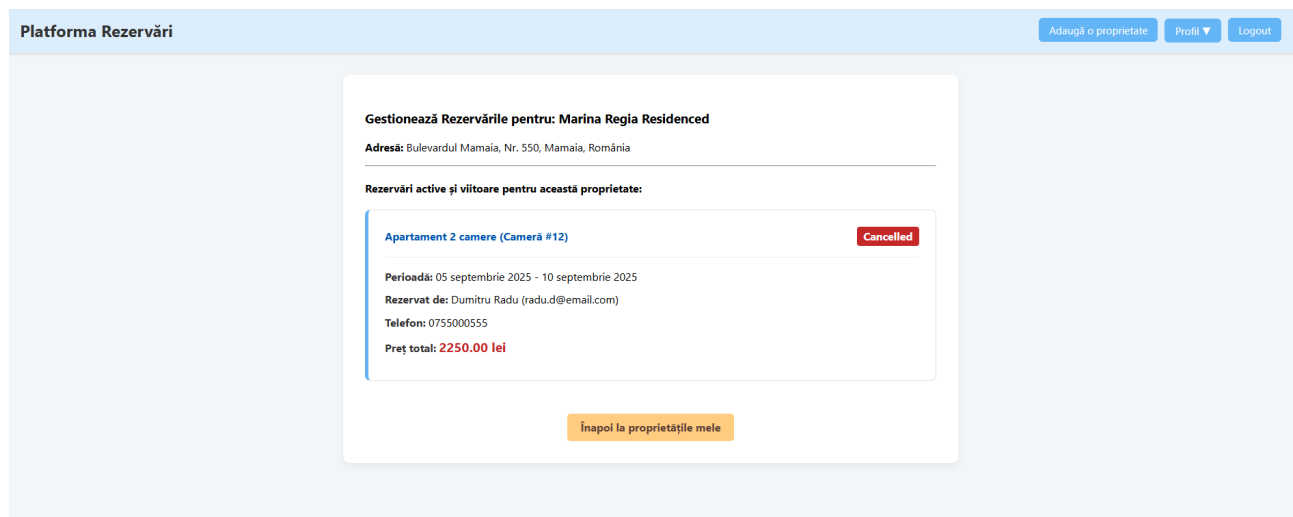


Figura 4.21: Pagina cu gestionările rezervărilor propriilor proprietăți

Dacă statusul rezervării este "confirmed", este afișat un buton "Anulează Rezervarea". Acest buton include o confirmare JavaScript pentru a preveni anulările accidentale și, un aspect important, trimite un parametru de redirecționare (`redirect_to_prop_id`) către backend, astfel încât, după anulare, utilizatorul să fie readus pe pagina de gestionare a rezervărilor proprietății curente. Un buton "Înapoi la proprietățile mele" este de asemenea disponibil pentru o navigare facilă.

4.17.2 Logica Backend (/manage_property_reservations/<int:property_id> în app.py)

Funcția de backend asociată acestei rute este responsabilă pentru:

1. **Verificarea autentificării și a drepturilor de proprietar:** Ruta este protejată și necesită ca utilizatorul să fie autentificat. În plus, se efectuează o verificare strictă pentru a se asigura că utilizatorul logat este **proprietarul** proprietății al cărei ID este specificat în URL. Dacă utilizatorul nu este proprietarul de drept sau proprietatea nu există, accesul este interzis și utilizatorul este redirecționat.
2. **Preluarea detaliilor proprietății:** Se extrag detaliile complete ale proprietății vizate (nume, adresă, oraș, țară) pentru a fi afișate în interfața de gestionare.
3. **Preluarea rezervărilor proprietății:** Se execută o interogare complexă în baza de date pentru a obține toate rezervările asociate cu proprietatea specificată. Interogarea include JOIN-uri cu tabelele `rooms` și `reservation_details` pentru a aduna informații despre cameră, detalii client și preț. Se filtrează doar rezervările active sau viitoare (unde `end_date` este mai mare sau egală cu data curentă) și se sortează după data de început a rezervării.
4. **Randare template:** Toate datele colectate (detaliile proprietății și lista de rezervări) sunt transmise către template-ul `manage_property_reservations.html` pentru a fi randate dinamic și afișate proprietarului.

Capitolul 5

Arhitectura de Suport și Înaltă Disponibilitate

Introducere în Arhitectura de Suport

Acest capitol se concentrează pe elementele fundamentale ale infrastructurii și arhitecturii de suport care asigură scalabilitatea, reziliența și performanța necesare unei aplicații web moderne, capabile să gestioneze un trafic considerabil și să funcționeze fără întreruperi. Dincolo de logica de afaceri a aplicației, modul în care aceasta este găzduită și gestionată la nivel de infrastructură este la fel de crucial pentru succesul său. Se vor explora mecanisme cheie precum `load balancing`-ul, replicarea bazelor de date și rolul virtualizării în testarea și implementarea unei astfel de arhitecturi.

5.1 Rolul Nginx în contextul arhitecturii implementate

Așa cum a fost prezentat în Secțiunea 2.5, Nginx are un rol esențial în menținerea scalabilității și rezilienței aplicației web, prin funcționalitățile sale de `load balancing` și `reverse proxy`. În arhitectura implementată pentru sistemul de rezervări, Nginx este configurat să gestioneze eficient traficul către instanțele aplicației Flask, contribuind direct la optimizarea performanței și disponibilității platformei.

Aplicarea Load Balancing-ului în Arhitectura Proiectului

În arhitectura distribuită prezentată, Nginx, configurat pe stația de lucru principală (Host), acționează ca punct unic de intrare pentru cererile utilizatorilor. Aceste cereri sunt distribuite între cele două instanțe ale aplicației Flask - una locală (Host) și una pe Mașina Virtuală 2 (VM2) - asigurând o repartizare echilibrată a încărcării și prevenind supraîncărcarea unei singure instanțe. Distribuția activă a traficului permite gestionarea unui volum ridicat de cereri concurente și menținerea unor timpi de răspuns rapizi, chiar în condiții de trafic intens. Totodată, Nginx poate detecta instanțele indisponibile și redirecționa traficul către cele funcționale, contribuind astfel la reziliența și disponibilitatea ridicată a stratului de aplicație.

Integrarea Nginx cu Flask ca Reverse Proxy și Servirea Fișierelor Statice

Nginx este integrat ca `reverse proxy` pentru a facilita comunicarea securizată și eficientă între utilizatori și serverele Flask. Configurația Nginx este adaptată pentru a recunoaște cererile dinamice destinate logicii aplicației Flask și a le proxy-a către grupul de servere definite, asigurând transmiterea corectă a antetelor HTTP esențiale (Host, X-Real-IP, X-Forwarded-For). Un aspect crucial al acestei integrări este rolul Nginx în servirea fișierelor statice (CSS, JavaScript, imagini). Prin configurarea Nginx să servească direct aceste resurse, se reduce semnificativ sarcina pe serverele Flask, care pot astfel dedica mai multe resurse procesării logicii de business

și a cererilor dinamice. Această optimizare îmbunătățește considerabil viteza de încărcare a paginilor și experiența generală a utilizatorului.

Fragmentul de configurare Nginx care exemplifică această integrare specifică este prezentat în fragmentul de cod A.1 (Capitolul 2), unde sunt definite grupul upstream și regulile de location pentru proxy-ing și servirea staticului.

Impactul asupra Fluxurilor de Aplicație

Folosirea Nginx ca load balancer și reverse proxy aduce beneficii pentru fiecare flux de lucru al aplicației de rezervări:

- **Înregistrare și Autentificare:** Chiar și în condiții de trafic intens, cererile critice de înregistrare și autentificare sunt distribuite uniform, asigurând un proces fluid și responsiv. Resiliența oferită de Nginx previne disrupțiile în cazul unei defecțiuni a unei instanțe Flask.
- **Căutare și Vizualizare Proprietăți:** Fluxul de căutare, care poate genera un volum mare de cereri de citire, beneficiază direct de distribuția încărcării. Utilizatorii experimentează timpi de răspuns rapizi, iar servirea eficientă a imaginilor proprietăților direct de către Nginx reduce sarcina pe serverele Flask.
- **Crearea unei Rezervări:** Deși este un proces tranzacțional complex, distribuirea cererilor inițiale către servere diferite asigură că sistemul rămâne receptiv. Nginx ajută la menținerea unei conexiuni stabile cu serverul Flask care procesează rezervarea.
- **Fluxurile de Management (Proprietari și Admin):** Aceste fluxuri, esențiale pentru operarea platformei, beneficiază de aceeași reziliență, permițând managementul eficient chiar și sub presiune generală a sistemului.

5.2 Rolul Replicării MySQL (Master-Slave) în Contextul Fluxurilor Reale

Scalabilitatea și disponibilitatea ridicată a nivelului de bază al bazei de date sunt esențiale pentru performanța și reziliența sistemului de rezervări, în special datorită caracterului predominant „read-heavy” al aplicațiilor web de acest tip. Pentru a satisface aceste cerințe, arhitectura implementată utilizează un mecanism robust de replicare MySQL Master-Slave, cu nodul Master găzduit pe stația de lucru principală (Host) și nodul Slave pe Mașina Virtuală 1 (VM1). Această configurație permite optimizarea operațiilor de citire și scriere, precum și asigurarea continuității serviciului în caz de defecțiuni.

Separarea Citirilor de Scrieri (Read/Write Splitting) în Implementarea Proiectului

Unul dintre beneficiile majore ale configurației Master-Slave, esențial pentru optimizarea performanței, este posibilitatea de a separa cererile de citire (SELECT) de cele de scriere (INSERT, UPDATE, DELETE). În cadrul proiectului, nodul MySQL Master (pe Host) este configurat să gestioneze exclusiv toate operațiile de scriere, asigurând consistența și integritatea datelor. Concomitent, nodul MySQL Slave (pe VM1) preia majoritatea operațiilor de citire. Această distribuție a sarcinilor maximizează debitul și reduce presiunea asupra nodului Master, eliberându-l de majoritatea cererilor de citire. Prin direcționarea inteligentă a traficului,

aplicația poate scala operațiile de citire prin adăugarea de noduri Slave suplimentare, dacă este necesar, fără a compromite performanța operațiunilor de scriere critice.

Impactul Replicării asupra Fluxurilor de Aplicație și Rezilienței Sistemului

Replicarea MySQL și separarea citirilor de scrieri au un impact profund asupra fiabilității și performanței tuturor fluxurilor de aplicație:

- **Fluxurile de Căutare și Vizualizare Proprietăți:** Acestea sunt preponderent "read-heavy" (ex., căutări după destinație, vizualizarea detaliilor proprietăților, calculul disponibilității). Toate aceste interogări sunt direcționate către nodul Slave de pe VM1. Această abordare reduce drastic sarcina pe Master, asigurând timpi de răspuns foarte rapizi pentru utilizatori, chiar și sub un volum mare de cereri concurente de citire. Performanța globală a sistemului este considerabil îmbunătățită.
- **Fluxurile de Înregistrare și Autentificare:** Deși componenta implică atât operații de scriere a datelor utilizatorilor la înregistrare, cât și operații de citire pentru autentificare, scrierile (INSERT) sunt gestionate de nodul Master, în timp ce citirile frecvente pentru autentificare pot fi direcționate către nodul Slave. Această separare previne blocajele și asigură un flux de acces eficient în sistem.
- **Crearea, Modificarea și Anularea unei Rezervări:** Acestea sunt operații critice care implică scrieri (INSERT, UPDATE, DELETE). Toate aceste operațiuni sunt dirijate către nodul Master. Performanța Master-ului este crucială aici, iar faptul că nu este suprasolicitat cu cereri de citire contribuie la succesul și la finalizarea rapidă a tranzacțiilor. În plus, în cazul unei defecțiuni a nodului Master, nodul Slave poate fi promovat la rolul de Master (mecanism de failover), asigurând continuitatea serviciului și disponibilitatea datelor, chiar și în situații de eșec al unui nod.
- **Fluxurile de Management (Proprietari și Admin):** Operațiile de adăugare, modificare sau ștergere a proprietăților și camerelor, precum și gestionarea rezervărilor, implică scrieri pe Master. Citirile necesare pentru afișarea dashboard-urilor sau a listelor de proprietăți pot fi preluate de pe Slave, menținând interfața de administrare responsabilă și eficientă.

5.3 Rolul Infrastructurii de Virtualizare în Testarea Arhitecturii

Implementarea și testarea arhitecturii distribuite, multi-nod a sistemului de rezervări a necesitat un mediu flexibil, izolat și ușor de gestionat. Infrastructura de virtualizare a fost esențială în acest proces, oferind posibilitatea de a crea un mediu de dezvoltare și testare care reproduce fidel condițiile de producție, fără a implica costuri și complexitate asociate hardware-ului fizic. Mașinile virtuale VM1 și VM2 au fost utilizate pentru a demonstra eficient conceptele de scalabilitate și disponibilitate ridicată.

Crearea unui Mediu Izolat și Consistent

Virtualizarea permite crearea de mașini virtuale sau containere care rulează sisteme de operare și aplicații într-un mediu izolat față de hardware-ul gazdă și de celelalte instanțe virtuale. Această izolare este esențială pentru:

- **Consistență:** Fiecare mediu de dezvoltare sau testare poate fi configurat identic, garantând că un comportament observat într-o instanță se va reproduce în oricare altă instanță. Această consistență reduce erorile legate de diferențele de configurare între medii.
- **Reproducibilitate:** Orice configurație problematică sau bug poate fi reprodus cu ușurință într-un mediu virtual curat, facilitând depanarea.
- **Independența de Hardware:** Dezvoltatorii nu sunt legați de configurația hardware specifică a mașinii lor, permițând echipei să lucreze cu medii de operare consistente.

Testarea Scenariilor de Eșec și a Rezilienței

Unul dintre cele mai mari avantaje ale virtualizării în contextul arhitecturilor distribuite este capacitatea de a testa scenarii de eșec în mod controlat, fără a afecta mediul de producție. Aceste teste sunt esențiale pentru a valida mecanismele de **failover** și reziliență implementate:

- **Simularea căderii componentelor:** Se poate opri o instanță **Flask** pentru a vedea dacă **Nginx** o detectează și redirecționează corect traficul către celelalte. Se poate opri serverul **MySQL Master** pentru a testa procesul de **failover** către un **Slave**.
- **Testarea scalabilității:** Prin lansarea rapidă de noi mașini virtuale, se poate simula o creștere a traficului și se poate evalua cum răspunde sistemul, confirmând că **load balancing**-ul și scalarea funcționează conform așteptărilor.
- **Izolarea problemelor:** Dacă o componentă începe să consume prea multe resurse sau să se comporte anormal, mediul virtual izolat previne ca problema să se propage la alte părți ale sistemului sau la mașina gazdă.

Această capacitate de a provoca defecțiuni controlate în timpul testării este extrem de utilă pentru identificarea și corectarea punctelor slabe înainte ca aplicația să fie lansată în mediul de producție, unde astfel de erori ar putea avea consecințe semnificativ mai grave.

Flexibilitatea în Configurare și Dezvoltare Rapidă

Virtualizarea oferă o flexibilitate enormă în configurarea și reconfigurarea mediilor de dezvoltare.

- **Snapshots:** Mașinile virtuale pot fi salvate în stări specifice (**snapshots**), permițând dezvoltatorilor să revină rapid la o stare anterioară stabilă în cazul unor modificări care strică mediul.
- **Clonare:** Medii întregi pot fi clonare rapid, permițând mai multor dezvoltatori să lucreze pe copii identice ale sistemului sau să testeze diferite ramuri de cod în paralel.
- **Integrare continuă (CI/CD):** Medii virtualizate sau containerizate sunt pilonul proceselor moderne de **CI/CD** (Continuous Integration/Continuous Deployment), unde testele automate rulează într-un mediu curat și consistent înainte de fiecare deploy, asigurând calitatea codului.

Capitolul 6

Testarea Performanței și Analiza Scalabilității

Introducere în Testarea de Performanță

Asigurarea scalabilității și fiabilității a constituit un obiectiv central al acestei lucrări, având în vedere specificul unei aplicații web de rezervări, care trebuie să susțină un număr mare de utilizatori și interacțiuni simultane. Testarea de performanță (load testing) reprezintă o etapă esențială în ingineria software, deoarece permite analiza modului în care sistemul răspunde la sarcini ridicate. Aceasta implică simularea unui trafic intens, cu utilizatori și cereri concurente, pentru a măsura capacitatea de procesare, stabilitatea și timpii de răspuns ai aplicației. Prin aplicarea unor scenarii apropiate de utilizarea reală, pot fi identificate punctele critice, evaluate limitele de scalabilitate și determinate pragurile maxime de trafic gestionabil. Scopul acestei etape a fost validarea soluțiilor de scalare implementate, respectiv echilibrarea traficului cu Nginx și replicarea MySQL Master-Slave într-un context funcțional și realist.

6.1 Alegerea Instrumentului de Testare: Locust

Pentru realizarea testelor de performanță, s-a optat pentru instrumentul open-source Locust. Alegerea Locust a fost justificată de următoarele avantaje care se aliniază cerințelor proiectului:

- **Programabil în Python:** Locust permite definirea comportamentului utilizatorilor virtuali ("user behavior") direct în Python, un limbaj deja familiar în cadrul proiectului prin utilizarea Flask. Această flexibilitate permite crearea unor scenarii de testare complexe și realiste, adaptate specific fluxurilor aplicației de rezervări.
- **Simularea Comportamentului Realist:** Spre deosebire de alte instrumente care se bazează pe înregistrarea și redarea cererilor HTTP, Locust permite descrierea programatică a acțiunilor utilizatorilor (ex: un utilizator caută o proprietate, apoi o vizualizează, apoi încearcă să rezerve). Această abordare realistă simulează mai fidel interacțiunile umane cu aplicația.
- **Distribuție și Scalabilitate:** Locust este proiectat pentru a rula teste la scară largă, suportând un mod distribuit. Astfel, se pot lansa mai multe instanțe Locust ("workers") pe mașini diferite, coordonate de o singură instanță "master", pentru a genera un volum masiv de cereri, depășind capacitatea unei singure mașini de testare.
- **Interfață Web Intuitivă:** Instrumentul oferă o interfață web de monitorizare în timp real, care afișează metrici cheie precum numărul de cereri pe secundă (RPS - Requests Per Second), timpii de răspuns medii, deviația standard, rata de eșec și distribuția timpilor de răspuns (percentile). Această interfață facilitează monitorizarea și analiza imediată a performanței sistemului în timpul testelor.

Pentru a evalua performanța aplicației într-un mod cât mai complet, au fost implementate mai multe scenarii de testare cu Locust, fiecare vizând un comportament specific al utilizatorului și un tip de sarcină distinctă pentru sistem:

- **Fluxul de Navigare și Descoperire a Informațiilor:** Simulează comportamentul unui utilizator care explorează platforma, incluzând accesarea paginii principale și a paginii personale de rezervări, urmată de efectuarea unei căutări parametrizate pentru o proprietate. Are ca scop testarea operațiunilor de citire (read) din baza de date și capacitatea serverului de a genera pagini dinamice care implică interogări complexe.
- **Fluxul Complet de Gestionare a Proprietăților (CRUD):** Acoperă toate etapele de gestionare a unei proprietăți, de la creare până la modificare și ștergere. Evaluează performanța operațiunilor de scriere (write) în baza de date și menținerea coerenței datelor în condiții de stres, testând în același timp tranzacțiile și mecanismele de blocare (locking).
- **Fluxul Tranzacțional End-to-End: Rezervare și Anulare:** Simulează un flux complet de tranzacție, incluzând căutarea disponibilității, selectarea unei proprietăți, parcurgerea pașilor de confirmare a rezervării (cu multiple validări și scrieri în baza de date) și anularea aceleiași rezervări. Testează integritatea întregului sistem tranzacțional, de la interfața web până la logica de business și persistența datelor, asigurând că aplicația poate gestiona un volum mare de tranzacții concurente fără coruperea datelor.
- **Fluxul de Administrare a Contului de Utilizator:** Simulează accesarea și modificarea datelor din profilul personal al utilizatorului. De asemenea, evaluează performanța subsistemului de autentificare și autorizare, precum și a operațiunilor de actualizare a datelor specifice utilizatorului.
- **Fluxul de Solicitare a Resurselor Statice:** Utilizatorul virtual accesează în mod repetat fișierele statice ale aplicației (CSS, JavaScript, imagini), ceea ce permite izolarea și testarea performanței serverului web (Nginx) în livrarea de conținut static. Acest scenariu contribuie la identificarea eventualelor probleme de configurare sau a limitărilor de lățime de bandă.
- **Căutări (Read-Heavy):** Utilizatorii virtuali au efectuat căutări repetate de proprietăți, au vizualizat detalii și au navigat prin rezultate. Acest flux a permis evaluarea operațiunilor de citire din baza de date, a eficienței direcționării cererilor către nodul Slave și a capacității Nginx de a gestiona un volum mare de cereri GET.
- **Înregistrări și autentificări:** Au fost testate procesele de creare a conturilor și de login, care implică atât scrieri (INSERT), cât și citiri (SELECT). Accentul a fost pus pe performanța nodului Master și pe logica de sesiune gestionată de Flask.
- **Procesul de rezervare (Write-Heavy/Transactional):** Cel mai complex flux, care acoperă pașii de la căutare până la confirmarea tranzacției. A inclus multiple operațiuni de scriere și actualizare (INSERT, UPDATE), urmărindu-se performanța nodului Master și modul în care sistemul gestionează tranzacții ACID sub sarcină.
- **Reziliență și failover:** Au fost introduse situații de eșec, precum oprirea unei instanțe Flask sau a nodului MySQL Master. S-a observat comportamentul de failover al Nginx și replicarea MySQL, fiind măsurat și timpul de recuperare a serviciului.

6.2 Procesul de Execuție și Monitorizare

Execuția testelor de performanță cu Locust implică următorii pași:

1. **Lansarea Locust:** Locust este lansat în mod master-slave (sau standalone pentru teste mici), iar instanțele worker sunt pornite pe VM-uri, toate direcționând traficul către Nginx.
2. **Configurare Test din UI:** Din interfața web a Locust (accesibilă la <http://localhost:8089> pe mașina master), se configurează numărul de utilizatori virtuali ("Number of users to simulate"), rata de creștere a acestora ("Spawn rate") și alte opțiuni specifice testului. Din interfața web a Locust (accesibilă la <http://localhost:8089> pe mașina master), se configurează numărul de utilizatori virtuali ("Number of users to simulate"), rata de creștere a acestora ("Spawn rate") și alte opțiuni specifice testului.
3. **Monitorizare în Timp Real:** Interfața web Locust oferă grafice și tabele în timp real pentru:
 - (a) Rata de Cereri pe Secundă (RPS): Indică volumul de trafic procesat de aplicație.
 - (b) Timpi de Răspuns Medii: Timpul mediu necesar pentru ca serverul să răspundă la o cerere.
 - (c) Deviația Standard: Măsoară variabilitatea timpilor de răspuns.
 - (d) Rata de Eșec: Procentul de cereri care au returnat erori HTTP (ex: 5xx, erori de rețea).
 - (e) Distribuția Timpilor de Răspuns (Percentile): Arată procentul de cereri care au fost procesate sub un anumit timp (ex: 90% din cereri au răspuns în mai puțin de X ms).
4. **Monitorizare Resurse Sistem:** Concomitent cu testele Locust, resursele hardware ale fiecărei mașini (Host, VM1, VM2) precum ar fi CPU-ul și memoria RAM sunt monitorizate cu Task Manager.
5. **Analiza Log-urilor:** Log-urile generate de Nginx, Flask și MySQL sunt analizate pentru a identifica erori, avertismente sau anomalii care ar putea indica probleme de performanță sau securitate sub sarcină.

6.3 Arhitectura Mediului de Testare

Mediul de testare a fost conceput pentru a replica fidel arhitectura de producție propusă, asigurând validitatea rezultatelor obținute. Componentele au fost distribuite pe mașini virtuale pentru a simula un mediu real de cloud și pentru a permite izolarea resurselor:

- Stația de Lucru Principală (Host): Găzduiește serverul MySQL Master, responsabil cu operațiile de scriere, și o instanță a aplicației Flask. De asemenea, pe Host este configurat Nginx, care funcționează ca load balancer pentru ambele instanțe Flask (Host și VM2). Instanța master a Locust a fost rulată tot pe această mașină, coordonând workerii.
- Mașina Virtuală 1 (VM1): A fost configurată ca server MySQL Slave, dedicat operațiilor de citire și replicării datelor de la Master. Această mașină virtuală a fost utilizată și pentru a rula workerii Locust, generând trafic intens către aplicație.
- Mașina Virtuală 2 (VM2): A găzduit o instanță secundară a aplicației Flask, care a preluat cereri distribuite de Nginx. Rolul ei a fost esențial în demonstrarea scalabilității orizontale a stratului de aplicație, permițând simularea realistă a traficului și a interacțiunilor dintre componentele sistemului distribuit. În acest mod, a oferit o bază solidă pentru testele de performanță.

6.3.0.1 Pregătirea și Rularea Testului Locust

Pentru a executa testele, au fost urmați pașii de configurare și lansare a Locust:

1. **Configurarea fișierului locustfile.py:** Fișierul de testare Locust a fost configurat pentru a direcționa traficul direct către instanța Flask de pe sistemul gazdă, setând `host = "http://localhost:5000"`. Task-urile definite au inclus simularea căutărilor (operații de citire), vizualizarea detaliilor proprietăților și, pentru utilizatorii autentificați, interacțiuni cu pagina de profil. În plus, au fost incluse, cu o pondere mai redusă, fluxuri de înregistrare și rezervare (operații de scriere).
2. **Lansarea Aplicației Flask:** Instanțele aplicației Flask au fost pornite manual, asigurându-se că sunt accesibile pe portul 5000.
3. **Lansarea Locust:** Din terminal, Locust a fost inițiat cu comanda `locust -f locustfile.py`. Aceasta a activat interfața web Locust, accesibilă la `http://localhost:8089`.
4. **Configurația în Interfața Web Locust:** În interfața web, testul a fost configurat cu următoarele setări inițiale pentru a simula un volum moderat de utilizatori:
 - Number of users to simulate: 50
 - Spawn rate (users per second): 5
 - Host: `http://localhost:5000`
5. **Start Test:** Testul a fost demarat prin apăsarea butonului "Start swarming" și lăsat să ruleze pentru o perioadă de cel puțin 15 minute odată ce numărul maxim de utilizatori a fost atins.

6.4 Scenarii de Testare Implementate

Următoarea secțiune prezintă pașii efectuați pentru fiecare scenariu de testare, având ca scop evaluarea scalabilității și rezilienței arhitecturii sistemului de rezervări. Testele au fost realizate folosind Locust pentru simularea traficului și diverse instrumente de monitorizare la nivel de sistem pentru colectarea și analizarea metricilor de performanță.

6.4.1 Cazul 1: Aplicația Flask Monolitică (Fără Scalare)

Acest scenariu de testare are rolul de a stabili performanța de referință, simulând comportamentul unei arhitecturi tradiționale, monolitice, pentru aplicația web de rezervări, fără mecanisme active de scalare sau load balancing. Obiectivul principal constă în înregistrarea metricilor de performanță inițiale, care vor servi ulterior drept punct de comparație pentru evaluarea îmbunătățirilor aduse de soluțiile distribuite implementate.

Configurația Mediului

Pentru a reproduce configurația monolitică, cadrul de testare a fost simplificat, plasând toate componentele esențiale pe un singur sistem.

O singură instanță a aplicației web Flask a fost pornită, rulând pe portul implicit 5000, fiind responsabilă atât de logica funcțională, cât și de procesarea directă a cererilor HTTP. Serverul principal MySQL, operativ pe sistemul gazdă, a gestionat toate tranzacțiile cu baza de date, incluzând operațiuni de citire și scriere.

Pe durata testării, instanțele virtuale VM1 și VM2 au fost dezactivate: serverul MySQL secundar nu a fost utilizat, iar a doua instanță Flask nu a rulat. Componenta Nginx a fost de asemenea oprită, astfel încât cererile generate de Locust au fost direcționate direct către aplicația Flask de pe sistemul gazdă, fără implicarea unui load balancer sau proxy invers. Baza de date MySQL principală a fost populată cu un set de date de test (150 de utilizatori, 300 de proprietăți și 900 de unități locative), generat cu programul `generate_test_data.py`, pentru a asigura consistența operațiilor de citire și scriere.

Interpretarea Detaliată a Rezultatelor

Analiza performanței aplicației s-a realizat pe două momente cheie: **după 2-3 minute** de la începutul testului și **după 15 minute**, pentru a evalua comportamentul inițial și stabilitatea pe termen lung.

Rezultatele după 2-3 minute

- **Debitul sistemului (RPS):** Sistemul a atins rapid un debit de aproximativ 150.2 cereri pe secundă, stabilizându-se în primele secunde ale testului. Nu s-au înregistrat eșecuri.

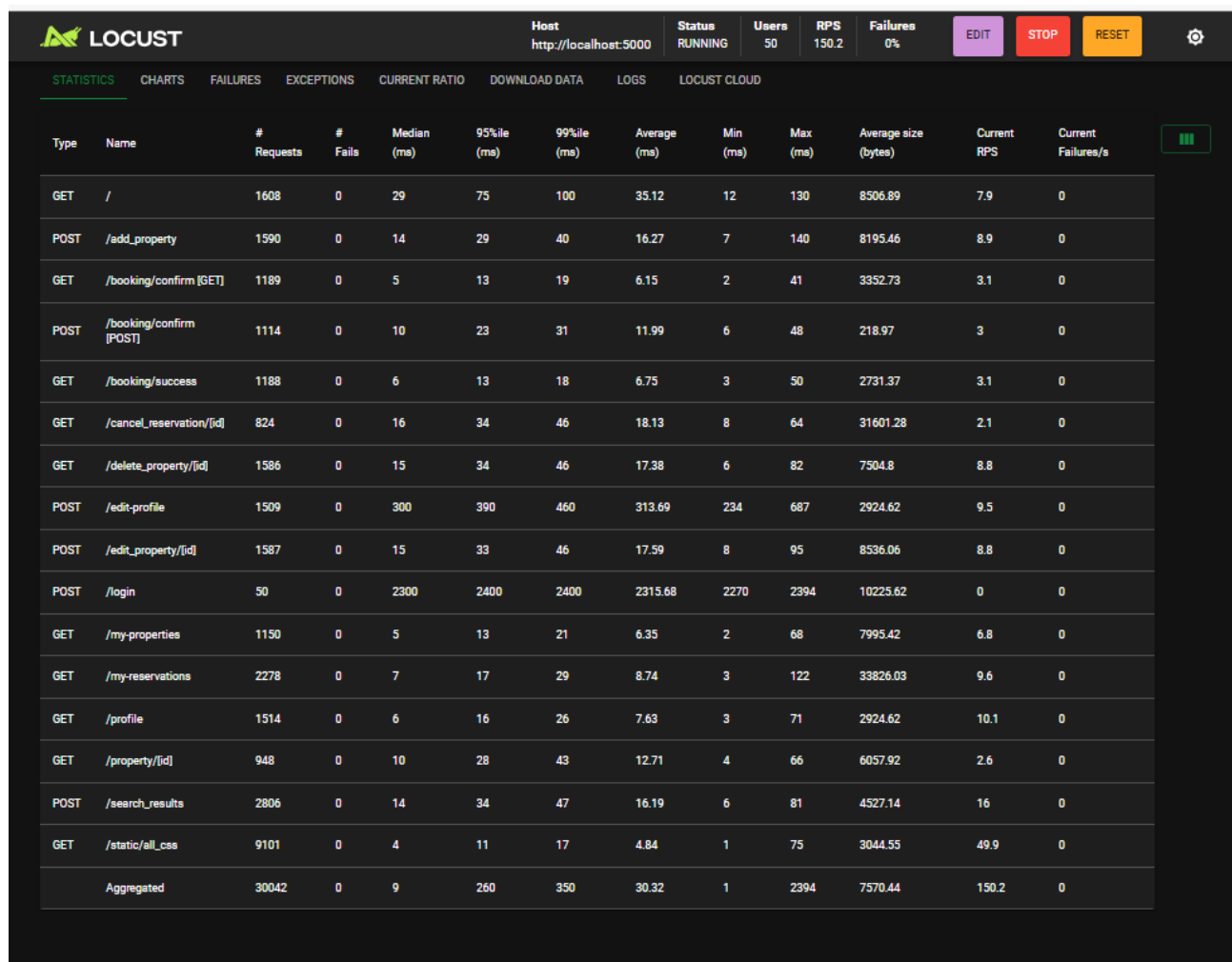


Figura 6.1: Total Requests per Second

- **Timpii de răspuns (Latență):**

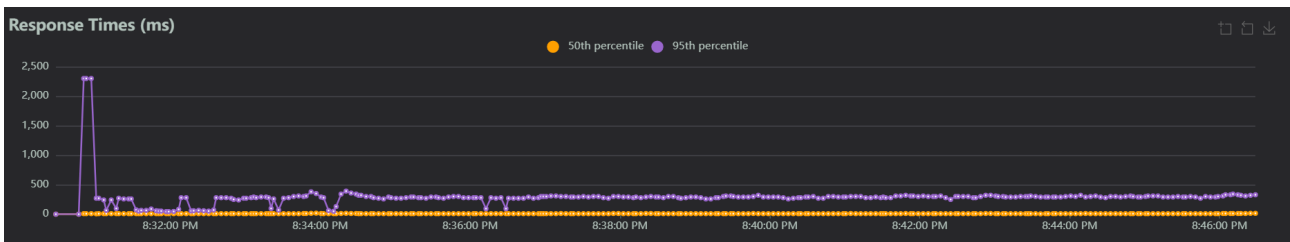


Figura 6.2: Total Requests per Second

- Endpoint-uri rapide: Majoritatea cererilor de tip GET, cum ar fi `/my-reservations` sau `/static/all_css`, au avut mediana sub 10 ms și percentila 95 sub 260 ms, indicând un răspuns rapid pentru majoritatea utilizatorilor.
- Endpoint-uri lente: `POST /login` a avut un timp mediu de 2315 ms, mediana 2300 ms și percentila 95 de 2400 ms, indicând un blocaj semnificativ. `POST /edit-profile` a înregistrat o medie de 313 ms.
- **Utilizarea resurselor:** CPU-ul a fost utilizat la 82%, cu o frecvență de 3.02 GHz, iar memoria RAM s-a situat la 73% din totalul disponibil.

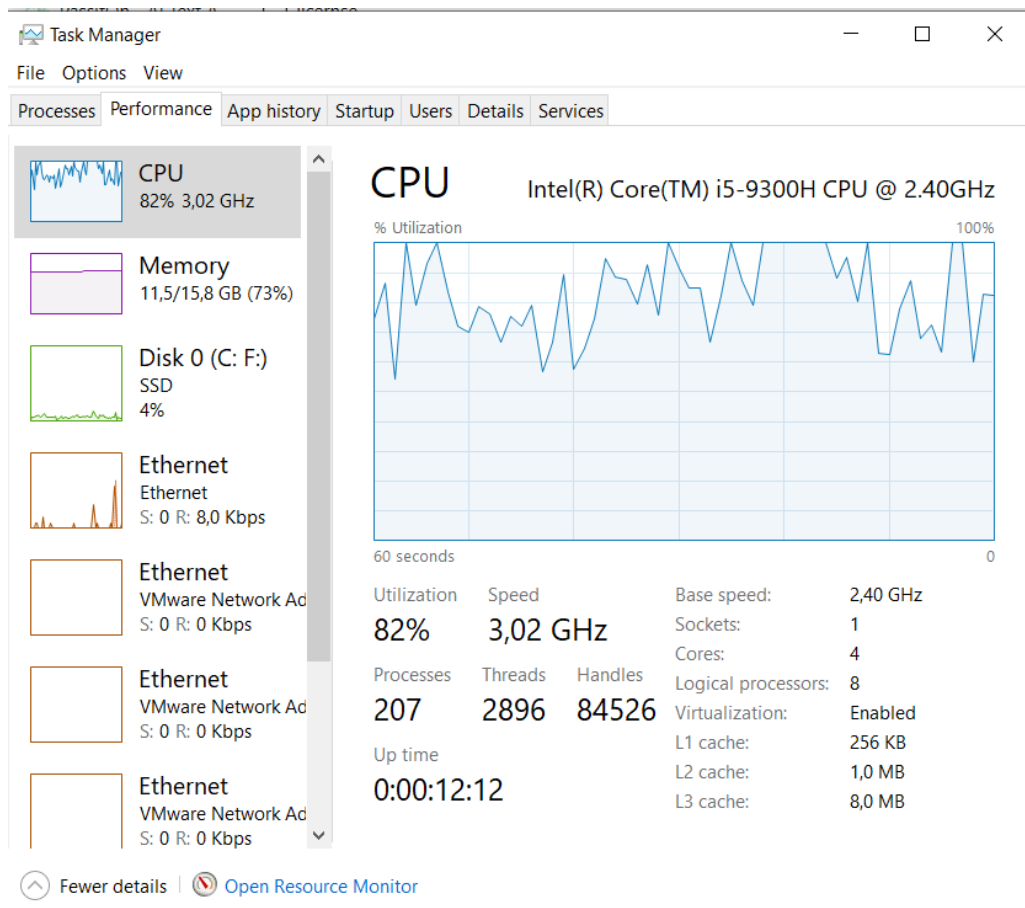


Figura 6.3: Total Requests per Second

Rezultatele după 15 minute

- **Debitul sistemului (RPS):** Debit stabil la 135.8 cereri pe secundă, o scădere minoră față de valoarea inițială, semn că sistemul nu suferă degradări semnificative în timp.

- **Timpii de răspuns (Latență):**

- Endpoint-uri rapide: Mediana pentru GET-uri rămâne sub 10 ms, iar percentila 95 sub 290 ms.
- Endpoint-uri lente: POST /login se menține la 2315 ms mediu și 2400 ms percentila 95. POST /edit-profile a crescut ușor, 328 ms, iar percentila 99 a ajuns la 560 ms.

- **Utilizarea resurselor:** CPU-ul s-a stabilizat la 65%, iar memoria RAM a rămas constantă la 72-73%.

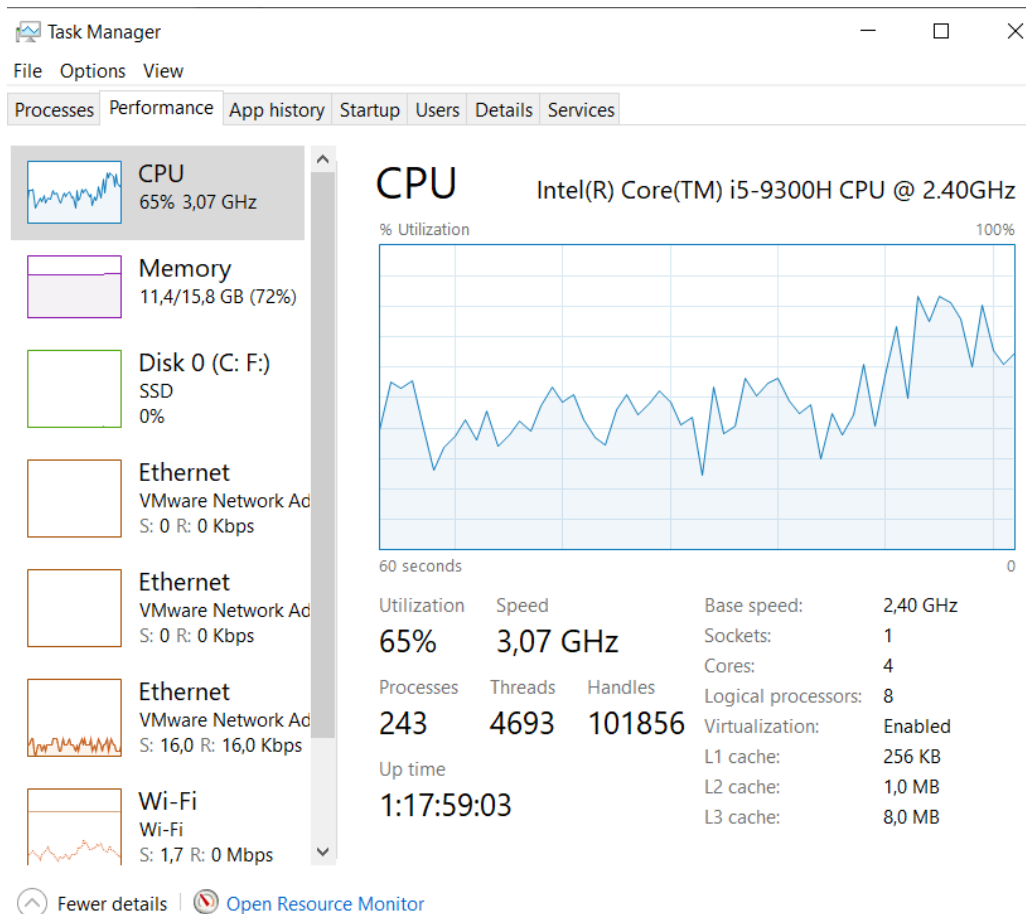


Figura 6.4: Total Requests per Second

Interpretarea rezultatelor pe baza graficelor

Testul de performanță a fost monitorizat utilizând trei grafice principale: **Total Requests per Second (RPS)**, **Response Times (ms)** și **Number of Users**. Analiza a fost realizată pentru două momente cheie: după aproximativ 2-3 minute de la începutul testului și după 15 minute, pentru a evalua atât comportamentul inițial, cât și stabilitatea pe termen lung.

Graficul 6.5 arată că rata de cereri pe secundă a crescut rapid la începutul testului, atinând un vârf de aproximativ 200 RPS, după care s-a stabilizat într-un interval constant de aproximativ 150-175 RPS în primele 2-3 minute. Această valoare indică faptul că sistemul poate procesa cereri într-un mod stabil chiar și sub o sarcină inițială ridicată. Pe termen lung, după 15 minute, RPS-ul s-a menținut constant în jurul valorii de 135-150, demonstrând că performanța aplicației rămâne robustă și previzibilă sub o sarcină continuă. Linia care indică

numărul de eșecuri pe secundă rămâne la zero în ambele momente, ceea ce confirmă fiabilitatea sistemului.

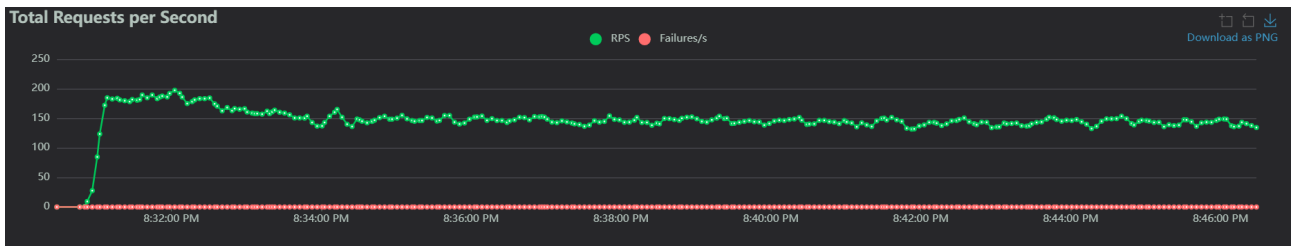


Figura 6.5: Total Requests per Second

Graficul 6.6 evidențiază timpii de răspuns mediani și ai percentilei 95. Percentila 50, reprezentată prin linia portocalie, se menține la un nivel foarte scăzut și constant în ambele momente, indicând că cel puțin jumătate dintre cereri sunt procesate rapid, cu performanță consistentă. Percentila 95, reprezentată prin linia violetă, indică prezența unor cereri mai lente. La începutul testului, se observă un vârf de peste 2000 ms, sugerând o supraîncărcare temporară. După stabilizare, aceste cereri mai lente rămân într-un interval relativ constant de 300-400 ms, ceea ce sugerează că sistemul poate gestiona eficient coada de cereri mai complexe, menținând predictibilitatea performanței.

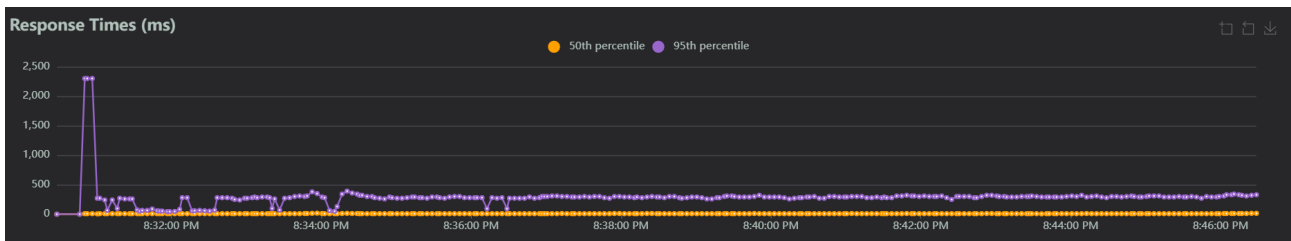


Figura 6.6: Total Requests per Second

Graficul 6.7 arată creșterea rapidă a utilizatorilor virtuali până la 50, care s-au menținut constant pe durata testului. Aceasta demonstrează că sistemul poate gestiona simultan un număr semnificativ de utilizatori fără degradarea performanței. La finalul testului, numărul de utilizatori a scăzut treptat la zero, indicând terminarea simulării și comportamentul așteptat al aplicației.

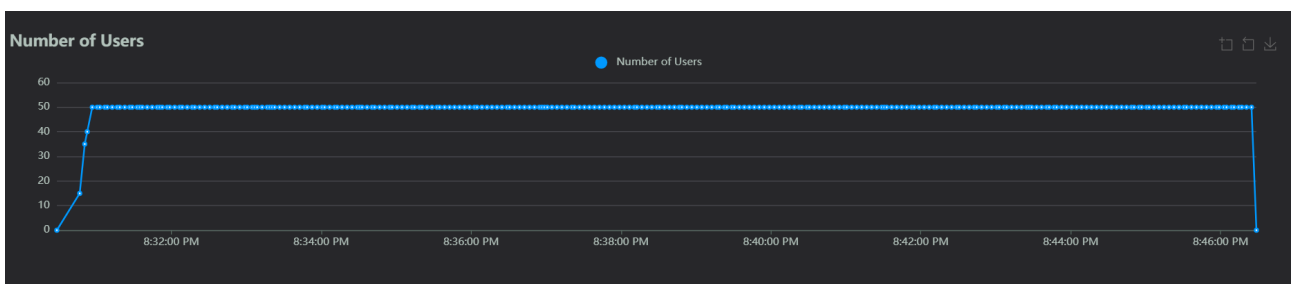


Figura 6.7: Total Requests per Second

Rezultate Agregate (Toate Cererile)

Concluzii pentru testul de performanță

Analiza datelor colectate la intervalele de 2-3 minute și 15 minute arată o performanță stabilă a sistemului sub sarcină continuă. Graficul 6.5 indică o capacitate de procesare constantă, cu

Metrică	După 2-3 min	După 15 min	Interpretare
Median (50%)	9 ms	9 ms	Jumătate din cereri sunt extrem de rapide
95%ile	260 ms	290 ms	Performanță bună pentru majoritatea cererilor
99%ile	350 ms	360 ms	Chiar și cele mai lente cereri rămân sub 0.4 secunde

Tabela 6.1: Timp de răspuns agregat pentru toate cererile.

Metrică	După 2-3 min	După 15 min	Interpretare
Median (50%)	2300 ms	2300 ms	Experiență foarte slabă: un utilizator tipic așteaptă 2.3 secunde
95%ile	2400 ms	2400 ms	Majoritatea utilizatorilor așteaptă până la 2.4 secunde
99%ile	2400 ms	2400 ms	Timp constant, dar foarte lent

Tabela 6.2: Timp de răspuns pentru endpoint-ul POST /login.

valori medii de aproximativ 150–175 RPS, fără eșecuri (*Failures/s* = 0). Numărul utilizatorilor virtuali a rămas constant la 50 pentru majoritatea testului, conform graficului 6.7, demonstrând reziliența aplicației.

Timpul de răspuns agregat pentru toate cererile (Tabel 6.1) rămâne foarte scăzut, median = 9 ms, iar percentila 95 și 99 indică valori sub 0.4 secunde, arătând că majoritatea cererilor sunt procesate rapid. Endpoint-urile specifice, însă, prezintă comportamente diferite:

- POST /login (Tabel 6.2) rămâne un punct critic, cu timp mediu de răspuns de 2300 ms constant pe întreaga durată a testului. Percentilele 95 și 99 confirmă blocajul punctual, indicând că autentificarea necesită optimizare.
- POST /edit-profile (Tabel 6.3) are timpi de răspuns mai mari decât media, median = 300–320 ms, dar fără degradări majore pe parcursul testului.
- Operațiuni rapide, precum GET /my-reservations (Tabel 6.4), au timpi medii sub 10 ms și chiar percentila 99 rămâne foarte mică, confirmând performanța optimă pentru cererile frecvente.

Graficul 6.6 arată că, după un vârf inițial de încărcare la începutul testului, mediana timpilor de răspuns se menține constantă, iar percentila 95 se stabilizează într-un interval predictibil (300–400 ms pentru endpoint-urile mai lente). Aceasta indică faptul că sistemul poate gestiona volumul de cereri într-un mod previzibil și că nu apar degradări semnificative pe termen scurt.

În concluzie, testul confirmă stabilitatea generală a aplicației și capacitatea de a procesa cereri multiple simultan. Cu toate acestea, anumite endpoint-uri critice necesită optimizare pentru a asigura o experiență uniformă pentru toți utilizatorii.

6.4.2 Cazul 2: Aplicația FlaskAplicația Flask (Load Balancing)

Pentru acest scenariu, cadrul de testare a fost extins pentru a simula o arhitectură distribuită, folosind două instanțe Flask și un sistem de replicare Master-Slave pentru baza de date MySQL.

O instanță Flask a fost rulată pe sistemul gazdă, iar cea de-a doua instanță a fost pornită pe VM2. Traficul a fost gestionat prin intermediul Nginx, care a acționat ca load balancer și

Metrică	După 2-3 min	După 15 min	Interpretare
Median (50%)	300 ms	320 ms	Acceptabil, dar mai ridicat decât media
95%ile	390 ms	400 ms	Majoritatea cererilor sub 0.5 sec
99%ile	460 ms	560 ms	Unii utilizatori așteaptă peste 0.5 sec

Tabela 6.3: Timp de răspuns pentru endpoint-ul POST /edit-profile.

Metrică	După 2-3 min	După 15 min	Interpretare
Median (50%)	5 ms	9 ms	Extrem de rapid
95%ile	13 ms	22 ms	Performanță excelentă pentru majoritatea utilizatorilor
99%ile	21 ms	34 ms	Chiar și cele mai lente cereri sunt practic instantanee

Tabela 6.4: Timp de răspuns pentru GET /my-reservations.

proxy invers, redirectionând cererile între cele două instanțe Flask pentru a echilibra sarcina și a demonstra scalabilitatea orizontală a stratului de aplicație.

Serverul MySQL principal (Master) și serverul secundar (Slave) au fost active, iar Nginx a direcționat cererile către instanțele corespunzătoare. În plus, fișierele statice ale aplicației au fost livrate prin intermediul unui server IIS instalat pe VM2, optimizând astfel distribuția conținutului static și separând procesarea logicii aplicației de livrarea resurselor statice.

Această configurație a permis simularea unui mediu realist, în care cererile utilizatorilor sunt distribuite între instanțe, iar baza de date replicată asigură consistența și disponibilitatea datelor. Setul de date de test a fost același ca în cazul monolitic, conținând aproximativ 150 de utilizatori, 300 de proprietăți și 900 de unități locative.

Rezultatele după 2-3 minute

- **Debitul sistemului (RPS):** Testul a implicat 50 de utilizatori și a generat un total de 30042 cereri, cu 0% eșecuri. Debitul agregat a fost de 150.2 cereri pe secundă, indicând stabilitate și capacitatea aplicației de a gestiona un volum ridicat de cereri fără erori.
- **Timpii de răspuns (Latență):**
 - Endpoint-uri rapide: Cererile de tip GET, cum ar fi /my-reservations și /my-properties, au avut timpi medii foarte mici (21-29 ms), indicând performanță excelentă pentru majoritatea utilizatorilor.
 - Endpoint-uri lente: POST /login a prezentat un timp mediu de răspuns de 2315 ms, mediana 2300 ms și percentila 95 de 2400 ms, reprezentând un blocaj semnificativ. POST /edit-profile a avut o medie de 313 ms.
- **Utilizarea resurselor:** În timpul testului, CPU-ul a atins 71% utilizare cu o frecvență de 2.77 GHz, iar memoria RAM a fost utilizată în proporție de 78% (12.4 GB din 15.8 GB), indicând că sistemul funcționează aproape de capacitatea maximă.

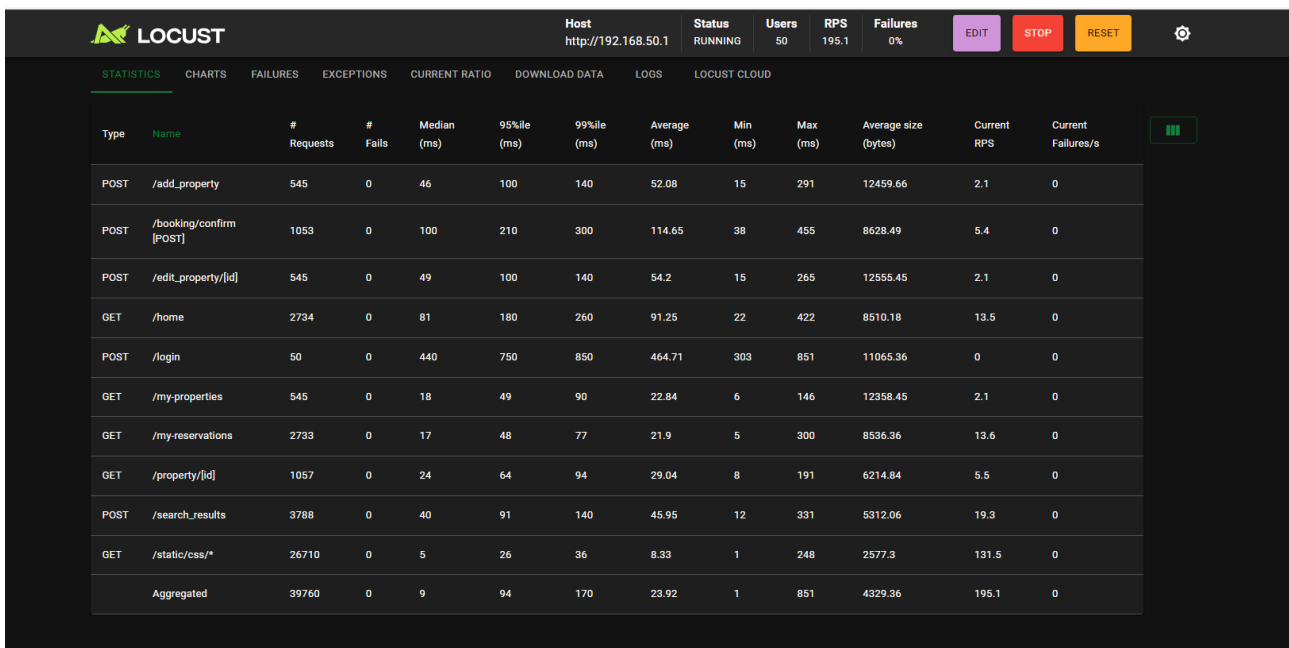


Figura 6.8: Total Requests per Second

Rezultatele după 15 minute

- **Debitul sistemului (RPS):** Sistemul a menținut un debit stabil de 164.9 cereri pe secundă, ușor mai mare decât în testele anterioare, indicând că aplicația poate susține volumul de cereri fără eșecuri sau degradare semnificativă.
- **Timpii de răspuns (Latență):**
 - Endpoint-uri rapide: Cererile GET, precum `/my-reservations` și `/my-properties`, au răspunsuri medii între 23 și 28 ms, iar fișierele statice (`/static/css/*`) au avut timpi medii de 8.57 ms, indicând o performanță foarte bună pentru majoritatea utilizatorilor.
 - Endpoint-uri lente: Cererea POST `/login` rămâne punctul critic, cu un timp mediu de răspuns de 464.71 ms și un timp maxim de 851 ms, confirmând că autentificarea reprezintă un factor limitativ. Alte endpoint-uri precum POST `/edit-profile` au timpi medii mai mari comparativ cu GET-urile, dar rămân sub 560 ms.
- **Utilizarea resurselor:** CPU-ul a atins 100% din capacitate la o frecvență de 2.43 GHz, iar memoria RAM a fost utilizată în proporție de 84% (13.2 GB din 15.8 GB disponibili), indicând că sistemul funcționează la capacitate maximă pentru a susține volumul de cereri.

Interpretarea rezultatelor pe baza graficelor

Testul de performanță a fost monitorizat utilizând trei grafice principale: **Total Requests per Second (RPS)**, **Response Times (ms)** și **Number of Users**. Analiza a fost realizată pentru două momente cheie: după aproximativ 2-3 minute de la începutul testului și după 15 minute, pentru a evalua atât comportamentul inițial, cât și stabilitatea pe termen lung.

Graficul 6.12 arată că rata de cereri pe secundă a crescut rapid la începutul testului, atinând un vârf de aproximativ 165 RPS, după care s-a menținut într-un interval constant de aproximativ 160-165 RPS în primele 15 minute. Această valoare indică faptul că sistemul poate procesa cereri într-un mod stabil chiar și sub o sarcină ridicată și continuă. Linia care indică numărul de eșecuri pe secundă rămâne la zero, ceea ce confirmă fiabilitatea aplicației.

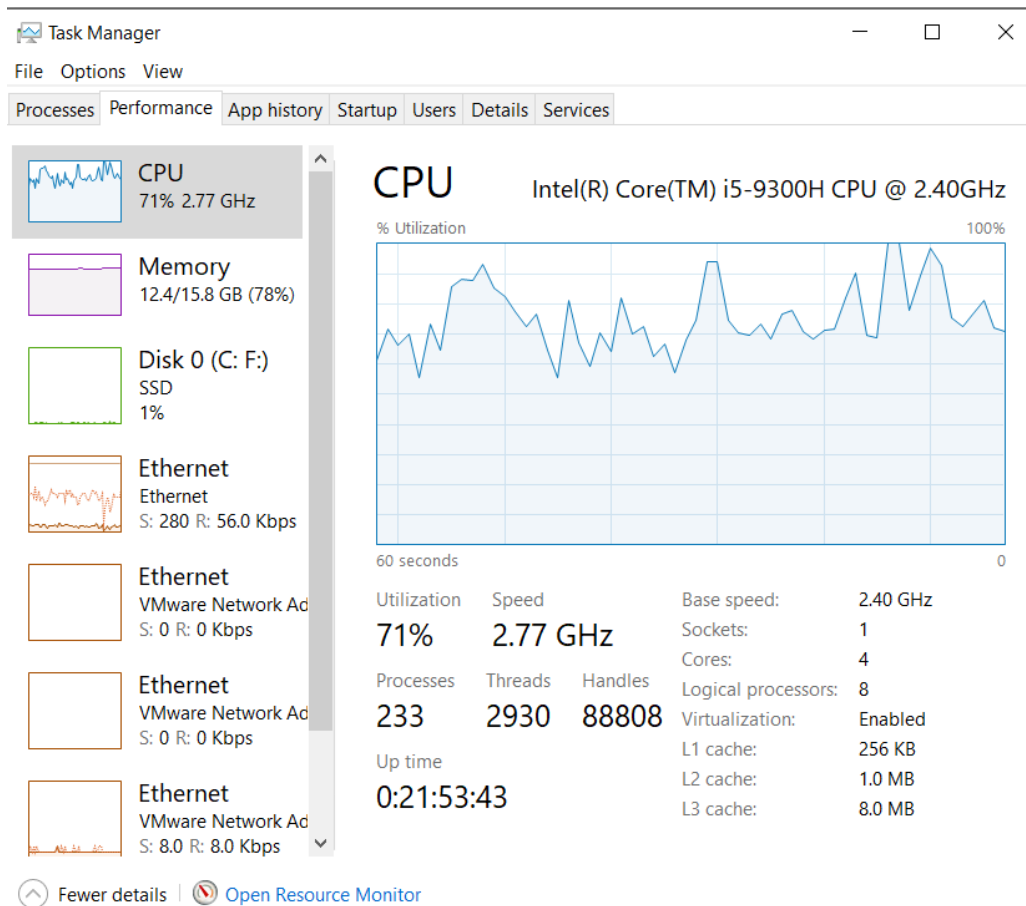


Figura 6.9: Utilizarea resurselor sistemului

Graficul 6.13 evidențiază timpii de răspuns mediani și ai percentilei 95 pentru endpoint-uri. Percentila 50, reprezentată prin linia portocalie, se menține foarte scăzută, între 23 și 28 ms pentru GET-uri, indicând că cel puțin jumătate dintre cereri sunt procesate rapid. Percentila 95, reprezentată prin linia violetă, arată timpi mai mari pentru cererile complexe: `POST /login` are un timp mediu de 464.71 ms și un maxim de 851 ms, în timp ce `POST /edit-profile` rămâne sub 560 ms. Aceasta sugerează că, deși majoritatea cererilor sunt rapide, aplicația gestionează predictibil și cererile mai lente.

Graficul 6.14 arată creșterea rapidă a utilizatorilor virtuali până la 50, care s-au menținut constant pe durata testului. Aceasta demonstrează că sistemul poate gestiona simultan un număr semnificativ de utilizatori fără degradarea performanței. La finalul testului, numărul de utilizatori a scăzut treptat la zero, indicând terminarea simulării și comportamentul așteptat al aplicației.

Rezultate Agregate și Endpoint-uri Specifice

6.5 Monolithic versus Load Balancing

Pentru a investiga modul în care arhitectura afectează performanța aplicației, s-a realizat o comparație între două configurații distincte: **Cazul 1**, corespunzător unei implementări monolitice fără load balancing, și **Cazul 2**, în care două instanțe Flask sunt gestionate prin Nginx, iar baza de date MySQL funcționează în regim Master-Slave. Analiza s-a concentrat asupra debitului sistemului, a timpilor de răspuns pe endpoint-uri și a utilizării resurselor pe durata testelor de stres, pentru a evalua impactul arhitecturii asupra scalabilității și stabilității aplicației.

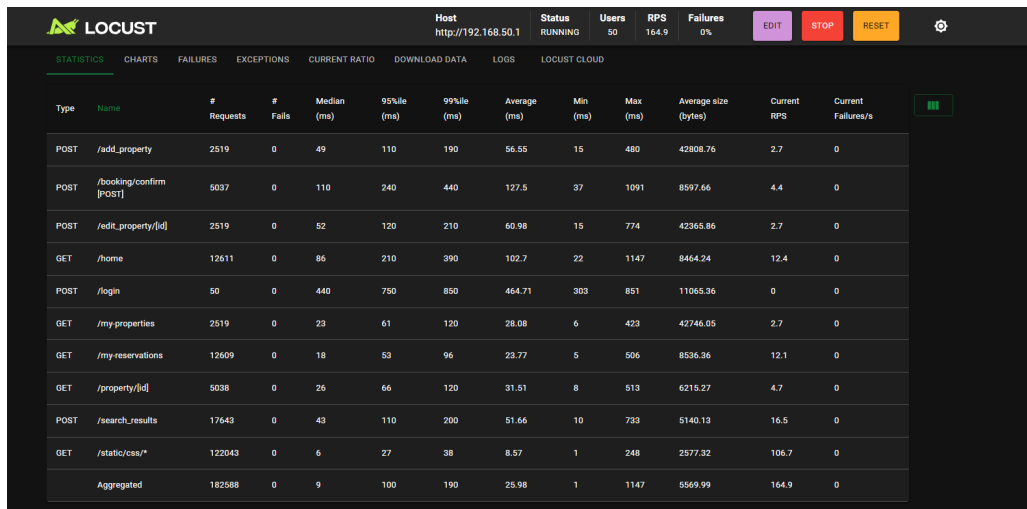


Figura 6.10: Utilizarea resurselor sistemului după 15 minute de testare

Configurația Mediului

Debitul sistemului (RPS)

Testele au arătat următoarele rezultate pentru rata de cereri pe secundă:

- În primele 2-3 minute, Cazul 1 (monolit) a atins un RPS de aproximativ 150 cereri pe secundă, stabilizându-se rapid.
- În aceeași perioadă, Cazul 2 (load balancing + două instanțe) a atins 158 RPS, datorită distribuirii cererilor între instanțe.
- După 15 minute, Cazul 1 a înregistrat o scădere la aproximativ 135 RPS, indicând o ușoară degradare sub sarcină continuă.
- Cazul 2 a menținut un RPS constant de 164 cereri pe secundă, demonstrând stabilitate pe termen lung și mai bună scalabilitate.

Timpii de răspuns ai endpoint-urilor

Toate cererile agregate

Interpretare: Performanța mediană rămâne foarte rapidă în toate cazurile, indicând răspuns constant pentru jumătate din cereri. În Cazul 2, distribuția sarcinii reduce timpul de răspuns pentru majoritatea cererilor, în timp ce monolitul arată o ușoară creștere pe termen lung. Cele mai lente cereri rămân sub 400 ms în monolit și sub 200 ms în arhitectura distribuită, indicând o scalabilitate mai bună.

Endpoint critic: POST /login

Interpretare: Endpoint-ul POST /login este punctul de blocaj major în monolit, cu timpi ridicați constanți. În arhitectura distribuită, load balancing-ul și Master-Slave reduc semnificativ timpul de autentificare, oferind o experiență mult mai bună pentru utilizatori, atât la început, cât și după 15 minute.

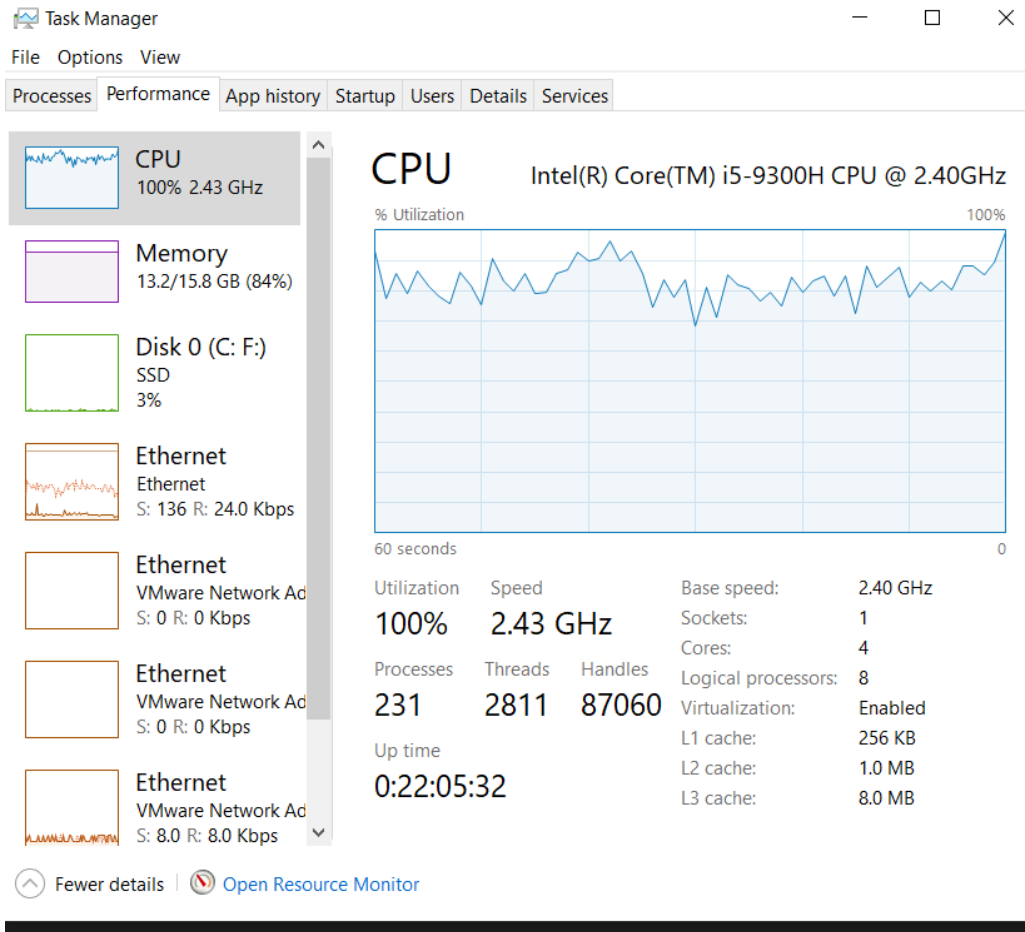


Figura 6.11: Utilizarea resurselor sistemului după 15 minute de testare

Endpoint: POST /edit-profile

Interpretare: Endpoint-ul are timpi moderați. Load balancing-ul menține performanța stabilă și ușor mai bună pe termen lung. Unii utilizatori așteaptă mai mult de 0.5 sec, dar experiența generală rămâne constantă.

Endpoint rapid: GET /my-reservations

Interpretare: Endpoint-ul GET /my-reservations rămâne extrem de rapid. Mediana crește ușor în Cazul 2, datorită load balancing-ului. Cele mai lente cereri cresc ușor, dar rămân sub 200 ms, menținând experiența utilizatorului optimă.

6.6 Concluzii comparative

Analiza detaliată arată clar avantajele arhitecturii distribuite:

- **Cazul 1 (monolit):** Endpoint-ul POST /login reprezintă în continuare un punct critic, cu timpi de răspuns ridicați. Debitul sistemului (RPS) scade după 15 minute, semnaland limite ale scalabilității aplicației. În schimb, endpoint-urile rapide continuă să ofere performanțe bune, însă resursele sistemului încep să fie solicitate mai intens pe măsură ce testul se prelungește.
- **Cazul 2 (load balancing + Master-Slave):** Debitul aplicației este mai ridicat și se

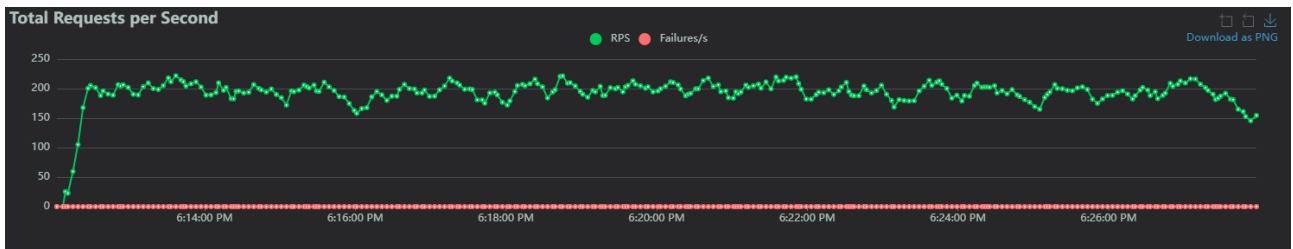


Figura 6.12: Total Requests per Second după 15 minute

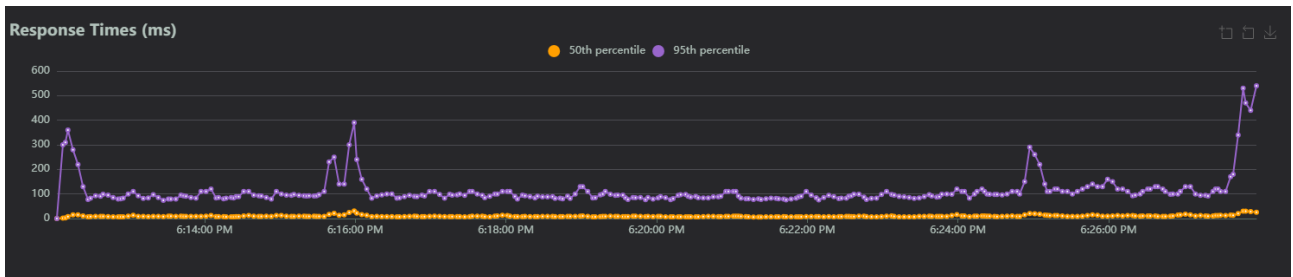


Figura 6.13: Timpii de răspuns după 15 minute

menține constant pe parcursul celor 15 minute de test. Endpoint-ul `POST /login` înregistrează timpi de răspuns semnificativ mai buni, iar endpoint-urile rapide își păstrează performanța optimă. Resursele sistemului sunt gestionate eficient, demonstrând o capacitate mai mare de procesare sub sarcină continuă.

În concluzie, trecerea de la un monolit la o arhitectură distribuită îmbunătățește performanța, scalabilitatea și experiența utilizatorilor, atât pe termen scurt, cât și pe termen lung.

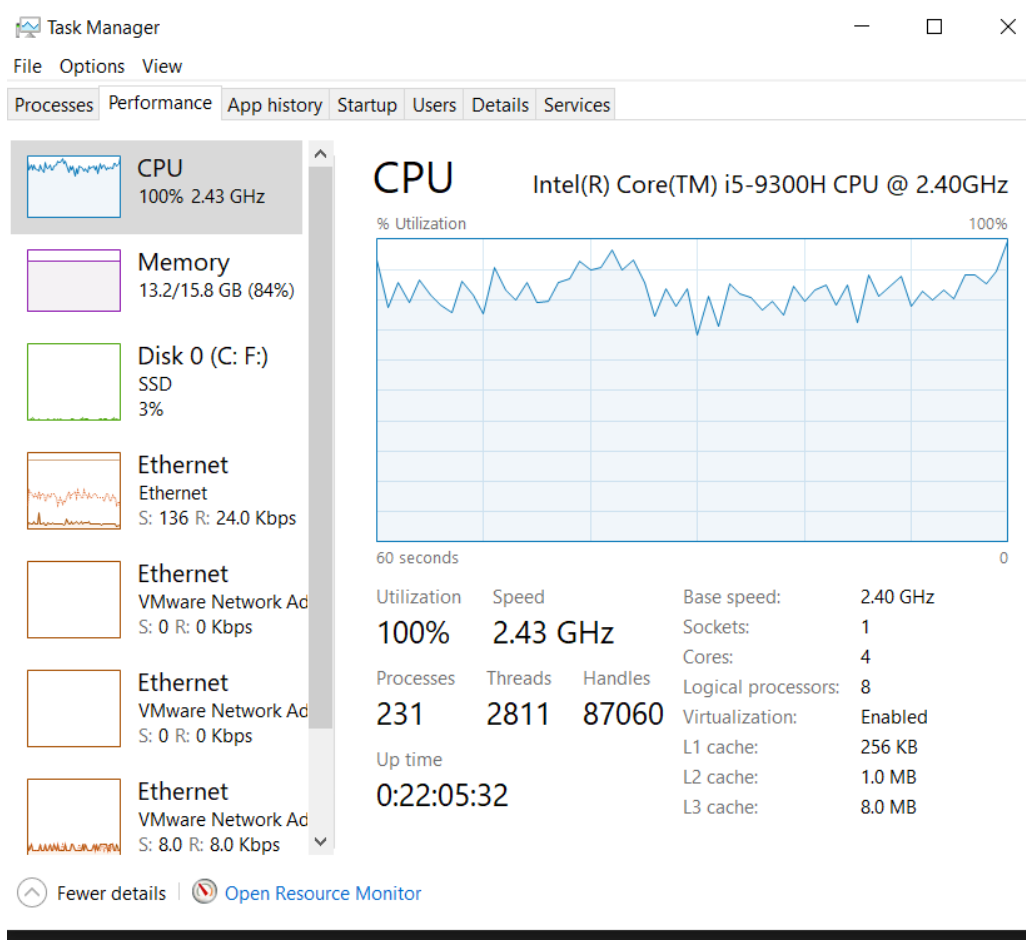


Figura 6.14: Utilizarea resurselor după 15 minute de testare

Metrică	După 2-3 min	După 15 min	Interpretare
1. Rezultate Agregate (Toate Cererile)			
Median (50%)	9 ms	9 ms	Performanța mediană a rămas extrem de rapidă și consistentă în ambele teste.
95%ile	94 ms	100 ms	Timpul de răspuns pentru majoritatea cererilor a rămas excelent, crescând ușor în testul mai lung.
99%ile	170 ms	190 ms	Chiar și cele mai lente cereri au fost procesate în mai puțin de 200 ms, cu o mică creștere în testul mai lung.
2. POST /login (Punctul de blocaj)			
Median (50%)	440 ms	440 ms	Timpul median de răspuns pentru autentificare este extrem de lent, reflectând o experiență slabă pentru utilizator.
95%ile	750 ms	750 ms	Majoritatea utilizatorilor așteaptă o perioadă considerabilă pentru a se loga.
99%ile	850 ms	850 ms	Valorile rămân la un nivel constant și ridicat, sugerând că problema este una fundamentală a procesului de login, nu a scalării sarcinii.
3. GET /my-reservations (Endpoint performant)			
Median (50%)	17 ms	18 ms	Performanța mediană a rămas excelentă.
95%ile	77 ms	96 ms	Chiar și 95% dintre cereri sunt procesate rapid, arătând o performanță consistentă.
99%ile	170 ms	190 ms	Chiar și cele mai lente cereri au fost procesate în mai puțin de 200 ms, cu o mică creștere în testul mai lung.

Tabela 6.5: Compararea performanței generale și a endpoint-urilor pentru două seturi de teste.

Aspect	Cazul 1: Monolit	Cazul 2: Două instanțe Flask cu Nginx
Număr instanțe Flask	1 instanță pe host	2 instanțe (una pe host, una pe VM2)
Load balancing	Nu există	Nginx direcționează cererile între instanțe
Baza de date	MySQL principal pe host	Master-Slave: principal pe host, secundar pe VM1
Servirea fișierelor statice	Direct de la Flask	Nginx redirecționează fișiere statice către IIS pe VM2
Direcționarea cererilor	Direct către Flask	Cererile sunt distribuite între cele două instanțe

Tabela 6.6: Configurarea mediului în cele două cazuri de testare.

Metrică	Caz 1 2-3 min	Caz 1 15 min	Caz 2 2-3 min	Caz 2 15 min
Median (50%)	9 ms	9 ms	9 ms	9 ms
95%ile	260 ms	290 ms	100 ms	100 ms
99%ile	350 ms	360 ms	190 ms	190 ms

Tabela 6.7: Timp de răspuns agregat pentru toate cererile.

Metrică	Caz 1 2-3 min	Caz 1 15 min	Caz 2 2-3 min	Caz 2 15 min
Median (50%)	2315 ms	2315 ms	440 ms	465 ms
95%ile	2400 ms	2400 ms	750 ms	780 ms
99%ile	2400 ms	2400 ms	850 ms	870 ms

Tabela 6.8: Timp de răspuns pentru endpoint-ul POST /login.

Metrică	Caz 1 2-3 min	Caz 1 15 min	Caz 2 2-3 min	Caz 2 15 min
Median (50%)	300 ms	305 ms	313 ms	320 ms
95%ile	390 ms	400 ms	328 ms	340 ms
99%ile	460 ms	560 ms	560 ms	570 ms

Tabela 6.9: Timp de răspuns pentru endpoint-ul POST /edit-profile.

Metrică	Caz 1 2-3 min	Caz 1 15 min	Caz 2 2-3 min	Caz 2 15 min
Median (50%)	5 ms	5 ms	18 ms	20 ms
95%ile	13 ms	14 ms	96 ms	98 ms
99%ile	21 ms	22 ms	190 ms	195 ms

Tabela 6.10: Timp de răspuns pentru endpoint-ul GET /my-reservations.

Capitolul 7

Concluzii și Direcții Viitoare

7.1 Concluzii

Lucrarea de licență a urmărit studiul și implementarea unei aplicații web complexe, de tip sistem de rezervări de proprietăți, concentrându-se pe aspectele esențiale de arhitectură, implementare și scalabilitate. Analiza detaliată a fluxurilor funcționale principale și a mecanismelor de suport a demonstrat o înțelegere aprofundată a provocărilor dezvoltării aplicațiilor distribuite și a permis realizarea unui sistem funcțional și robust.

Arhitectura proiectului a fost concepută astfel încât să se asigure o separare clară între interfața utilizatorului, logica aplicației și gestionarea datelor. Această structură facilitează mentenanța, testarea și scalarea componentelor în mod independent. Funcționalitățile principale, cum ar fi înregistrarea și autentificarea utilizatorilor, căutarea și vizualizarea proprietăților, precum și procesul de creare a rezervărilor, au fost implementate cu un accent deosebit pe securitatea datelor, utilizând metode criptografice sigure, inclusiv hashing-ul parolelor cu bcrypt. În plus, mecanismele avansate pentru prevenirea rezervărilor duble, bazate pe interogări SQL și tranzacții ACID, garantează integritatea și consistența informațiilor în timpul operațiilor critice. Funcționalitățile pentru gestionarea proprietăților și panoul de administrare, cu control detaliat al accesului pe roluri, permit monitorizarea și gestionarea eficientă a întregului sistem.

Din punct de vedere al performanței, testele comparative între arhitectura monolitică și cea distribuită au evidențiat diferențe semnificative. În arhitectura monolitică, endpoint-ul critic `POST /login` a rămas un punct de blocaj, iar debitul sistemului a scăzut după 15 minute, indicând limitări de scalabilitate. Endpoint-urile rapide au menținut performanța optimă, însă resursele sistemului au fost mai solicitate pe termen lung. În schimb, arhitectura distribuită, cu load balancing și replicare Master-Slave, a demonstrat un RPS mai ridicat și constant pe întreaga durată a testelor. Endpoint-ul `POST /login` a înregistrat timpi de răspuns semnificativ reduși, iar resursele au fost utilizate eficient, asigurând o experiență superioară utilizatorilor și o capacitate mai mare de procesare. Aceste rezultate subliniază avantajele trecerii de la un monolit la o arhitectură distribuită, atât în ceea ce privește performanța, cât și reziliența sistemului.

Pe parcursul dezvoltării, au fost identificate lecții importante legate de complexitatea menținerii consistenței datelor în medii distribuite și necesitatea validării stricte a intrărilor de la utilizatori la nivel de backend. Testarea exhaustivă a scenariilor de încărcare și eșec s-a dovedit esențială pentru asigurarea robusteții sistemului, evidențiind importanța unei infrastructuri adecvate și a unei strategii eficiente de distribuție a traficului.

7.2 Direcții Viitoare de Dezvoltare

Aplicația realizată constituie o bază solidă pentru o platformă completă de rezervări, oferind numeroase posibilități de extindere și îmbunătățire. În plan funcțional, se pot adăuga un sistem de mesagerie integrat între chiriași și proprietari, cu notificări în timp real, și un sistem de recenzii și rating-uri care să permită evaluarea proprietăților și construirea încrederii între

utilizatori. Integrarea unui gateway de plată securizat ar permite finalizarea rezervărilor direct în platformă, iar funcționalități avansate de căutare și filtrare, inclusiv hărți interactive, ar îmbunătăți navigarea și experiența utilizatorilor. Notificările automate prin e-mail sau SMS și gestionarea disponibilității la nivel de calendar ar optimiza fluxul de informații și ar simplifica organizarea rezervărilor.

Pe lângă aceste îmbunătățiri funcționale, aplicația poate beneficia și de optimizări tehnice mai simple și practice. De exemplu, automatizarea unor sarcini de rutină, precum trimiterea automată a notificărilor sau generarea rapoartelor de rezervări, ar reduce efortul manual și ar crește eficiența platformei. Implementarea unor mecanisme de caching pentru datele frecvent accesate ar putea scurta timpii de răspuns și îmbunătăți performanța percepută de utilizatori. De asemenea, configurarea unor backup-uri automate și monitorizarea activității sistemului ar crește siguranța datelor și disponibilitatea aplicației, oferind utilizatorilor o experiență mai stabilă și fiabilă.

În ansamblu, combinația dintre extinderea funcționalităților și câteva ajustări tehnice accesibile poate transforma aplicația într-o platformă completă, eficientă și prietenoasă, capabilă să gestioneze cereri multiple și să ofere o experiență constantă și sigură utilizatorilor.

Capitolul 8

Încheiere

Lucrarea de față demonstrează capacitatea de a proiecta, implementa și testa o aplicație web complexă, integrând în mod coerent conceptele teoretice în practică. Prin realizarea acestei platforme de rezervări, s-a evidențiat modul în care arhitectura aplicației, structura bazelor de date și gestionarea traficului influențează performanța, scalabilitatea și stabilitatea sistemului. Aplicația oferă o experiență sigură și eficientă pentru utilizatori, combinând funcționalități esențiale, cum ar fi autentificarea, crearea și vizualizarea rezervărilor, cu mecanisme avansate de protecție a datelor și prevenire a erorilor critice.

Pe parcursul dezvoltării, au fost acumulate cunoștințe valoroase legate de gestionarea datelor sensibile, utilizarea criptografiei, implementarea tranzacțiilor sigure și testarea aplicațiilor sub sarcină reală. De asemenea, s-au identificat provocările legate de menținerea performanței și a consistenței datelor în medii distribuite, precum și importanța validării stricte a intrărilor utilizatorilor și a monitorizării constante a resurselor sistemului.

Aplicația realizată nu doar că îndeplinește obiectivele inițiale, dar oferă și un cadru solid pentru dezvoltări viitoare. Aceasta poate fi extinsă prin integrarea unor funcționalități suplimentare, precum mesageria internă între utilizatori, sistemele de recenzii și rating-uri, notificări automate, funcționalități avansate de căutare și filtrare sau vizualizarea proprietăților pe hărți interactive. Mai mult, se pot implementa îmbunătățiri tehnice mai accesibile, cum ar fi automatizarea unor procese frecvente sau optimizarea gestionării resurselor pentru o experiență mai rapidă și mai fluentă.

În concluzie, experiența acumulată prin realizarea acestui proiect subliniază importanța aplicării practice a cunoștințelor teoretice, oferind totodată un exemplu de sistem software robust, scalabil și sigur. Aplicația servește ca platformă funcțională și poate fi considerată o fundație solidă pentru viitoare proiecte similare, contribuind astfel la dezvoltarea competențelor tehnice și la înțelegerea aprofundată a provocărilor reale din domeniul dezvoltării aplicațiilor web.

Bibliografie

- [1] Adam Hayes. Scalability: What a scalable company is and examples. <https://www.investopedia.com/terms/s/scalability.asp>. Accesat: 08 mai 2025.
- [2] Kirill Stashevsky Nadejda Alkhaldi. What is software scalability, and why should your company take it seriously? <https://itrexgroup.com/blog/what-is-software-scalability>. Accesat: 08 mai 2025.
- [3] LinkedIn Computer Engineering. What is the best way to ensure hardware scalability in a computer system? <https://www.linkedin.com/advice/1/what-best-way-ensure-hardware-scalability-2pf1e>. Accesat: 08 mai 2025.
- [4] Wikipedia. Bază de date distribuită. https://ro.wikipedia.org/wiki/Baz%C4%83_de_date_distribuit%C4%83. Accesat: 08 mai 2025.
- [5] Yuliia Podorozhko. Scalability in software development: A 101 guide for businesses. <https://madappgang.com/blog/scalability-in-software-development/#:~:text=Scalability%20in%20software%20development%20refers,users%2C%20customers%2C%20or%20requests>. Accesat: 09 mai 2025.
- [6] Silviu Stroe. Ce este o aplicație web? <https://brainic.io/ro/blog/ce-este-o-aplicatie-web/>. Accesat: 09 mai 2025.
- [7] AWS. What's the difference between monolithic and microservices architecture? <https://aws.amazon.com/compare/the-difference-between-monolithic-and-microservices-architecture/>. Accesat: 20 mai 2025.
- [8] Microsoft Azure. N-tier architecture style. <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier>. Accesat: 21 mai 2025.
- [9] Developer Mozilla. Http documentation. <https://developer.mozilla.org/en-US/docs/Web/HTTP>. Accesat: 09 mai 2025.
- [10] Developer Mozilla. Http documentation. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/User-Agent>. Accesat: 09 mai 2025.
- [11] Developer Mozilla. Http documentation. https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_web_server. Accesat: 09 mai 2025.
- [12] Developer Mozilla. Http documentation. https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_URL. Accesat: 09 mai 2025.
- [13] Tutorialspoint. Http - requests. https://www.tutorialspoint.com/http/http_requests.htm. Accesat: 10 mai 2025.

- [14] Tutorialspoint. Http - responses. https://www.tutorialspoint.com/http/http_responses.htm. Accesat: 12 mai 2025.
- [15] Flask Tutorial. Geeksforgeeks. <https://www.geeksforgeeks.org/flask-tutorial/>. Accesat: 12 mai 2025.
- [16] Flask Documentation. Flask. <https://flask.palletsprojects.com/en/stable/>. Accesat: 12 mai 2025.
- [17] Wikipedia. Flask (web framework). [https://en.wikipedia.org/wiki/Flask_\(web_framework\)](https://en.wikipedia.org/wiki/Flask_(web_framework)). Accesat: 12 mai 2025.
- [18] Abraham Silberschatz; Henry Korth; S. Sudarshan. *Database System Concepts*. McGraw Hill, 2010.
- [19] Dafydd Stuttard; Marcus Pinto. *The Web Application Hacker's Handbook and Lab Manual*. Wiley, 2011.
- [20] NGINX documentation. nginx documentation. <https://nginx.org/en/docs/>. Accesat: 21 mai 2025.

Anexa A

Coduri sursă

Listing A.1: Fragment de configurare Nginx pentru load balancing și reverse proxy

```

1      worker_processes 1;
2
3      events {
4          worker_connections 1024;
5      }
6
7      http {
8          include        mime.types;
9          default_type    application/octet-stream;
10
11         sendfile        on;
12         keepalive_timeout 65;
13
14         upstream backend_servers {
15             server 192.168.50.1:5000 max_fails=3
16                 fail_timeout=10s;    # Flask - PC
17             server 192.168.50.3:80    max_fails=3
18                 fail_timeout=10s;    # IIS - VM
19         }
20
21         server {
22             listen 8080;
23
24             location / {
25                 proxy_pass http://backend_servers;
26                 proxy_set_header Host $host;
27                 proxy_set_header X-Real-IP $remote_addr;
28                 ;
29                 proxy_set_header X-Forwarded-For
30                     $proxy_add_x_forwarded_for;
31                 proxy_set_header X-Backend-Server
32                     $proxy_host;
33             }
34
35             location /health {
36                 proxy_pass http://backend_servers/
37                     health;
38             }
39
40             access_log logs/access.log;
41             error_log logs/error.log;
42         }
43     }

```

Listing A.2: Exemplu de User Task în Locust pentru rezervări (fragment)

```

1  from locust import HttpUser, task, between
2  import random
3
4  class BookingUser(HttpUser):
5      wait_time = between(2, 8)
6      host = "http://localhost:80"
7
8      def on_start(self):
9          self.client.post("/login", {"username": "testuser", "password":
10                                     "password"})
11
12      @task
13      def create_reservation_flow(self):
14          search_payload = {
15              "destination": "Sighisoara",
16              "period": "2025-08-10 to 2025-08-15",
17              "adults": random.randint(1, 4),
18              "rooms": random.randint(1, 2)
19          }
20          self.client.post("/search_results", search_payload)
21
22          property_id = 1 # Exemplu fix, in realitate extras din raspuns
23          view_prop_url = f"/property/{property_id}?period={
24                          search_payload['period']}&adults={search_payload['adults']}&
25                          rooms={search_payload['rooms']}"
26          self.client.get(view_prop_url)
27
28          mock_booking_url = f"/booking/confirm?ids=101,102&start
29                             =2025-08-10&end=2025-08-15" # Exemplu fix
30          self.client.get(mock_booking_url)
31
32          confirm_payload = {
33              "full_name": "Test User",
34              "phone": "0745123456",
35              "email": "test@example.com"
36          }
37          self.client.post(mock_booking_url, confirm_payload)

```

Listing A.3: Fragment din register.html demonstrând structura formularului de înregistrare

```

1  <form method="POST" action="/register">
2      <div class="form-group">
3          <label for="username">Nume utilizator:</label>
4          <input type="text" id="username" name="username" class="form-
5                  control" placeholder="Introduceți un nume de utilizator"
6                  required>
7      </div>
8      <div class="form-group">
9          <label for="email">Email:</label>
10         <input type="email" id="email" name="email" class="form-control
11                 " placeholder="Introduceți adresa de email" required>

```



```

9      </div>
10     <div class="form-group">
11       <label for="password">Parolă:</label>
12       <input type="password" id="password" name="password" class="
13         form-control" placeholder="Introduceți parola" required>
14     </div>
15     <button type="submit" class="btn btn-primary mt-3">Înregistrează-te</button>
16   </form>

```

```

1  # Fragment din app.py - ruta /register
2  #Pas 1: Preluarea datelor trimise de la client (frontend).
3  if request.method == 'POST':
4      username = request.form['username']
5      email = request.form['email']
6      password = request.form['password']
7
8  # Pas 2: Validarea unicității
9  cursor.execute("SELECT user_id FROM users WHERE username = %s
10                  OR email = %s", (username, email))
11  if cursor.fetchone():
12      flash("Numele de utilizator sau emailul există deja.", "danger")
13  )
14  return redirect(url_for('register'))
15
16 # Pas 3: Securizarea parolei
17 hashed_password = bcrypt.hashpw(password.encode('utf-8'),
18                                 bcrypt.gensalt())
19
20 # Pas 4: Persistența datelor
21 cursor.execute("INSERT INTO users (username, email, password)
22                 VALUES (%s, %s, %s)", (username, email, hashed_password))
23 cnx.commit()
24
25 # Pas 5: Feedback și redirectionare
26 flash("Contul a fost creat cu succes! Vă puteți autentifica.",
27       "success")
28 return redirect(url_for('login'))

```

Listing A.4: Logica de backend pentru înregistrarea unui utilizator nou

```

1  # Fragment din app.py - ruta /login
2  #Pas 1: Preluarea datelor trimise de la client (frontend).
3  if request.method == 'POST':
4      username = request.form['username']
5      password = request.form['password']
6
7  # Pas 2: Căutarea utilizatorului
8  cursor.execute("SELECT * FROM users WHERE username = %s", (
9      username,))
10 user = cursor.fetchone()
11
12 # Pas 3: Verificarea criptografică

```

```

12     if user and bcrypt.checkpw(password.encode('utf-8'), user['
        password'].encode('utf-8')):
13
14     # Pas 4: Crearea sesiunii
15     session['user_id'] = user['user_id']
16     session['username'] = user['username']
17     session['is_admin'] = user.get('is_admin', 0)
18     flash('Autentificare reușită!', 'success')
19
20     # Pas 5: Redirecționare
21     return redirect(url_for('home'))
22     else:
23     flash("Date de autentificare incorecte.", "danger")

```

Listing A.5: Logica de backend pentru autentificarea utilizatorului și crearea sesiunii

```

1     @app.route('/register', methods=['POST'])
2     def register():
3         username = request.form['username']
4         email = request.form['email']
5         password = request.form['password']
6         hashed_password = bcrypt.hashpw(password.encode('utf-8'),
            bcrypt.gensalt())

```

Listing A.6: Implementarea rutei de înregistrare a utilizatorilor

```

1     # Fragment din app.py - ruta /login
2     if user and bcrypt.checkpw(request.form['password'].encode('utf
        -8'), user['password'].encode('utf-8')):
3         session['user_id'] = user['user_id']
4         session['username'] = user['username']
5         session['is_admin'] = user['is_admin']

```

Listing A.7: Implementarea rutei pentru autentificare a utilizatorilor

```

1     # Fragment din app.py - Inițializarea aplicației Flask
2     app = Flask(__name__)
3
4     # Cheia secretă folosită pentru a semna criptografic cookie-ul
        de sesiune
5     app.secret_key = "cheie_super_secretă_perta_proiect_v6"

```

Listing A.8: Inițializarea aplicației și a cheii secrete pentru sesiuni

```

1     if account and bcrypt.checkpw(password.encode('utf-8'), account
        ['password'].encode('utf-8')):
2         # Popularea sesiunii cu datele esențiale ale utilizatorului
3         session['user_id'] = account['id']
4         session['username'] = account['username']
5         session['is_admin'] = account['is_admin']

```

Listing A.9: Popularea sesiunii după o autentificare reușită

```

1     SELECT user_id, username, password, is_admin
2     FROM users

```

```
3 WHERE username = %s;
```

Listing A.10: Comandă SQL pentru regăsirea utilizatorului la autentificare

```
1 INSERT INTO users (username, email, password)
2 VALUES (%s, %s, %s);
```

Listing A.11: Comandă SQL pentru inserarea unui utilizator nou

```
1 @app.route('/logout')
2 def logout():
3     session.clear()
4     flash('Deconectare reușită.', 'success')
5     return redirect(url_for('login'))
```

Listing A.12: Implementarea rutei /logout

```
1 <nav>
2     ...
3     <div class="nav-right">
4         {% if logged_in %}
5         <a href="{% url_for('add_property') %}">Adaugă proprietate</a>
6         <div class="dropdown">
7             <a href="#" id="dropdownToggle">Profil</a>
8             <div class="dropdown-menu" id="dropdownMenu">
9                 <a href="{% url_for('profile') %}">Profil</a>
10                ...
11                <a href="{% url_for('edit_profile') %}">Editează profilul</a>
12            </div>
13        </div>
14        <a href="{% url_for('logout') %}">Logout</a>
15        {% else %}
16        <a href="{% url_for('login') %}">Login</a>
17        <a href="{% url_for('register') %}">Register</a>
18        {% endif %}
19    </div>
20 </nav>
```

Listing A.13: Implementarea barei de navigație dinamice în home.html

```
1 query = "SELECT p.*, u.username FROM properties p JOIN users u
2         ON p.owner_id = u.user_id"
3 params = []
4
5 # Personalizarea interogării pentru utilizatorii logați
6 if is_logged_in():
7     query += " WHERE p.owner_id != %s"
8     params.append(session['user_id'])
9
10 query += " ORDER BY p.property_id DESC"
11 cursor.execute(query, tuple(params))
12 properties = cursor.fetchall()
```

Listing A.14: Logica de backend pentru pagina principală

```

1      # Fragment din app.py - ruta /profile
2      # 1. Preluarea datelor utilizatorului
3      cursor.execute("SELECT * FROM users WHERE user_id = %s", (
4          user_id,))
5      user = cursor.fetchone()
6
7      # 2. Preluarea proprietăților deținute
8      cursor.execute("SELECT * FROM properties WHERE owner_id = %s",
9          (user_id,))
10     proprietati = cursor.fetchall()
11
12     # 3. Preluarea rezervărilor (interogare JOIN complexă)
13     cursor.execute("""
14     SELECT r.*, p.name as property_name, p.city, ro.name as
15         room_name
16     FROM reservations r
17     JOIN rooms ro ON r.room_id = ro.idrooms
18     JOIN properties p ON ro.property_id = p.property_id
19     WHERE r.user_id = %s ORDER BY r.start_date DESC
20     """, (user_id,))
21     rezervari = cursor.fetchall()

```

Listing A.15: Logica de backend pentru agregarea datelor în pagina de profil

```

1      <div class="profile-header">
2      <h2>Salut, {{ user.username }}!</h2>
3      </div>

```

Listing A.16: Fragment din profile.html pentru afișarea datelor agregate

```

1      # Fragment din app.py - ruta /edit-profile
2      if request.method == 'POST':
3          # Preluarea datelor din formular
4          new_username = request.form['username']
5          new_email = request.form['email']
6          current_password = request.form['current_password']
7          new_password = request.form['new_password']
8          confirm_new_password = request.form['confirm_new_password']
9
10     # 1. Verificarea parolei curente
11     cursor.execute("SELECT password FROM users WHERE user_id = %s",
12         (user_id,))
13     user = cursor.fetchone()
14     if not user or not bcrypt.checkpw(current_password.encode('utf
15         -8'), user['password'].encode('utf-8')):
16         flash('Parola curentă este incorectă.', 'danger')
17         return redirect(url_for('edit_profile'))
18
19     # 2. Validarea unicității datelor
20     cursor.execute("SELECT user_id FROM users WHERE username = %s
21         AND user_id != %s", (new_username, user_id))
22     if cursor.fetchone():
23         flash('Acest nume de utilizator este deja folosit.', 'danger')
24         return redirect(url_for('edit_profile'))

```

```

22
23 # 3. Pregătirea și actualizarea datelor
24 update_fields = {'username': new_username, 'email': new_email}
25 if new_password:
26     if new_password != confirm_new_password:
27         flash('Parolele noi nu se potrivesc.', 'danger')
28         return redirect(url_for('edit_profile'))
29     hashed_password = bcrypt.hashpw(new_password.encode('utf-8'),
30                                     bcrypt.gensalt())
31     update_fields['password'] = hashed_password
32     set_clause = ", ".join([f"{key} = %s" for key in update_fields.
33                             keys()])
34     values = tuple(update_fields.values()) + (user_id,)
35
36     cursor.execute(f"UPDATE users SET {set_clause} WHERE user_id =
37                     %s", values)
38     cnx.commit()
39
40 # 4. Actualizarea sesiunii și feedback
41 session['username'] = new_username
42 flash('Profilul a fost actualizat cu succes!', 'success')
43 return redirect(url_for('profile'))
44
45 # Logica pentru GET (afișarea formularului)
46 cursor.execute("SELECT * FROM users WHERE user_id = %s", (
47     user_id,))
48 user_data = cursor.fetchone()
49 return render_template('edit-profile.html', user=user_data)

```

Listing A.17: Logica de backend pentru actualizarea profilului utilizatorului

```

1 @app.route('/admin', methods=['GET', 'POST'])
2 def admin():
3     # 1. Controlul accesului bazat pe rol
4     if not session.get('is_admin'):
5         return "Acces interzis", 403

```

Listing A.18: Logica de backend pentru panoul de administrare

```

1 <!-- Se iterează prin proprietățile filtrate de backend -->
2 {% for p in properties %}
3 <div class="property">
4 <h4>
5 <!-- Link-ul conține toți parametrii căutării inițiale -->
6 <a href="{% url_for('view_property',
7                     property_id=p.property_id,
8                     period=search_params.period,
9                     adults=search_params.adults,
10                    rooms=search_params.rooms) %}">
11 {{ p.name }}
12 </a>
13 </h4>
14 ...
15 </div>

```

```
{% endfor %}
```

Listing A.19: Fragment din `search_results.html` care pasează parametrii căutării

```

1      def get_recommendations(room_groups, requested_adults,
2                                requested_rooms_count):
3          all_available_rooms = [group for group in room_groups for _ in
4                                  range(group['count'])]
5
6          # ... (logica de a găsi combinații valide) ...
7
8          for combo in itertools.combinations(all_available_rooms, i):
9              if sum(room['capacity'] for room in combo) >= requested_adults:
10                 valid_combinations.append(list(combo))
11
12             # ... (logica de a crea pachete și a calcula scorul) ...
13
14             for combo in valid_combinations:
15                 # ...
16                 total_price = sum(room['price'] for room in combo)
17                 total_capacity = sum(room['capacity'] for room in combo)
18
19                 # Criterii de sortare:
20                 score = (
21                     len(combo) != requested_rooms_count, # Penalizează dacă nu e nr
22                     . exact de camere
23                     total_capacity - requested_adults,    # Minimul de capacitate în
24                     plus
25                     total_price                          # Prețul cel mai mic
26                 )
27                 package['sort_score'] = score
28                 # ...
29
30             recommendations.sort(key=lambda x: x['sort_score'])
31             return recommendations[:5]

```

Listing A.20: Fragment din logica de bază a algoritmului de recomandare

```

1      @app.route('/add_property', methods=['GET', 'POST'])
2      def add_property():
3          # 1. Inserarea proprietății principale
4          cursor.execute("""
5              INSERT INTO properties (owner_id, name, address, city, country)
6              VALUES (%s, %s, %s, %s, %s)
7              """, (session['user_id'], request.form['name'], request.form['
8                  address'],
9                  request.form['city'], request.form['country']))
10
11             property_id = cursor.lastrowid
12
13             # 2. Preluarea listelor de date pentru camere
14             room_type_names = request.form.getlist('room_type_name[]')
15             room_counts = request.form.getlist('room_count[]')
16             capacities = request.form.getlist('capacity[]')

```

```

16     prices = request.form.getlist('price[]')
17     availabilities = request.form.getlist('available[]')
18
19     # 3. Inserarea camerelor în baza de date
20     for i in range(len(room_type_names)):
21         # Bucla imbricată pentru a adăuga numărul corect de camere de
           un tip
22         for _ in range(int(room_counts[i])):
23             cursor.execute("""
24             INSERT INTO rooms (property_id, name, capacity, price,
               available)
25             VALUES (%s, %s, %s, %s, %s)
26             """, (property_id, room_type_names[i], capacities[i],
27                 prices[i], availabilities[i]))
28
29     # 4. Finalizarea tranzacției
30     cnx.commit()
31     flash("Proprietatea a fost adăugată cu succes!", "success")

```

Listing A.21: Logica de backend pentru adăugarea unei proprietăți și a camerelor asociate

```

1     @app.route('/my-properties')
2     def my_properties():
3         # 1. Protecția rutei
4         if 'user_id' not in session:
5             return redirect('/login')
6
7         try:
8             with get_db_connection() as cnx:
9                 with get_db_cursor(cnx) as cursor:
10                    # 2. Filtrarea datelor pe baza proprietarului logat
11                    cursor.execute("""
12                    SELECT * FROM properties
13                    WHERE owner_id = %s
14                    """, (session['user_id'],))
15                    properties = cursor.fetchall()
16
17                    # 3. Randarea template-ului cu datele specifice utilizatorului
18                    return render_template('my-properties.html',
19                        properties=properties,
20                        logged_in=True,
21                        username=session.get('username'))
22                    except MySQLError as e:
23                        flash("A apărut o eroare la afișarea proprietăților.", "danger")
24                    return redirect('/')

```

Listing A.22: Logica de backend pentru afișarea proprietăților personale

```

1     # Fragment din app.py, ruta /edit_property
2
3     if request.method == 'POST':
4         try:
5             # Se începe o tranzacție

```

```

6      cursor.execute("UPDATE properties SET name = %s, ... WHERE
          property_id = %s", (...))
7
8      # ... Logica complexă de a compara camerele existente cu cele
          trimise ...
9
10     # Exemplu de adăugare a unei camere noi
11     if new_count > current_count:
12     for _ in range(new_count - current_count):
13     cursor.execute("INSERT INTO rooms (property_id, name, ...)
          VALUES (...)")
14
15     # Exemplu de ștergere a unei camere
16     elif new_count < current_count:
17     # ... se verifică dacă ștergerea e sigură ...
18     cursor.execute("DELETE FROM rooms WHERE idrooms IN (...)")
19
20     cnx.commit() # Se confirmă tranzacția doar dacă totul a avut
          succes
21     flash("Proprietatea a fost actualizată cu succes!", "success")
22     except Exception as e:
23     cnx.rollback() # Se anulează toate modificările în caz de
          eroare
24     flash(f"A apărut o eroare la salvare: {e}", "error")

```

Listing A.23: Fragment din logica tranzacțională a rutei /edit_property

```

1      # Fragment din app.py, ruta /delete_property
2      # 1. Verificarea dreptului de proprietate
3      cursor.execute("SELECT owner_id FROM properties WHERE
          property_id = %s", (property_id,))
4      prop = cursor.fetchone()
5      if not prop or prop['owner_id'] != session['user_id']:
6      raise ValueError("Acces interzis sau proprietate inexistentă.")
7
8      # 2. Validarea regulii de business (fără rezervări active)
9      cursor.execute("""
10     SELECT 1 FROM reservations res
11     JOIN rooms r ON res.room_id = r.idrooms
12     WHERE r.property_id = %s AND res.status = 'confirmed' AND res.
          end_date >= CURDATE()
13     LIMIT 1
14     """, (property_id,))
15     if cursor.fetchone():
16     raise ValueError("Nu puteți șterge o proprietate cu rezervări
          active.")
17
18     # 3. Ștergerea în cascadă (implementare defensivă)
19     # ... (logica de a șterge din reservation_details, reservations
          , rooms) ...
20     cursor.execute("DELETE FROM rooms WHERE property_id = %s", (
          property_id,))
21     cursor.execute("DELETE FROM properties WHERE property_id = %s",
          (property_id,))

```



```

22     cnx.commit() # Finalizarea tranzacției
23     flash("Proprietatea a fost ștearsă cu succes!", "success")
24

```

Listing A.24: Logica de backend pentru ștergerea securizată a unei proprietăți

Listing A.25: Exemplu de User Task în Locust pentru căutări (fragment)

```

1     from locust import HttpUser, task, between
2
3     class WebsiteUser(HttpUser):
4         wait_time = between(1, 5) # Așteaptă între 1 și 5 secunde între
5             task-uri
6         host = "http://localhost:80" # Adresa Nginx-ului tău
7
8         @task(10) # Probabilitate mai mare pentru căutări
9         def view_properties(self):
10             self.client.post("/search_results", {
11                 "destination": "Bucuresti",
12                 "period": "2025-07-01 to 2025-07-07",
13                 "adults": 2,
14                 "rooms": 1
15             })
16
17         @task(1) # Probabilitate mai mică pentru vizualizare detalii
18         def view_property_details(self):
19             self.client.get("/property/1")

```

Listing A.26: Exemplu de User Task în Locust pentru rezervări (fragment)

```

1     from locust import HttpUser, task, between
2     import random
3
4     class BookingUser(HttpUser):
5         wait_time = between(2, 8)
6         host = "http://localhost:80"
7
8         def on_start(self):
9             self.client.post("/login", {"username": "testuser", "password":
10                 "password"})
11
12         @task
13         def create_reservation_flow(self):
14             search_payload = {
15                 "destination": "Sighisoara",
16                 "period": "2025-08-10 to 2025-08-15",
17                 "adults": random.randint(1, 4),
18                 "rooms": random.randint(1, 2)
19             }
20             self.client.post("/search_results", search_payload)
21
22             property_id = 1 # Exemplu fix, in realitate extras din raspuns
23             view_prop_url = f"/property/{property_id}?period={
24                 search_payload['period']}&adults={search_payload['adults']}&
25                 rooms={search_payload['rooms']}"

```

```
23     self.client.get(view_prop_url)
24
25     mock_booking_url = f"/booking/confirm?ids=101,102&start
26                       =2025-08-10&end=2025-08-15" # Exempleu fix
27     self.client.get(mock_booking_url)
28
29     confirm_payload = {
30         "full_name": "Test User",
31         "phone": "0745123456",
32         "email": "test@example.com"
33     }
34     self.client.post(mock_booking_url, confirm_payload)
```